

BUKU AJAR

Pengenalan Ekosistem *Machine Learning* dan Eksplorasi Data (*Exploratory Data Analysis*)

Mata Kuliah Praktik: Machine Learning

Program Studi D4 Teknologi Rekayasa Informatika Industri

Politeknik Manufaktur Bandung (POLMAN Bandung)

Dosen Pengampu: Putra, S.T., M.T. (harry@ae.polman-bandung.ac.id)

Tahun Akademik 2026/2027 — Semester Ganjil

DAFTAR ISI

Bagian	Halaman
PENDAHULUAN BAB	4
PENYAJIAN MATERI	8
1. Ekosistem Perangkat Lunak untuk Data Science dan Machine Learning	8
2. Menyiapkan Lingkungan Kerja	9
3. Dasar-Dasar Python untuk Analisis Data	13
4. Pandas: DataFrame dan Series sebagai Struktur Data Utama	14
5. Matplotlib dan Seaborn: Pengantar Awal Visualisasi	15
6. Memuat dan Memahami Dataset	17
7. Inspeksi Data Mendalam	19
8. Penanganan Missing Value Sederhana	22
9. Visualisasi Data Dasar untuk Eksplorasi	25
10. Sintesis: Merangkai Alur Kerja EDA yang Utuh	29
LATIHAN MANDIRI TERBIMBING	31
STUDI KASUS MANDIRI	38
Studi Kasus Mandiri 1 – Eksplorasi Data Produksi Harian Pabrik	38
Studi Kasus Mandiri 2 – Eksplorasi Data Downtime Mesin Produksi	39
Studi Kasus Mandiri 3 – Eksplorasi Data Sensor Kualitas Produk pada Lini Perakitan Elektronik	41
STUDI KASUS KELOMPOK	44
Studi Kasus Kelompok 1 – Analisis Kualitas Udara di Lingkungan Kerja Industri	44
Studi Kasus Kelompok 2 – Analisis Data Logistik dan Ketepatan Waktu Pengiriman Komponen	47
RANGKUMAN	50
TES FORMATIF	52
A. Soal Pilihan Ganda	52
B. Soal Esai Analitis	57
KUNCI JAWABAN DAN PEMBAHASAN	60
C. Kunci Jawaban dan Pembahasan Soal Pilihan Ganda	60
D. Jawaban Model dan Pembahasan Soal Esai	65
UMPAN BALIK DAN TINDAK LANJUT	70
DAFTAR PUSTAKA	73

PENDAHULUAN BAB

Deskripsi Singkat Isi Bab

Bab ini merupakan bab pembuka dari rangkaian bahan ajar mata kuliah praktik Machine Learning yang diselenggarakan pada Program Studi D4 Teknologi Rekayasa Informatika Industri, Politeknik Manufaktur Bandung. Sebagai bab pertama dari delapan pertemuan, bab ini berfungsi sebagai jembatan yang menghubungkan pengetahuan umum mahasiswa tentang pemrograman dengan dunia analitik data dan kecerdasan buatan terapan. Materi yang disajikan mencakup dua pilar besar: pertama, pengenalan ekosistem perangkat lunak yang digunakan sepanjang perkuliahan, yaitu bahasa pemrograman Python beserta pustaka-pustaka inti seperti NumPy, pandas, Matplotlib, dan Seaborn, serta dua lingkungan kerja yang lazim dipakai, yakni instalasi lokal berbasis Anaconda dan lingkungan awan (*cloud*) Google Colaboratory; kedua, penguasaan keterampilan dasar Eksplorasi Data atau *Exploratory Data Analysis* (EDA), yaitu serangkaian teknik untuk memuat, memeriksa, membersihkan, dan memvisualisasikan data sebelum data tersebut digunakan untuk membangun model pembelajaran mesin pada pertemuan-pertemuan berikutnya.

Bab ini disusun sebagai bahan bacaan yang bersifat *self-instruction*, artinya mahasiswa diharapkan dapat mempelajarinya secara mandiri, baik sebagai persiapan sebelum sesi tatap muka di kelas, sebagai pendamping saat praktikum berlangsung, maupun sebagai bahan ulasan (*review*) setelah perkuliahan usai. Setiap konsep dijelaskan secara naratif dan bertahap, dimulai dari pengertian dasar, dilanjutkan dengan uraian mendalam mengenai cara kerja dan alasan penggunaannya, kemudian diperkuat dengan contoh kode yang dapat dijalankan langsung beserta penjelasan atas hasilnya, dan diakhiri dengan penjelasan mengenai keterkaitannya dengan konsep lain maupun dengan konteks industri manufaktur.

Relevansi dan Manfaat Materi bagi Mahasiswa

Pertanyaan yang wajar muncul di benak mahasiswa Teknologi Rekayasa Informatika Industri adalah: mengapa seorang calon sarjana terapan yang berorientasi pada dunia manufaktur perlu mempelajari Machine Learning, dan mengapa harus dimulai dari eksplorasi data yang tampak sederhana? Jawabannya terletak pada transformasi industri yang tengah berlangsung. Revolusi Industri 4.0 menempatkan data sebagai aset produksi yang setara pentingnya dengan mesin, tenaga kerja, dan bahan baku. Setiap lini produksi modern kini dilengkapi sensor yang merekam suhu, tekanan, getaran, kecepatan putaran mesin, hingga hasil inspeksi visual produk secara terus-menerus. Data tersebut tidak memiliki nilai apa pun apabila hanya disimpan tanpa dianalisis. Di sinilah kompetensi yang dibangun pada pertemuan ini menjadi fondasi yang tidak dapat dilewati.

Sebelum sebuah model prediktif — misalnya model yang memperkirakan kapan mesin akan mengalami kegagalan (*predictive maintenance*), model yang mendeteksi produk cacat dari citra kamera inspeksi (*quality control*), atau model yang meramalkan permintaan untuk mengoptimalkan rantai pasok (*supply chain optimization*) — dapat dibangun, seorang praktisi data harus terlebih dahulu memahami data mentah yang tersedia. Ia harus mengetahui berapa banyak baris dan kolom yang dimiliki data tersebut, jenis data apa yang

terkandung di setiap kolom, apakah ada nilai yang hilang (*missing value*) akibat sensor yang gagal membaca atau operator yang lupa mencatat, seberapa besar sebaran nilai suatu variabel, apakah terdapat nilai-nilai ekstrem yang mengindikasikan anomali proses, dan apakah ada hubungan antar-variabel yang bisa dimanfaatkan. Seluruh pertanyaan ini dijawab melalui proses yang disebut Eksplorasi Data. Tanpa penguasaan EDA yang baik, model Machine Learning secanggih apa pun yang dibangun pada pertemuan-pertemuan selanjutnya berisiko dibangun di atas data yang keliru dipahami, sehingga menghasilkan kesimpulan yang menyesatkan — sebuah prinsip yang dalam ilmu komputer dikenal dengan istilah *garbage in, garbage out*.

Selain manfaat konseptual tersebut, bab ini juga membekali mahasiswa dengan keterampilan teknis yang bersifat mendasar dan akan terus digunakan sepanjang delapan pertemuan mata kuliah ini, bahkan sepanjang karier sebagai praktisi data maupun rekayasa industri. Kemampuan menyiapkan lingkungan kerja Python, memuat berkas data, memeriksa struktur data dengan *pandas*, dan menghasilkan visualisasi informatif adalah keterampilan yang bersifat *transferable* — dapat diterapkan pada domain data apa pun, baik data penjualan ritel, data produksi pabrik, data kualitas udara, data logistik, maupun data-data industri lain yang akan dijumpai mahasiswa pada masa perkuliahan lanjut maupun dunia kerja.

Capaian Pembelajaran Mata Kuliah (CPMK) dan Sub-CPMK

Mata kuliah praktik Machine Learning secara keseluruhan dirancang agar mahasiswa mampu menguasai siklus pengembangan solusi pembelajaran mesin secara utuh, mulai dari pemahaman data, pra-pemrosesan, pemodelan prediktif dan deskriptif, hingga evaluasi dan penerapan model pada konteks industri manufaktur. Pertemuan pertama ini menjadi fondasi bagi capaian-capaian pembelajaran pada pertemuan berikutnya, sehingga CPMK dan Sub-CPMK yang dibebankan pada bab ini dirumuskan sebagai berikut, diturunkan secara langsung dari tujuan pembelajaran khusus yang telah dirancang dalam rencana pembelajaran pertemuan ini.

CPMK yang didukung oleh bab ini:

CPMK-1: Mahasiswa mampu menyiapkan dan menggunakan lingkungan kerja perangkat lunak untuk analisis data dan pembelajaran mesin secara mandiri, baik pada komputer pribadi maupun platform komputasi awan.

CPMK-2: Mahasiswa mampu melakukan eksplorasi dan analisis data awal (*Exploratory Data Analysis*) terhadap kumpulan data tabular untuk mengidentifikasi struktur, kualitas, dan karakteristik data sebagai dasar pengambilan keputusan pada tahap pra-pemrosesan dan pemodelan.

Sub-CPMK dan indikator capaian untuk Pertemuan 1:

Sub-CPMK 1.1 — Mahasiswa mampu menyiapkan lingkungan kerja Python, baik melalui instalasi lokal (Anaconda dan Jupyter Notebook) maupun melalui platform Google Colaboratory, serta memverifikasi bahwa pustaka *pandas*, *numpy*, *matplotlib*, dan *seaborn* telah siap digunakan. Indikator: mahasiswa dapat menjalankan kode verifikasi lingkungan

tanpa galat (*error*) dan menjelaskan perbedaan mendasar antara kedua opsi lingkungan kerja tersebut.

Sub-CPMK 1.2 — Mahasiswa mampu memuat data berformat CSV ke dalam struktur data DataFrame pandas dan melakukan inspeksi awal terhadap data tersebut. Indikator: mahasiswa dapat menggunakan fungsi `pd.read_csv()`, `df.head()`, `df.tail()`, `df.shape`, `df.info()`, `df.describe()`, dan `df.dtypes` secara tepat serta menafsirkan keluaran (*output*) dari setiap fungsi tersebut.

Sub-CPMK 1.3 — Mahasiswa mampu mengidentifikasi dan menangani nilai yang hilang (*missing value*) pada suatu dataset menggunakan teknik penghapusan (*drop*) dan pengisian (*imputasi*) sederhana, disertai justifikasi pemilihan teknik berdasarkan konteks data. Indikator: mahasiswa dapat menjelaskan kapan sebaiknya menghapus data yang hilang dan kapan sebaiknya mengisinya, serta menerapkan `dropna()` dan `fillna()` dengan benar.

Sub-CPMK 1.4 — Mahasiswa mampu membangun dan menginterpretasikan visualisasi data dasar — histogram, *boxplot*, *scatter plot*, dan *heatmap* korelasi — menggunakan pustaka Matplotlib dan Seaborn untuk menggali wawasan (*insight*) dari data. Indikator: mahasiswa dapat memilih jenis visualisasi yang tepat sesuai tujuan analisis dan mampu menuliskan interpretasi naratif dari setiap visualisasi yang dihasilkan.

Sub-CPMK 1.5 — Mahasiswa mampu menerapkan seluruh keterampilan EDA di atas secara mandiri pada dataset baru yang belum pernah dijumpai sebelumnya, serta mampu bekerja sama dalam tim untuk menganalisis dataset yang lebih kompleks dan menyusun laporan temuan secara ringkas dan sistematis. Indikator: mahasiswa dapat menyelesaikan studi kasus mandiri dan studi kasus kelompok dengan menghasilkan minimal tiga visualisasi bermakna beserta insight tertulis, dan kelompok dapat menyusun laporan satu halaman yang memuat temuan utama dan rekomendasi.

Keterkaitan antara CPMK, Sub-CPMK, indikator, materi, dan asesmen pada bab ini bersifat selaras: seluruh materi pada bagian Penyajian Materi disusun untuk memenuhi kelima Sub-CPMK tersebut, latihan terbimbing dan studi kasus dirancang sebagai wahana penerapan, sedangkan Tes Formatif pada akhir bab dirancang untuk mengukur ketercapaian kelima Sub-CPMK tersebut secara menyeluruh melalui soal-soal yang menguji definisi, penerapan, analisis kode, dan interpretasi hasil.

Petunjuk Penggunaan Bab Ini

Agar proses belajar mandiri menggunakan bab ini berjalan efektif, mahasiswa disarankan mengikuti alur berikut. Pertama, bacalah bagian Penyajian Materi secara berurutan dari sub-bab pertama hingga terakhir, karena setiap sub-bab disusun secara kumulatif — pemahaman terhadap Pandas, misalnya, menjadi prasyarat untuk memahami bagian inspeksi data dan visualisasi. Kedua, jangan hanya membaca kode secara pasif; salin (*copy*) setiap cuplikan kode ke dalam Jupyter Notebook atau Google Colab milik Anda sendiri, jalankan, dan amati keluarannya, lalu bandingkan dengan penjelasan naratif yang menyertainya. Ketiga, setelah menuntaskan bagian Penyajian Materi, kerjakan bagian Latihan Mandiri Terbimbing langkah demi langkah tanpa melompati satu langkah pun, karena setiap langkah dibangun di atas hasil langkah sebelumnya. Keempat, pilih dan

kerjakan Studi Kasus Mandiri secara individu untuk menguji kemampuan menerapkan EDA pada data yang belum pernah dijumpai. Kelima, jika bab ini dipelajari dalam konteks kerja kelompok, kerjakan pula Studi Kasus Kelompok bersama rekan tim. Keenam, setelah seluruh latihan dan studi kasus selesai, ujilah pemahaman Anda secara mandiri menggunakan Tes Formatif pada bagian akhir bab, kerjakan seluruh soal terlebih dahulu tanpa melihat kunci jawaban, baru kemudian cocokkan jawaban dan pelajari pembahasannya. Gunakan rumus penghitungan skor pada bagian Umpan Balik dan Tindak Lanjut untuk menilai tingkat penguasaan Anda secara objektif dan menentukan apakah Anda siap melanjutkan ke Pertemuan 2 atau perlu mengulang bagian tertentu terlebih dahulu.

Sebagai catatan tambahan, seluruh kode pada bab ini ditulis menggunakan bahasa pemrograman Python versi 3.9 ke atas dan mengasumsikan pustaka `pandas`, `numpy`, `matplotlib`, dan `seaborn` telah terpasang, baik melalui distribusi Anaconda maupun secara bawaan pada Google Colab. Dataset yang dirujuk sepanjang bab ini — `retail_sales.csv`, `factory_production.csv`, dan `air_quality.csv` — disediakan oleh dosen pengampu melalui tautan berbagi (*Google Drive*) atau platform pembelajaran (LMS) Polman Bandung, sebagaimana diumumkan pada sesi perkuliahan.

PENYAJIAN MATERI

Bagian ini merupakan inti dari bab pertama buku ajar ini. Materi disajikan secara bertahap, dimulai dari pemahaman terhadap ekosistem perangkat lunak yang akan digunakan sepanjang mata kuliah, dilanjutkan dengan penguasaan struktur data fundamental dalam Python, kemudian keterampilan memuat dan memeriksa data, penanganan data yang tidak lengkap, dan diakhiri dengan visualisasi data untuk menggali wawasan. Setiap konsep dijelaskan dengan pola definisi, uraian, contoh kode yang dapat dijalankan, dan keterkaitannya dengan konsep lain maupun konteks industri manufaktur.

1. Ekosistem Perangkat Lunak untuk Data Science dan Machine Learning

1.1 Mengapa Python?

Python adalah bahasa pemrograman tingkat tinggi (*high-level programming language*) yang bersifat *general-purpose*, artinya dapat digunakan untuk berbagai keperluan komputasi, mulai dari pengembangan aplikasi web, otomasi tugas administratif, hingga komputasi ilmiah dan kecerdasan buatan. Python diciptakan dengan filosofi mengutamakan keterbacaan kode (*readability*) dan kesederhanaan sintaksis, sehingga baris kode Python cenderung menyerupai bahasa Inggris sehari-hari dibandingkan bahasa pemrograman lain seperti C++ atau Java. Karakteristik inilah yang menjadikan Python sebagai bahasa pilihan utama di bidang ilmu data (*data science*) dan pembelajaran mesin (*machine learning*) selama lebih dari satu dekade terakhir.

Ada beberapa alasan konkret mengapa Python mendominasi bidang ini. Pertama, Python memiliki ekosistem pustaka (*library*) pihak ketiga yang sangat kaya dan matang, seperti NumPy untuk komputasi numerik, pandas untuk manipulasi data tabular, Matplotlib dan Seaborn untuk visualisasi, serta scikit-learn, TensorFlow, dan PyTorch untuk pembelajaran mesin dan pembelajaran mendalam. Kedua, Python bersifat sumber terbuka (*open source*) dan gratis, sehingga dapat diakses oleh siapa pun tanpa hambatan biaya lisensi — hal ini penting bagi institusi pendidikan maupun industri kecil dan menengah yang ingin mengadopsi analitik data tanpa investasi awal yang besar. Ketiga, Python memiliki komunitas pengguna global yang sangat besar, sehingga dokumentasi, forum diskusi, dan sumber belajar tersedia melimpah, memudahkan pemula maupun praktisi berpengalaman mencari solusi atas permasalahan yang dihadapi. Keempat, Python bersifat *interpreted* (kode dijalankan baris demi baris oleh interpreter, bukan dikompilasi terlebih dahulu menjadi kode mesin), sehingga cocok digunakan dalam lingkungan interaktif seperti Jupyter Notebook, di mana seorang analis data dapat menjalankan satu blok kode, memeriksa hasilnya, lalu memutuskan langkah berikutnya secara iteratif — sebuah pola kerja yang sangat sesuai dengan sifat eksploratif dari analisis data.

Dalam konteks industri manufaktur, penguasaan Python memberi mahasiswa kemampuan untuk membangun jembatan antara data operasional pabrik (yang sering tersimpan dalam format seperti CSV, Excel, atau basis data SQL) dengan teknik analitik dan pembelajaran mesin modern, tanpa harus bergantung pada perangkat lunak analitik berbayar yang mahal dan seringkali kurang fleksibel untuk kebutuhan khusus suatu lini produksi.

1.2 Komponen Ekosistem yang Digunakan dalam Mata Kuliah Ini

Ekosistem Python untuk data science dapat dibayangkan sebagai sebuah rangkaian alat kerja (*toolchain*) yang saling melengkapi, di mana setiap alat memiliki peran spesifik dalam siklus kerja analisis data. Empat pustaka inti yang akan digunakan secara intensif pada mata kuliah ini adalah sebagai berikut.

NumPy (*Numerical Python*) adalah pustaka fundamental untuk komputasi numerik di Python. Pustaka ini menyediakan struktur data *array* multidimensi yang jauh lebih efisien secara memori dan kecepatan komputasi dibandingkan struktur data *list* bawaan Python, terutama untuk operasi matematis pada kumpulan angka berukuran besar. Hampir seluruh pustaka data science lain, termasuk *pandas* dan *scikit-learn*, dibangun di atas fondasi NumPy.

pandas adalah pustaka manipulasi dan analisis data tabular yang menyediakan dua struktur data utama, yaitu *Series* (data satu dimensi, setara satu kolom) dan *DataFrame* (data dua dimensi berbentuk tabel dengan baris dan kolom, mirip lembar kerja Excel atau tabel basis data). *pandas* adalah alat utama yang akan digunakan sepanjang bab ini untuk memuat, memeriksa, membersihkan, dan mentransformasi data.

Matplotlib adalah pustaka visualisasi data tingkat dasar (*low-level*) yang menyediakan kendali penuh atas setiap elemen visual sebuah grafik — sumbu, warna, label, ukuran, dan sebagainya. Karena sifatnya yang mendasar, Matplotlib menjadi fondasi bagi banyak pustaka visualisasi lain, termasuk Seaborn.

Seaborn adalah pustaka visualisasi statistik tingkat tinggi (*high-level*) yang dibangun di atas Matplotlib. Seaborn menyederhanakan pembuatan visualisasi statistik yang kompleks — seperti *boxplot*, *heatmap*, dan *pairplot* — menjadi satu atau dua baris kode, sekaligus menghasilkan tampilan visual yang lebih estetik secara *default* dibandingkan Matplotlib murni.

Keempat pustaka ini bekerja secara komplementer: NumPy menjadi fondasi numerik, *pandas* menjadi alat manipulasi data yang dibangun di atas NumPy, sementara Matplotlib dan Seaborn menjadi alat untuk memvisualisasikan data yang telah diproses oleh *pandas*. Memahami peran masing-masing pustaka ini penting agar mahasiswa tidak sekadar menghafal fungsi, melainkan memahami mengapa suatu fungsi berasal dari pustaka tertentu dan bagaimana pustaka-pustaka tersebut saling berinteraksi dalam satu alur kerja analisis data.

2. Menyiapkan Lingkungan Kerja

Sebelum menulis satu baris kode analisis data pun, seorang praktisi data perlu menyiapkan lingkungan kerja (*development environment*) tempat kode tersebut akan ditulis dan dijalankan. Terdapat dua opsi utama yang akan digunakan dalam mata kuliah ini: instalasi lokal menggunakan Anaconda dan Jupyter Notebook, atau layanan komputasi awan Google Colaboratory. Mahasiswa bebas memilih salah satu opsi, atau bahkan menggunakan keduanya secara bergantian sesuai kebutuhan, karena kode yang ditulis pada kedua lingkungan tersebut pada dasarnya identik.

2.1 Instalasi Lokal: Anaconda dan Jupyter Notebook

Anaconda Distribution adalah sebuah paket distribusi Python yang dirancang khusus untuk keperluan data science dan komputasi ilmiah. Anaconda tidak hanya menyertakan interpreter Python, tetapi juga ratusan pustaka populer — termasuk NumPy, pandas, Matplotlib, dan Seaborn — yang sudah terpasang secara otomatis, sehingga pengguna tidak perlu memasang setiap pustaka satu per satu secara manual. Anaconda juga menyertakan Jupyter Notebook, yaitu aplikasi antarmuka web interaktif yang memungkinkan pengguna menulis dan menjalankan kode Python secara bertahap dalam unit-unit kecil yang disebut *cell*, sambil menyisipkan teks penjelasan, gambar, dan hasil visualisasi dalam satu dokumen yang sama.

Langkah-langkah instalasi Anaconda pada komputer pribadi adalah sebagai berikut. Pertama, unduh installer Anaconda sesuai sistem operasi yang digunakan (Windows, macOS, atau Linux) dari laman resmi <https://www.anaconda.com/download>. Kedua, jalankan berkas installer yang telah diunduh dan ikuti petunjuk pada layar; pada sistem operasi Windows, disarankan mencentang opsi “Add Anaconda to my PATH environment variable” apabila muncul, agar perintah Anaconda dapat diakses dari command prompt mana pun. Ketiga, setelah proses instalasi selesai, buka aplikasi Anaconda Navigator dari menu Start (Windows) atau Launchpad (macOS). Keempat, pada antarmuka Anaconda Navigator, klik tombol **Launch** di bawah ikon Jupyter Notebook; langkah ini akan membuka sebuah tab peramban (*browser*) baru yang menampilkan antarmuka Jupyter. Kelima, arahkan navigasi berkas pada antarmuka Jupyter menuju folder tempat mahasiswa ingin menyimpan proyek perkuliahan, kemudian klik tombol **New** dan pilih **Python 3** untuk membuat sebuah *notebook* baru.

Sebagai alternatif bagi pengguna yang tidak ingin memasang Anaconda karena keterbatasan ruang penyimpanan, Python dapat dipasang secara mandiri dari laman resmi python.org, kemudian Jupyter Notebook dipasang secara terpisah melalui perintah `pip install notebook` pada terminal, dan dijalankan dengan perintah `jupyter notebook`. Namun demikian, pendekatan Anaconda lebih disarankan bagi pemula karena mengurangi risiko konflik antar-versi pustaka yang sering menjadi sumber kebingungan pada tahap awal belajar.

2.2 Google Colaboratory (Colab): Alternatif Berbasis Awan

Bagi mahasiswa yang mengalami kendala teknis dalam instalasi lokal — misalnya keterbatasan ruang penyimpanan, spesifikasi komputer yang rendah, atau kendala sistem operasi — Google Colaboratory (disingkat Google Colab) menjadi alternatif yang sangat praktis. Google Colab adalah layanan *notebook* Jupyter berbasis awan yang disediakan gratis oleh Google, berjalan sepenuhnya di peramban web tanpa memerlukan instalasi apa pun di komputer pengguna. Seluruh proses komputasi dijalankan pada server milik Google, sehingga komputer pengguna hanya berfungsi sebagai jendela tampilan.

Untuk mulai menggunakan Google Colab, mahasiswa cukup membuka laman <https://colab.research.google.com> pada peramban web, masuk (*login*) menggunakan akun Google pribadi, kemudian memilih menu **File** → **New notebook** untuk membuat *notebook* baru. Google Colab telah menyediakan Python beserta sebagian besar pustaka

data science populer secara *default*, termasuk *pandas*, *numpy*, *matplotlib*, dan *seaborn*, sehingga pada umumnya mahasiswa dapat langsung menulis kode analisis data tanpa proses instalasi tambahan. Apabila suatu pustaka khusus belum tersedia, mahasiswa dapat memasangnya melalui perintah `!pip install nama_pustaka` yang dijalankan langsung di dalam sel kode notebook — tanda seru (!) di awal perintah menandakan bahwa baris tersebut dijalankan sebagai perintah *shell* sistem operasi, bukan sebagai kode Python.

Google Colab menawarkan sejumlah keunggulan signifikan: tidak memerlukan instalasi apa pun, dapat diakses dari perangkat manapun yang terhubung internet, gratis, menyediakan akses terbatas ke akselerator perangkat keras seperti GPU dan TPU yang berguna untuk pertemuan-pertemuan lanjutan mata kuliah ini (khususnya pada materi *deep learning*), serta menyimpan berkas *notebook* secara otomatis ke Google Drive sehingga mudah dibagikan kepada dosen atau rekan kelompok. Meskipun demikian, terdapat pula kelemahan yang perlu diperhatikan, yaitu ketergantungan pada koneksi internet yang stabil, serta sesi kerja yang dapat terputus secara otomatis apabila dibiarkan tidak aktif (*idle*) dalam jangka waktu tertentu, sehingga mahasiswa disarankan untuk menyimpan pekerjaan secara berkala menggunakan kombinasi tombol `Ctrl+S` atau `Cmd+S`.

2.3 Verifikasi Lingkungan Kerja

Setelah lingkungan kerja — baik *Anaconda* maupun *Google Colab* — berhasil disiapkan, langkah pertama yang selalu dilakukan sebelum memulai analisis data adalah memverifikasi bahwa seluruh pustaka yang dibutuhkan telah terpasang dan dapat diimpor tanpa galat. Berikut adalah cuplikan kode verifikasi standar yang akan digunakan sepanjang mata kuliah ini.

```
# Cell 1: Verifikasi versi Python dan ketersediaan pustaka inti
import sys
print("Python version:", sys.version)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print("Semua library siap digunakan.")
```

Kode di atas melakukan beberapa hal secara berurutan. Baris pertama mengimpor modul *sys* bawaan Python, yang kemudian digunakan untuk mencetak informasi versi Python yang sedang berjalan pada baris kedua — langkah ini penting karena beberapa fitur pustaka data science hanya kompatibel dengan versi Python tertentu (mata kuliah ini mengasumsikan Python 3.9 ke atas). Empat baris `import` berikutnya memuat pustaka *pandas*, *numpy*, *matplotlib.pyplot*, dan *seaborn* ke dalam ruang kerja (*namespace*) Python, masing-masing diberi nama alias yang telah menjadi konvensi universal di komunitas data science: *pd* untuk *pandas*, *np* untuk *numpy*, *plt* untuk *matplotlib.pyplot*, dan *sns* untuk *seaborn*. Penggunaan alias ini bukan keharusan teknis, melainkan konvensi yang disepakati luas sehingga kode menjadi lebih ringkas dan mudah dipahami oleh siapa

pun yang membaca kode tersebut, di mana pun di dunia. Baris terakhir mencetak pesan konfirmasi jika seluruh proses impor berhasil dijalankan tanpa galat.

Apabila salah satu baris `import` menghasilkan pesan galat berupa `ModuleNotFoundError: No module named 'pandas'` (atau nama pustaka lain), artinya pustaka tersebut belum terpasang pada lingkungan kerja yang digunakan. Solusinya adalah menjalankan perintah pemasangan pustaka pada sel kode terpisah, sebagai berikut.

```
# Cell alternatif: memasang pustaka yang belum tersedia
!pip install pandas numpy matplotlib seaborn
```

Perintah di atas menggunakan `pip`, yaitu manajer paket (*package manager*) standar Python, untuk mengunduh dan memasang keempat pustaka tersebut dari repositori resmi Python Package Index (PyPI). Tanda seru di awal baris menginstruksikan Jupyter maupun Colab untuk menjalankan baris tersebut sebagai perintah sistem operasi, bukan sebagai kode Python biasa. Setelah proses pemasangan selesai, mahasiswa perlu menjalankan kembali sel kode verifikasi pada Cell 1 untuk memastikan seluruh pustaka kini dapat diimpor dengan sukses. Langkah verifikasi ini sederhana namun sangat penting sebagai kebiasaan kerja yang baik (*good practice*), karena mendeteksi permasalahan lingkungan kerja sejak awal jauh lebih mudah diselesaikan dibandingkan menemukannya di tengah proses analisis yang sudah berjalan panjang.

2.4 Struktur dan Kegunaan Notebook: Sel Kode dan Sel Markdown

Baik Jupyter Notebook maupun Google Colab menggunakan format dokumen yang sama, yaitu *notebook* yang tersusun atas rangkaian sel (*cell*). Terdapat dua jenis sel utama yang perlu dipahami mahasiswa. Sel **Code** adalah sel yang berisi kode Python yang dapat dijalankan (*executed*); ketika dijalankan, hasil keluaran kode tersebut — baik berupa teks, tabel, maupun grafik — akan ditampilkan tepat di bawah sel yang bersangkutan. Sel **Markdown** adalah sel yang berisi teks terformat menggunakan sintaksis Markdown, seperti judul, sub-judul, teks tebal, teks miring, daftar berpoin, dan tautan; sel ini tidak berisi kode yang dijalankan, melainkan digunakan untuk menuliskan penjelasan, dokumentasi, atau catatan analisis di antara sel-sel kode.

Kombinasi kedua jenis sel ini menjadikan *notebook* sebagai medium yang ideal untuk analisis data eksploratif, karena memungkinkan seorang analis menuliskan narasi penjelasan tepat berdampingan dengan kode dan hasilnya — sebuah praktik yang dikenal dengan istilah *literate programming*. Sebagai latihan awal, mahasiswa disarankan menambahkan sel Markdown pada bagian paling atas *notebook* yang berisi judul latihan dan identitas diri, sebagai berikut.

```
# Pertemuan 1 - Pengenalan Ekosistem ML & Eksplorasi Data
## Latihan Terbimbing
**Nama:** (isi nama Anda)
**NIM:** (isi NIM Anda)
**Kelas:** (isi kelas Anda)
```

Cuplikan di atas bukan kode Python, melainkan teks Markdown yang ketika dijalankan pada sel Markdown akan diterjemahkan menjadi judul berjenjang dan teks tebal. Tanda

pagar tunggal (#) menghasilkan judul tingkat satu, tanda pagar ganda (##) menghasilkan judul tingkat dua (sub-judul), sedangkan teks yang diapit oleh dua tanda bintang (**...**) akan ditampilkan tebal. Kebiasaan mendokumentasikan identitas dan tujuan pada setiap *notebook* yang dibuat merupakan praktik profesional yang akan terus dituntut sepanjang mata kuliah ini, sekaligus mempermudah dosen dalam memeriksa dan memberi umpan balik atas tugas yang dikumpulkan.

3. Dasar-Dasar Python untuk Analisis Data

Sebelum masuk lebih jauh ke pustaka *pandas*, mahasiswa perlu memahami beberapa struktur data fundamental dalam Python yang menjadi dasar bagi struktur data yang lebih kompleks pada *pandas* dan *NumPy*. Bagi mahasiswa yang telah mengambil mata kuliah pemrograman dasar, bagian ini berfungsi sebagai penyegaran (*refresher*); bagi yang belum terbiasa, bagian ini menjadi fondasi penting yang wajib dikuasai.

3.1 List dan Dictionary

List adalah struktur data bawaan Python yang menampung kumpulan nilai secara terurut (*ordered*) dan dapat diubah (*mutable*) setelah dibuat. Sebuah *list* dituliskan dengan mengapit nilai-nilai di antara tanda kurung siku, dipisahkan tanda koma, misalnya [1, 2, 3] untuk *list* berisi tiga bilangan bulat, atau ['Pagi', 'Siang', 'Malam'] untuk *list* berisi tiga nilai teks yang merepresentasikan shift kerja pada sebuah pabrik. Setiap elemen dalam *list* dapat diakses melalui indeks numeriknya, yang di Python selalu dimulai dari angka nol; misalnya pada *list* `shift = ['Pagi', 'Siang', 'Malam']`, ekspresi `shift[0]` akan menghasilkan nilai 'Pagi'.

Dictionary adalah struktur data yang menyimpan data dalam bentuk pasangan kunci-nilai (*key-value pairs*), dituliskan dengan mengapit pasangan tersebut di antara tanda kurung kurawal, misalnya {'nama': 'Budi', 'usia': 21}. Setiap nilai diakses bukan melalui indeks numerik seperti pada *list*, melainkan melalui kuncinya (*key*); ekspresi `data['nama']` pada contoh tersebut akan menghasilkan nilai 'Budi'. Struktur *dictionary* ini penting dipahami karena menjadi salah satu cara paling umum untuk membangun sebuah *DataFrame* *pandas* dari awal, sebagaimana akan ditunjukkan pada sub-bab berikutnya.

3.2 Array NumPy sebagai Fondasi Komputasi Numerik

Meskipun *list* Python dapat menampung kumpulan angka, *list* tidak dirancang untuk operasi matematis berskala besar secara efisien. Sebagai ilustrasi, menjumlahkan dua *list* berisi angka menggunakan operator + pada Python murni tidak menghasilkan penjumlahan elemen-per-elemen sebagaimana yang diharapkan dalam konteks matematika, melainkan menggabungkan (*concatenate*) kedua *list* tersebut menjadi satu *list* yang lebih panjang. Untuk mengatasi keterbatasan ini, pustaka *NumPy* menyediakan struktur data array, yaitu struktur data multidimensi bertipe homogen (seluruh elemen harus bertipe data yang sama) yang dioptimalkan untuk operasi matematis dan statistik berskala besar.

```
# Cell: Membuat dan mengoperasikan array NumPy
import numpy as np
```

```
arr = np.array([1, 2, 3, 4])
print(arr.mean())    # menghasilkan 2.5
print(arr * 2)       # menghasilkan array([2, 4, 6, 8])
```

Kode di atas mengimpor pustaka NumPy dengan alias np, kemudian membuat sebuah *array* satu dimensi berisi empat bilangan bulat menggunakan fungsi np.array(). Fungsi .mean() yang dipanggil pada objek *array* tersebut menghitung nilai rata-rata seluruh elemennya secara otomatis, menghasilkan 2.5. Baris terakhir menunjukkan salah satu keunggulan utama *array* NumPy dibandingkan *list* biasa: operasi `arr * 2` secara otomatis mengalikan **setiap elemen** dalam *array* dengan angka dua, menghasilkan *array* baru [2, 4, 6, 8] — perilaku yang dikenal sebagai *vectorization* atau operasi terpola-vektor. Perilaku ini sangat berbeda dari *list* Python murni, di mana `[1,2,3,4] * 2` justru akan menghasilkan *list* yang diulang dua kali menjadi [1, 2, 3, 4, 1, 2, 3, 4]. Kemampuan *vectorization* inilah yang membuat NumPy — dan pustaka-pustaka yang dibangun di atasnya seperti pandas — jauh lebih cepat dan ringkas dalam menangani komputasi numerik berskala besar, seperti yang lazim dijumpai pada data sensor produksi yang terdiri atas ribuan hingga jutaan baris pencatatan.

Beberapa fungsi NumPy lain yang sering dijumpai dalam analisis data adalah np.arange(start, stop, step), yang berfungsi serupa dengan range() bawaan Python namun mengembalikan objek *array*; np.random.rand(n), yang menghasilkan *array* berisi n bilangan acak antara 0 dan 1, berguna untuk membangkitkan data sintetis atau simulasi; dan yang tidak kalah penting, np.nan, yaitu representasi standar untuk nilai yang hilang atau tidak diketahui (*Not a Number*) dalam ekosistem NumPy dan pandas. Konsep np.nan ini akan menjadi sangat relevan ketika membahas penanganan *missing value* pada sub-bab berikutnya, karena pandas menggunakan representasi ini secara konsisten untuk menandai sel data yang kosong.

4. Pandas: DataFrame dan Series sebagai Struktur Data Utama

4.1 Pengertian DataFrame dan Series

DataFrame adalah struktur data inti dari pustaka pandas, berbentuk tabel dua dimensi yang tersusun atas baris (*rows*) dan kolom (*columns*), sangat mirip dengan lembar kerja pada Microsoft Excel atau sebuah tabel dalam basis data relasional. Setiap kolom dalam sebuah DataFrame dapat memiliki tipe data yang berbeda-beda — misalnya kolom Tanggal bertipe data tanggal, kolom Shift bertipe teks, dan kolom Jumlah_Produksi bertipe bilangan bulat — namun seluruh nilai dalam satu kolom yang sama harus konsisten tipe datanya. Series adalah struktur data satu dimensi yang merepresentasikan satu kolom tunggal dari sebuah DataFrame, atau dapat pula berdiri sendiri sebagai struktur data mandiri. Hubungan antara keduanya bersifat hierarkis: sebuah DataFrame pada dasarnya adalah kumpulan beberapa objek Series yang berbagi indeks baris yang sama.

Pemahaman terhadap perbedaan DataFrame dan Series penting secara praktis, karena banyak metode (*method*) pandas menghasilkan objek Series ketika dipanggil pada satu kolom tunggal (misalnya `df['Quantity']`), tetapi menghasilkan objek DataFrame ketika

dipanggil pada beberapa kolom sekaligus (misalnya `df[['Quantity', 'UnitPrice']]`). Kesalahan memahami tipe objek yang dihasilkan seringkali menjadi sumber kebingungan bagi pemula, terutama ketika suatu metode hanya tersedia pada satu jenis objek namun tidak pada jenis lainnya.

4.2 Membangun DataFrame dari Awal

Salah satu cara paling sederhana untuk memahami struktur DataFrame adalah dengan membangunnya secara manual dari sebuah *dictionary* Python, sebagaimana ditunjukkan pada cuplikan kode berikut.

```
# Cell: Membangun DataFrame dari dictionary
import pandas as pd

data = {'Nama': ['Ali', 'Budi'], 'Nilai': [85, 90]}
df = pd.DataFrame(data)
print(df)
```

Keluaran dari kode di atas adalah sebagai berikut.

	Nama	Nilai
0	Ali	85
1	Budi	90

Pada kode tersebut, sebuah *dictionary* bernama `data` dibuat dengan dua kunci, yaitu `'Nama'` dan `'Nilai'`, di mana masing-masing kunci memiliki nilai berupa *list*. Fungsi `pd.DataFrame()` kemudian mengubah struktur *dictionary* tersebut menjadi tabel DataFrame, di mana setiap kunci pada *dictionary* menjadi nama kolom, dan setiap elemen dalam *list* menjadi baris data pada kolom yang bersangkutan. Perhatikan pula kolom paling kiri pada keluaran, yang berisi angka 0 dan 1 — ini disebut **indeks** (*index*), yaitu label unik yang diberikan pandas secara otomatis untuk setiap baris, dimulai dari angka nol, kecuali ditentukan lain oleh pengguna. Indeks ini berfungsi sebagai “alamat” setiap baris data dan akan sering digunakan ketika melakukan pemilihan (*selection*) atau penggabungan (*merge*) data pada pertemuan-pertemuan berikutnya.

Meskipun membangun DataFrame secara manual berguna untuk memahami struktur dasarnya, dalam praktik analisis data sesungguhnya, seorang analis jarang membangun DataFrame dari nol. Sebagian besar pekerjaan analisis data dimulai dengan memuat data yang sudah tersedia dalam suatu berkas — paling umum dalam format CSV (*Comma-Separated Values*) — sebagaimana akan dibahas secara rinci pada sub-bab berikutnya.

5. Matplotlib dan Seaborn: Pengantar Awal Visualisasi

Sebelum membahas visualisasi data secara mendalam pada sub-bab tersendiri, ada baiknya mahasiswa mengenal terlebih dahulu sintaksis paling dasar dari Matplotlib dan Seaborn, karena keduanya akan menjadi alat utama untuk menggali wawasan dari data sepanjang bab ini dan mata kuliah secara keseluruhan.

```
# Cell: Contoh dasar Matplotlib - plot garis
```

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4]
```

```
y = [10, 20, 25, 30]
```

```
plt.plot(x, y)
```

```
plt.title('Contoh Plot Garis')
```

```
plt.show()
```

Kode tersebut membuat sepasang *list* *x* dan *y* yang merepresentasikan koordinat titik-titik pada sebuah grafik, kemudian fungsi `plt.plot()` menggambar garis yang menghubungkan titik-titik tersebut secara berurutan. Fungsi `plt.title()` menambahkan judul di atas grafik, sementara `plt.show()` menampilkan grafik yang telah dibangun ke layar. Pola tiga langkah ini — memanggil fungsi plotting, menambahkan elemen dekoratif seperti judul dan label sumbu, lalu memanggil `plt.show()` — akan berulang kali dijumpai sepanjang bab ini.

```
# Cell: Contoh dasar Seaborn - scatter plot statistik
```

```
import seaborn as sns
```

```
sns.set_style('darkgrid')
```

```
tips = sns.load_dataset('tips') # dataset bawaan Seaborn
```

```
sns.scatterplot(data=tips, x='total_bill', y='tip')
```

```
plt.show()
```

Pada cuplikan di atas, fungsi `sns.set_style('darkgrid')` mengubah tampilan visual default seluruh grafik yang akan dibuat setelahnya menjadi bergaya latar belakang gelap bergaris kisi (*grid*), salah satu dari beberapa gaya visual yang disediakan Seaborn untuk mempercantik tampilan grafik tanpa konfigurasi manual yang rumit. Fungsi `sns.load_dataset('tips')` memuat sebuah dataset contoh bernama *tips* yang telah disertakan langsung di dalam pustaka Seaborn untuk keperluan pembelajaran dan demonstrasi — dataset ini berisi data transaksi di sebuah restoran, mencakup nilai tagihan (*total_bill*) dan besaran uang tip yang diberikan pelanggan. Fungsi `sns.scatterplot()` kemudian membuat *scatter plot* (diagram pencar) yang menampilkan hubungan antara kedua variabel tersebut, di mana parameter *data* menunjuk ke *DataFrame* sumber data, sedangkan parameter *x* dan *y* menentukan kolom mana yang menjadi sumbu horizontal dan vertikal. Perhatikan bahwa sintaksis Seaborn secara konsisten menerima parameter *data* sebagai sumber *DataFrame*, kemudian merujuk nama kolom secara langsung sebagai string pada parameter *x* dan *y* — pola ini jauh lebih ringkas dibandingkan pendekatan Matplotlib murni yang mengharuskan pengguna mengekstraksi nilai setiap kolom secara eksplisit sebelum memplotnya.

Dengan bekal pemahaman terhadap ekosistem perangkat lunak, struktur data dasar Python, NumPy, dan pengantar pandas, Matplotlib, serta Seaborn di atas, mahasiswa kini siap untuk mempelajari langkah pertama yang sesungguhnya dalam setiap proyek analisis data: memuat dataset nyata ke dalam lingkungan kerja dan memahaminya secara mendalam, sebagaimana akan dibahas pada sub-bab berikutnya.

6. Memuat dan Memahami Dataset

6.1 Dataset Retail Sales sebagai Bahan Latihan Utama

Sepanjang bagian Penyajian Materi ini, mahasiswa akan menggunakan dataset `retail_sales.csv` sebagai bahan latihan utama. Dataset ini diadaptasi dari *UCI Online Retail Dataset*, sebuah kumpulan data transaksi penjualan ritel daring yang populer digunakan dalam pembelajaran analisis data. Untuk keperluan mata kuliah ini, dataset telah disederhanakan menjadi 5.000 baris pertama agar ringan diproses oleh komputer mahasiswa maupun lingkungan Google Colab. Dataset ini terdiri atas delapan kolom sebagai berikut: `InvoiceNo` (nomor faktur transaksi), `StockCode` (kode unik produk), `Description` (nama atau deskripsi produk), `Quantity` (jumlah unit yang dibeli dalam satu transaksi), `InvoiceDate` (tanggal dan jam transaksi terjadi), `UnitPrice` (harga satuan produk dalam mata uang Poundsterling), `CustomerID` (nomor identitas pelanggan), dan `Country` (negara asal pelanggan).

Meskipun dataset ini berasal dari konteks ritel dan bukan manufaktur, dataset ini dipilih sebagai latihan pembuka karena strukturnya yang relatif sederhana dan representatif terhadap permasalahan data nyata: terdapat data yang hilang, terdapat nilai ekstrem (outlier), dan terdapat campuran tipe data numerik dan kategorik — karakteristik yang juga lazim dijumpai pada data produksi pabrik, data downtime mesin, maupun data sensor kualitas yang akan dibahas pada Studi Kasus Mandiri dan Studi Kasus Kelompok di bagian akhir bab ini. Dengan kata lain, keterampilan yang dilatih pada dataset ritel ini bersifat *transferable* sepenuhnya ke konteks data industri manufaktur.

6.2 Membaca Berkas CSV dengan Pandas

Format CSV (*Comma-Separated Values*) adalah format berkas teks sederhana yang merepresentasikan data tabular, di mana setiap baris mewakili satu baris data dan setiap nilai dalam baris tersebut dipisahkan oleh tanda koma (atau terkadang titik koma). Format ini menjadi salah satu format pertukaran data paling universal karena dapat dibuka oleh hampir semua perangkat lunak pengolah data, mulai dari Microsoft Excel, Google Sheets, basis data, hingga bahasa pemrograman seperti Python. Untuk memuat berkas CSV ke dalam struktur `DataFrame` pandas, digunakan fungsi `pd.read_csv()`.

```
# Cell: Memuat dataset retail_sales.csv
import pandas as pd
df = pd.read_csv('retail_sales.csv')
```

Baris kode di atas memanggil fungsi `pd.read_csv()` dengan parameter berupa nama berkas `'retail_sales.csv'`, yang mengasumsikan berkas tersebut berada pada folder kerja (*working directory*) yang sama dengan lokasi notebook. Fungsi ini akan membaca seluruh isi berkas CSV, secara otomatis mendeteksi baris pertama sebagai nama-nama kolom (*header*), lalu mengonstruksi sebuah objek `DataFrame` yang disimpan ke dalam variabel `df` — nama variabel yang menjadi konvensi hampir universal di komunitas pandas untuk merepresentasikan `DataFrame` utama yang sedang dianalisis, meskipun secara teknis mahasiswa bebas menggunakan nama variabel lain.

Apabila mahasiswa bekerja pada Google Colab, berkas CSV tidak secara otomatis tersedia di lingkungan kerja awan tersebut, sehingga diperlukan langkah tambahan untuk mengunggahnya. Terdapat dua cara umum untuk melakukan hal ini. Cara pertama adalah mengunggah berkas secara langsung ke sesi Colab yang sedang berjalan, menggunakan modul `google.colab`.

```
# Cell: Mengunggah berkas CSV secara langsung ke sesi Google Colab
from google.colab import files
uploaded = files.upload()
# Setelah dijalankan, akan muncul tombol untuk memilih berkas dari komputer lokal
```

Cara kedua, yang lebih disarankan untuk berkas yang akan digunakan berulang kali, adalah dengan menghubungkan (*mount*) Google Drive milik mahasiswa ke sesi Colab, sehingga berkas yang tersimpan di Google Drive dapat diakses seolah-olah berada pada sistem berkas lokal.

```
# Cell: Menghubungkan Google Drive ke sesi Google Colab
from google.colab import drive
drive.mount('/content/drive')
# Berkas dapat diakses misalnya melalui path
'/content/drive/MyDrive/retail_sales.csv'
```

Perbedaan mendasar kedua pendekatan tersebut adalah bahwa cara pertama (`files.upload()`) mengharuskan mahasiswa mengunggah ulang berkas setiap kali sesi Colab dimulai dari awal (karena berkas yang diunggah hanya tersimpan sementara di lingkungan sesi tersebut), sementara cara kedua (`drive.mount()`) memungkinkan berkas tetap tersimpan secara permanen di Google Drive dan dapat diakses kembali kapan pun tanpa perlu mengunggah ulang, sehingga lebih efisien untuk proyek yang dikerjakan dalam beberapa sesi terpisah.

6.3 Melihat Sekilas Data yang Telah Dimuat

Setelah data berhasil dimuat ke dalam `DataFrame`, langkah pertama yang selalu dilakukan seorang analis data — sebelum melakukan analisis apa pun — adalah melihat sekilas isi data tersebut untuk memastikan proses pemuatan berjalan sebagaimana mestinya dan untuk membangun intuisi awal terhadap bentuk data yang akan dianalisis.

```
# Cell: Melihat sekilas isi DataFrame
df.head()      # menampilkan 5 baris pertama
df.tail()      # menampilkan 5 baris terakhir
df.shape       # menampilkan dimensi (jumlah baris, jumlah kolom)
df.columns     # menampilkan daftar nama-nama kolom
```

Metode `df.head()` menampilkan lima baris pertama dari `DataFrame`, dan merupakan salah satu perintah yang paling sering digunakan dalam analisis data karena memberikan gambaran cepat mengenai struktur dan isi data — jenis nilai apa yang ada di setiap kolom, bagaimana format penulisannya, dan apakah data terlihat masuk akal sekilas. Parameter opsional dapat disertakan untuk mengubah jumlah baris yang ditampilkan, misalnya

`df.head(10)` untuk menampilkan sepuluh baris pertama. Metode `df.tail()` bekerja dengan cara serupa namun menampilkan baris-baris terakhir, yang berguna untuk memeriksa apakah ada anomali pada bagian akhir data, seperti baris ringkasan (*summary row*) yang terkadang secara tidak sengaja ikut terbawa saat data diekspor dari sistem sumbernya.

Atribut `df.shape` mengembalikan sepasang angka dalam bentuk *tuple*, yaitu (`jumlah_baris`, `jumlah_kolom`). Pada dataset `retail_sales.csv` yang telah disederhanakan, `df.shape` akan menghasilkan (`5000`, `8`), menunjukkan bahwa dataset memiliki 5.000 baris transaksi dan 8 kolom atribut. Atribut `df.columns` menampilkan daftar nama-nama kolom yang tersedia, sangat berguna terutama ketika berhadapan dengan dataset yang memiliki puluhan atau bahkan ratusan kolom, di mana mengingat seluruh nama kolom secara manual menjadi tidak praktis.

7. Inspeksi Data Mendalam

Setelah memperoleh gambaran sekilas mengenai data, langkah berikutnya dalam alur kerja EDA adalah melakukan inspeksi yang lebih mendalam terhadap struktur, tipe data, dan kualitas data. Tahap ini krusial karena menentukan keputusan-keputusan pra-pemrosesan yang akan diambil pada tahap selanjutnya.

7.1 Informasi Struktur Dataset dengan `df.info()`

Metode `df.info()` merupakan salah satu perintah paling penting dalam tahap inspeksi awal, karena dalam satu pemanggilan fungsi tunggal, metode ini menyajikan ringkasan komprehensif mengenai struktur dataset: jumlah total baris dan kolom, nama setiap kolom, jumlah nilai yang tidak kosong (*non-null count*) pada setiap kolom, tipe data setiap kolom, serta perkiraan penggunaan memori oleh dataset tersebut.

```
# Cell: Memeriksa struktur dataset
df.info()
```

Contoh keluaran (disederhanakan) dari `df.info()` pada dataset `retail_sales.csv` adalah sebagai berikut.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 8 columns):
InvoiceNo      5000 non-null object
StockCode      5000 non-null object
Description    4990 non-null object
Quantity       5000 non-null int64
InvoiceDate    5000 non-null object
UnitPrice      5000 non-null float64
CustomerID     4000 non-null float64
Country        5000 non-null object
dtypes: float64(2), int64(1), object(5)
memory usage: 312.6+ KB
```

Interpretasi terhadap keluaran ini adalah keterampilan yang sangat penting untuk dikuasai. Baris RangeIndex: 5000 entries, 0 to 4999 mengonfirmasi bahwa dataset memiliki 5.000 baris dengan indeks berjalan dari 0 hingga 4999. Pada setiap baris kolom, angka di sisi kiri (misalnya 4990 pada kolom Description, atau 4000 pada kolom CustomerID) menunjukkan jumlah nilai yang **tidak kosong** (*non-null*) pada kolom tersebut. Karena total baris dataset adalah 5.000, maka kolom Description yang hanya memiliki 4.990 nilai non-null mengindikasikan terdapat 10 baris dengan nilai kosong pada kolom tersebut, sementara kolom CustomerID yang hanya memiliki 4.000 nilai non-null mengindikasikan terdapat 1.000 baris (atau 20% dari keseluruhan data) dengan nilai CustomerID yang hilang. Inilah cara paling cepat untuk mendeteksi keberadaan *missing value* tanpa perlu menjalankan perintah terpisah.

Kolom paling kanan pada setiap baris (object, int64, float64) menunjukkan tipe data (*dtype*) kolom yang bersangkutan. Tipe object pada pandas umumnya merepresentasikan data bertipe teks (*string*) atau data campuran; tipe int64 merepresentasikan bilangan bulat; dan tipe float64 merepresentasikan bilangan pecahan (desimal). Perhatikan bahwa kolom CustomerID, meskipun secara konseptual berupa nomor identitas (yang biasanya diperlakukan sebagai data kategorik), terdeteksi bertipe float64 — hal ini terjadi karena keberadaan nilai kosong (NaN) pada kolom tersebut memaksa pandas merepresentasikan seluruh kolom sebagai bilangan desimal, sebab tipe data NaN hanya dapat direpresentasikan dalam format bilangan pecahan, bukan bilangan bulat murni. Fenomena ini merupakan salah satu contoh nyata bagaimana keberadaan *missing value* dapat memengaruhi bahkan tipe data suatu kolom, sebuah wawasan yang akan sangat relevan ketika mahasiswa menjumpai kasus serupa pada data industri, misalnya kolom ID_Mesin atau Nomor_Batch yang seharusnya bertipe bilangan bulat namun terbaca sebagai desimal akibat data yang hilang.

Metode terkait yang juga berguna adalah `df.dtypes`, yang menampilkan tipe data setiap kolom secara ringkas tanpa informasi tambahan lain seperti jumlah non-null atau penggunaan memori — berguna ketika seorang analis hanya ingin memeriksa tipe data tanpa membutuhkan ringkasan struktural yang lebih lengkap.

```
# Cell: Memeriksa tipe data setiap kolom
df.dtypes
```

7.2 Statistik Deskriptif dengan `df.describe()`

Setelah memahami struktur dataset, langkah berikutnya adalah memahami sebaran nilai (*distribution*) pada setiap kolom numerik melalui statistik deskriptif. Metode `df.describe()` menghitung dan menampilkan delapan ukuran statistik standar untuk setiap kolom bertipe numerik dalam dataset: count (jumlah nilai tidak kosong), mean (rata-rata), std (simpangan baku atau standar deviasi, ukuran seberapa tersebar nilai-nilai data dari rata-ratanya), min (nilai minimum), 25% (kuartil pertama, nilai di mana 25% data berada di bawahnya), 50% (median atau kuartil kedua), 75% (kuartil ketiga), dan max (nilai maksimum).

```
# Cell: Statistik deskriptif kolom numerik
df.describe()
```

Sebagai ilustrasi interpretasi, andaikan `df.describe()` pada kolom `Quantity` menghasilkan nilai rata-rata (mean) sekitar 9, namun nilai maksimum (max) mencapai 8.099 — perbedaan yang sangat jauh antara rata-rata dan nilai maksimum ini mengindikasikan keberadaan **nilai ekstrem** (*outlier*), yaitu satu atau beberapa transaksi dengan jumlah pembelian yang jauh melampaui pola transaksi pada umumnya, kemungkinan besar merepresentasikan pembelian dalam jumlah grosir. Pola serupa juga dapat terjadi pada kolom `UnitPrice`, di mana rata-rata harga sekitar £4 namun nilai maksimum mencapai £8.142, mengindikasikan adanya produk bernilai sangat tinggi atau, tidak menutup kemungkinan, kesalahan pencatatan data (*data entry error*). Kemampuan membaca sinyal semacam ini dari sekadar delapan angka statistik deskriptif merupakan keterampilan inti seorang analis data yang baik, dan akan terus dilatih sepanjang mata kuliah ini.

Secara *default*, `df.describe()` hanya menghitung statistik untuk kolom bertipe numerik. Untuk memperoleh ringkasan statistik pada kolom kategorik (bertipe object), digunakan parameter tambahan `include='object'`.

```
# Cell: Statistik deskriptif kolom kategorik
df.describe(include='object')
```

Keluaran dari perintah di atas menampilkan ukuran statistik yang berbeda, yang lebih sesuai untuk data kategorik: `count` (jumlah nilai tidak kosong), `unique` (jumlah nilai unik yang berbeda), `top` (nilai yang paling sering muncul, atau modus), dan `freq` (frekuensi kemunculan nilai `top` tersebut). Sebagai pelengkap, untuk memeriksa frekuensi kemunculan setiap kategori secara lebih rinci pada satu kolom tertentu, digunakan metode `value_counts()`.

```
# Cell: Frekuensi kategori pada kolom Country dan Description
df['Country'].value_counts()
df['Description'].value_counts().head(10)
```

Perintah pertama menghitung dan menampilkan jumlah transaksi untuk setiap negara yang ada pada kolom `Country`, diurutkan dari yang paling sering muncul hingga paling jarang, sehingga mahasiswa dapat dengan cepat mengidentifikasi negara mana yang paling banyak bertransaksi. Perintah kedua menampilkan sepuluh produk (`Description`) yang paling sering muncul dalam transaksi, memberikan gambaran mengenai produk-produk terlaris. Kedua jenis analisis ini merupakan bentuk sederhana namun bermakna dari analisis frekuensi, yang dalam konteks manufaktur dapat diterapkan misalnya untuk mengidentifikasi jenis cacat produk yang paling sering terjadi, atau mesin mana yang paling sering mengalami downtime.

7.3 Mengecek dan Mengukur Missing Value

Sebagaimana telah disinggung sekilas pada pembahasan `df.info()`, *missing value* atau nilai yang hilang adalah kondisi di mana suatu sel data tidak memiliki nilai — direpresentasikan pandas sebagai `NaN` (*Not a Number*). Missing value dapat muncul karena berbagai sebab: sensor yang gagal membaca data, operator yang lupa mengisi formulir, kesalahan saat proses ekspor data dari satu sistem ke sistem lain, atau memang data tersebut secara alami

tidak berlaku pada baris tertentu. Sebelum menentukan cara penanganan yang tepat, seorang analis data perlu terlebih dahulu mengukur secara presisi seberapa besar dan di kolom mana saja *missing value* tersebut terjadi.

```
# Cell: Menghitung jumlah dan persentase missing value per kolom
missing = df.isnull().sum()
missing_pct = (df.isnull().sum() / len(df)) * 100

missing_df = pd.DataFrame({'Missing': missing, 'Percent': missing_pct})
missing_df[missing_df['Missing'] > 0]
```

Kode tersebut bekerja melalui beberapa tahap. Pertama, `df.isnull()` menghasilkan sebuah DataFrame baru berukuran identik dengan `df`, namun setiap selnya berisi nilai boolean: `True` jika sel yang bersangkutan kosong (*null*), dan `False` jika sel tersebut memiliki nilai. Kedua, metode `.sum()` yang dirantai (*chained*) setelahnya menjumlahkan nilai boolean tersebut per kolom — karena Python memperlakukan `True` setara dengan angka 1 dan `False` setara dengan angka 0 dalam konteks penjumlahan, hasilnya adalah jumlah total sel kosong pada setiap kolom. Ketiga, `missing_pct` menghitung persentase dengan membagi jumlah missing tersebut dengan `len(df)` (jumlah total baris) dan mengalikannya dengan 100. Keempat, kedua hasil tersebut digabungkan ke dalam sebuah DataFrame baru bernama `missing_df` untuk memudahkan pembacaan, dan baris terakhir memfilter hasil tersebut agar hanya menampilkan kolom-kolom yang benar-benar memiliki nilai hilang (mengabaikan kolom dengan nilai missing sama dengan nol), sehingga tampilan menjadi lebih ringkas dan fokus.

Hasil analisis semacam ini pada dataset `retail_sales.csv` akan menunjukkan bahwa kolom `CustomerID` memiliki 1.000 nilai kosong (setara 20% dari total data) dan kolom `Description` memiliki 10 nilai kosong (setara 0,2%). Perbedaan besaran persentase ini menjadi dasar penting bagi pengambilan keputusan penanganan yang akan dibahas pada sub-bab berikutnya: kolom dengan persentase missing yang kecil umumnya lebih aman ditangani dengan imputasi, sementara kolom dengan persentase missing yang besar memerlukan pertimbangan lebih cermat, termasuk kemungkinan menghapus baris yang bersangkutan sama sekali.

8. Penanganan Missing Value Sederhana

Setelah mengidentifikasi dan mengukur besaran *missing value* pada tahap inspeksi, seorang analis data harus mengambil keputusan mengenai cara menanganinya. Prinsip yang harus selalu dipegang adalah bahwa **tidak ada satu metode penanganan yang berlaku universal untuk semua kasus** — keputusan harus didasarkan pada pemahaman konteks data, besaran persentase nilai yang hilang, dan konsekuensi analitis dari setiap pilihan. Sub-bab ini membahas dua pendekatan dasar: penghapusan (*deletion*) dan pengisian (*imputation*).

8.1 Menghapus Baris atau Kolom yang Mengandung Missing Value

Pendekatan pertama adalah menghapus data yang mengandung nilai kosong, baik pada level baris maupun kolom. Pandas menyediakan metode `dropna()` untuk keperluan ini, dengan beberapa variasi penggunaan sesuai kebutuhan.

```
# Cell: Menghapus baris dengan CustomerID kosong
df_clean = df.dropna(subset=['CustomerID'])
df_clean.shape
```

Kode di atas menggunakan parameter `subset=['CustomerID']` untuk menginstruksikan pandas agar hanya menghapus baris yang memiliki nilai kosong **khusus pada kolom CustomerID**, tanpa mempertimbangkan kolom lain. Hasilnya disimpan ke dalam variabel baru `df_clean`, bukan menimpa (*overwrite*) variabel `df` yang asli — sebuah praktik yang baik agar data mentah asli tetap tersimpan dan dapat dirujuk kembali jika diperlukan. Pemanggilan `df_clean.shape` setelahnya akan menunjukkan bahwa jumlah baris berkurang dari 5.000 menjadi 4.000, konsisten dengan temuan sebelumnya bahwa 1.000 baris (20%) memiliki CustomerID yang kosong.

Keputusan untuk menghapus baris berdasarkan kolom CustomerID bukan tanpa pertimbangan. Karena CustomerID berfungsi sebagai penanda identitas pelanggan yang krusial untuk analisis lanjutan seperti segmentasi pelanggan atau analisis perilaku pembelian berulang, baris data tanpa CustomerID yang valid tidak dapat dimanfaatkan secara bermakna untuk tujuan tersebut, sehingga penghapusan menjadi pilihan yang wajar meskipun berarti kehilangan seperlima dari keseluruhan data.

Selain menghapus berdasarkan kolom tertentu, terdapat variasi lain dari `dropna()` yang perlu dipahami. Untuk menghapus **kolom** yang memiliki proporsi *missing value* sangat tinggi (misalnya lebih dari 50% data kosong, sehingga kolom tersebut dianggap tidak lagi informatif), digunakan metode `drop()` dengan menyebutkan nama kolom secara eksplisit.

```
# Cell: Menghapus kolom dengan proporsi missing value sangat tinggi
df = df.drop(columns=['NamaKolom'])
```

Sementara itu, memanggil `df.dropna()` tanpa parameter apa pun akan menghapus **setiap baris** yang memiliki setidaknya satu nilai kosong pada kolom mana pun — pendekatan yang paling agresif dan harus digunakan dengan sangat hati-hati, karena berpotensi menghapus proporsi data yang sangat besar apabila missing value tersebar di banyak kolom berbeda, meskipun pada masing-masing kolom persentasenya kecil.

```
# Cell: Menghapus seluruh baris yang memiliki minimal satu nilai kosong
(gunakan dengan hati-hati)
df.dropna()
```

8.2 Mengisi Missing Value melalui Imputasi

Pendekatan kedua, yang menjadi alternatif dari penghapusan, adalah mengisi (*impute*) nilai yang kosong dengan suatu nilai pengganti yang masuk akal, sehingga baris data tersebut

tetap dapat dipertahankan untuk analisis. Pandas menyediakan metode `fillna()` untuk keperluan ini, dengan berbagai strategi pengisian yang dapat dipilih sesuai konteks data.

Untuk kolom kategorik dengan proporsi *missing value* yang kecil, seperti kolom `Description` pada dataset `retail_sales.csv` yang hanya memiliki 0,2% nilai kosong, strategi paling sederhana adalah mengisi nilai kosong tersebut dengan sebuah konstanta teks yang secara eksplisit menandakan bahwa data memang tidak tersedia.

```
# Cell: Mengisi missing value pada kolom kategorik dengan konstanta
df['Description'] = df['Description'].fillna('Unknown')
```

Untuk kolom numerik, strategi yang lebih umum adalah mengisi nilai kosong dengan ukuran pemusatan data (*central tendency*) seperti median atau rata-rata, dengan asumsi bahwa nilai yang hilang kemungkinan besar tidak jauh berbeda dari pola umum data yang ada.

```
# Cell: Mengisi missing value pada kolom numerik dengan median
median_price = df['UnitPrice'].median()
df['UnitPrice'] = df['UnitPrice'].fillna(median_price)
```

Kode di atas terlebih dahulu menghitung nilai median dari kolom `UnitPrice` menggunakan seluruh data yang tersedia (mengabaikan nilai kosong secara otomatis), kemudian menggunakan nilai median tersebut untuk mengisi setiap sel kosong pada kolom yang sama. Median dipilih sebagai ukuran pengisi yang lebih tahan (*robust*) terhadap pengaruh nilai ekstrem dibandingkan rata-rata (*mean*), karena median tidak terpengaruh oleh nilai-nilai yang sangat jauh dari mayoritas data — sebuah pertimbangan yang penting mengingat telah diketahui dari analisis `describe()` sebelumnya bahwa kolom `UnitPrice` mengandung *outlier*.

Untuk data yang bersifat runtut waktu (*time series*), seperti data sensor pabrik yang direkam setiap menit atau setiap jam, terdapat strategi imputasi khusus yang memanfaatkan urutan waktu tersebut, yaitu *forward fill* (mengisi nilai kosong dengan nilai valid terakhir sebelumnya) dan *backward fill* (mengisi dengan nilai valid berikutnya).

```
# Cell: Imputasi forward fill untuk data time series
df.fillna(method='ffill') # mengisi nilai kosong dengan nilai valid
sebelumnya
```

Strategi ini masuk akal secara kontekstual untuk data sensor karena mengasumsikan bahwa kondisi mesin atau lingkungan tidak berubah drastis dalam interval waktu yang sangat singkat, sehingga nilai bacaan sensor terakhir yang valid merupakan estimasi wajar untuk nilai yang hilang sesaat setelahnya. Pendekatan ini akan menjadi sangat relevan pada Studi Kasus Kelompok mengenai data kualitas udara di bagian akhir bab ini, di mana data direkam per jam secara berurutan.

Sebagai catatan penutup yang sangat penting, mahasiswa harus selalu menghindari kebiasaan mengisi nilai kosong secara sembarangan tanpa memahami konteks data terlebih dahulu. Sebagai ilustrasi, pada dataset ritel, `CustomerID` yang hilang kemungkinan besar menandakan transaksi yang dilakukan oleh pembeli tanpa akun keanggotaan (*guest*

checkout) — dalam kasus ini, mengisi CustomerID dengan nilai rata-rata atau median sama sekali tidak masuk akal, karena CustomerID bersifat kategorik-identitas, bukan kuantitas yang dapat dirata-ratakan. Oleh sebab itu, keputusan menghapus baris (bukan mengisi) menjadi pilihan yang lebih tepat untuk kasus tersebut. Prinsip umum yang dapat dipegang adalah: pahami dahulu **mengapa** suatu nilai bisa hilang, baru kemudian tentukan metode penanganan yang paling sesuai dengan alasan tersebut — bukan sekadar memilih metode yang paling mudah diterapkan secara teknis.

9. Visualisasi Data Dasar untuk Eksplorasi

Visualisasi data adalah komponen inti dari EDA yang memungkinkan seorang analis “melihat” pola, sebaran, dan hubungan dalam data secara jauh lebih intuitif dibandingkan hanya membaca angka-angka statistik. Sub-bab ini membahas empat jenis visualisasi dasar yang paling sering digunakan dalam tahap eksplorasi data: histogram, *boxplot*, *scatter plot*, dan *heatmap* korelasi. Sepanjang sub-bab ini, digunakan `df_clean`, yaitu versi dataset `retail_sales.csv` yang telah melalui tahap penanganan *missing value* pada sub-bab sebelumnya.

9.1 Histogram: Memvisualisasikan Distribusi Data

Histogram adalah jenis grafik yang menunjukkan distribusi frekuensi suatu variabel numerik dengan cara membagi rentang nilai variabel tersebut menjadi sejumlah interval (disebut *bin*), lalu menghitung dan menampilkan berapa banyak data yang jatuh pada masing-masing interval dalam bentuk batang vertikal. Histogram menjawab pertanyaan mendasar: “Bagaimana bentuk sebaran nilai suatu variabel? Apakah simetris, condong ke satu sisi, atau memiliki beberapa puncak?”

```
# Cell: Histogram distribusi Quantity
plt.figure(figsize=(10, 5))
df_clean['Quantity'].hist(bins=50, color='steelblue', edgecolor='black')
plt.title('Distribusi Quantity Pembelian')
plt.xlabel('Quantity')
plt.ylabel('Frekuensi')
plt.show()
```

Kode di atas terlebih dahulu memanggil `plt.figure(figsize=(10, 5))` untuk menentukan ukuran kanvas grafik (lebar 10 satuan, tinggi 5 satuan, dalam inci), memastikan grafik yang dihasilkan cukup besar dan mudah dibaca. Metode `.hist()` yang dipanggil langsung pada Series `df_clean['Quantity']` adalah cara paling ringkas untuk membuat histogram di pandas — pandas sesungguhnya memanggil fungsi Matplotlib di baliknya secara otomatis. Parameter `bins=50` menentukan bahwa rentang nilai `Quantity` akan dibagi menjadi 50 interval; semakin banyak jumlah *bin*, semakin detail (namun juga semakin berisik/*noisy*) histogram yang dihasilkan, sehingga pemilihan jumlah *bin* yang tepat memerlukan sedikit eksperimen. Parameter `color` dan `edgecolor` mengatur warna isi dan warna garis tepi setiap batang histogram, murni untuk keperluan estetika dan keterbacaan visual. Tiga baris terakhir menambahkan judul serta label sumbu horizontal dan vertikal, sebuah praktik yang **wajib** dilakukan pada setiap grafik yang dibuat, karena grafik tanpa label yang jelas kehilangan sebagian besar nilai komunikatifnya.

Interpretasi umum dari histogram Quantity pada dataset ini adalah bahwa sebagian besar transaksi memiliki nilai Quantity yang kecil, terkonsentrasi pada rentang 1 hingga 10 unit, sementara terdapat sejumlah kecil transaksi dengan Quantity yang sangat besar, menciptakan bentuk distribusi yang **condong ke kanan** (*right-skewed* atau *positively skewed*) — istilah teknis untuk menggambarkan distribusi yang memiliki “ekor” panjang di sisi kanan akibat keberadaan nilai-nilai besar yang jarang terjadi. Distribusi yang sangat condong semacam ini seringkali dipertimbangkan untuk ditransformasi (misalnya melalui transformasi logaritmik) pada tahap pra-pemrosesan lanjutan, sebuah teknik yang akan dibahas lebih mendalam pada Pertemuan 2 mata kuliah ini.

9.2 Boxplot: Ringkasan Lima Serangkai dan Deteksi Outlier

Boxplot (disebut juga *box-and-whisker plot* atau diagram kotak-garis) adalah visualisasi yang meringkas sebaran data melalui lima nilai statistik utama sekaligus: nilai minimum, kuartil pertama (Q1), median (kuartil kedua), kuartil ketiga (Q3), dan nilai maksimum — dikenal sebagai *ringkasan lima serangkai* (*five-number summary*). Keunggulan utama *boxplot* dibandingkan histogram adalah kemampuannya mengidentifikasi *outlier* secara visual dan eksplisit.

```
# Cell: Boxplot Harga Satuan (UnitPrice)
plt.figure(figsize=(8, 5))
sns.boxplot(x=df_clean['UnitPrice'])
plt.title('Boxplot Harga Satuan')
plt.show()
```

Pada visualisasi *boxplot* yang dihasilkan Seaborn, sebuah kotak (*box*) merepresentasikan rentang antar-kuartil (*Interquartile Range*, disingkat IQR), yaitu rentang antara kuartil pertama (Q1, di mana 25% data berada di bawahnya) hingga kuartil ketiga (Q3, di mana 75% data berada di bawahnya). Garis vertikal di dalam kotak menandai median data. Dua garis yang menjulur dari kotak, disebut *whisker* (kumis), secara default membentang sejauh 1,5 kali nilai IQR dari tepi kotak. Titik-titik individual yang muncul di luar jangkauan *whisker* tersebut dianggap sebagai **outlier** — nilai yang secara statistik dianggap tidak biasa dibandingkan mayoritas data.

Persamaan untuk menentukan batas *outlier* berdasarkan IQR dirumuskan sebagai berikut.

Persamaan 1.1 — Batas Bawah dan Batas Atas Outlier berdasarkan IQR

```
IQR = Q3 - Q1
Batas Bawah = Q1 - 1,5 × IQR
Batas Atas = Q3 + 1,5 × IQR
```

Nilai apa pun yang berada di bawah Batas Bawah atau di atas Batas Atas dianggap sebagai *outlier*. Penerapan rumus ini dalam kode Python adalah sebagai berikut, sekaligus mendemonstrasikan bagaimana *outlier* dapat difilter keluar dari dataset apabila diperlukan untuk analisis lanjutan.

```
# Cell: Menghitung batas outlier dan memfilter data
Q1 = df_clean['UnitPrice'].quantile(0.25)
```

```

Q3 = df_clean['UnitPrice'].quantile(0.75)
IQR = Q3 - Q1
low = Q1 - 1.5 * IQR
high = Q3 + 1.5 * IQR

df_filtered = df_clean[(df_clean['UnitPrice'] >= low) &
(df_clean['UnitPrice'] <= high)]
print("Shape sebelum filtering:", df_clean.shape)
print("Shape sesudah filtering:", df_filtered.shape)

```

Kode di atas menggunakan metode `.quantile(0.25)` dan `.quantile(0.75)` untuk menghitung nilai Q1 dan Q3 secara langsung, kemudian menerapkan Persamaan 1.1 untuk menghitung batas bawah dan batas atas. Baris `df_clean[(df_clean['UnitPrice'] >= low) & (df_clean['UnitPrice'] <= high)]` menunjukkan teknik **pemfilteran boolean** (*boolean filtering* atau *boolean indexing*) pada pandas, salah satu teknik paling penting dan paling sering digunakan dalam manipulasi data: ekspresi di dalam tanda kurung siku menghasilkan sebuah Series bernilai True/False untuk setiap baris (bernilai True jika baris tersebut memenuhi kedua kondisi yang dihubungkan operator `&`, yaitu “dan”), dan pandas hanya akan mempertahankan baris-baris yang bernilai True. Perhatikan bahwa setiap kondisi individual harus diapit tanda kurung terpisah ketika digabungkan dengan operator `&` atau `|` (atau) — sebuah aturan sintaksis pandas yang sering menjadi sumber kesalahan bagi pemula apabila dilupakan. Penerapan teknik pemfilteran outlier semacam ini sangat relevan dalam konteks manufaktur, misalnya untuk memfilter pembacaan sensor suhu mesin yang secara jelas keliru akibat gangguan sinyal, sebelum data tersebut digunakan untuk analisis lebih lanjut.

9.3 Scatter Plot: Mengungkap Hubungan Antar-Variabel

Scatter plot (diagram pencar) adalah visualisasi yang menampilkan hubungan antara dua variabel numerik dengan cara memplot setiap pasangan nilai sebagai satu titik pada bidang dua dimensi, di mana satu variabel menempati sumbu horizontal dan variabel lainnya menempati sumbu vertikal.

```

# Cell: Scatter plot antara Quantity dan UnitPrice
plt.figure(figsize=(8, 6))
plt.scatter(df_clean['Quantity'], df_clean['UnitPrice'], alpha=0.3)
plt.xlabel('Quantity')
plt.ylabel('Unit Price')
plt.title('Quantity vs UnitPrice')
plt.show()

```

Fungsi `plt.scatter()` menerima dua argumen posisi utama: nilai-nilai untuk sumbu horizontal (Quantity) dan nilai-nilai untuk sumbu vertikal (UnitPrice). Parameter `alpha=0.3` mengatur tingkat transparansi setiap titik menjadi 30% opak — parameter ini sangat penting ketika memplot dataset berukuran besar, karena tanpa transparansi, ribuan titik yang saling bertumpuk (*overplotting*) akan tampak sebagai satu massa gelap yang tidak informatif; dengan transparansi, area yang memiliki kepadatan titik tinggi akan

tampak lebih gelap secara alami dibandingkan area dengan kepadatan rendah, memberikan indikasi visual tambahan mengenai konsentrasi data.

Interpretasi terhadap *scatter plot* Quantity versus UnitPrice pada dataset ini umumnya menunjukkan bahwa tidak terdapat pola hubungan linear yang kuat dan jelas antara kedua variabel tersebut — titik-titik data tersebar tanpa membentuk garis atau kurva yang tegas. Namun demikian, pengamatan yang lebih cermat dapat mengungkap kecenderungan tertentu, misalnya titik-titik dengan UnitPrice yang sangat tinggi cenderung memiliki Quantity yang kecil, sementara titik-titik dengan Quantity sangat besar (kemungkinan transaksi grosir) cenderung memiliki UnitPrice yang relatif rendah — sebuah pola yang secara bisnis masuk akal, karena pembelian dalam jumlah besar seringkali diasosiasikan dengan produk bernilai satuan rendah. Ketiadaan pola linear yang kuat pada *scatter plot* semacam ini juga menjadi petunjuk awal yang berguna: apabila kelak mahasiswa membangun model regresi linear sederhana untuk memprediksi salah satu variabel dari variabel lainnya (topik yang akan dibahas pada Pertemuan 3), hasil *scatter plot* semacam ini mengindikasikan bahwa model linear sederhana kemungkinan tidak akan memberikan performa prediktif yang baik untuk pasangan variabel ini.

9.4 Heatmap Korelasi: Mengukur Kekuatan Hubungan Antar-Variabel Secara Kuantitatif

Sementara *scatter plot* memvisualisasikan hubungan antara dua variabel secara visual, **korelasi** memberikan ukuran kuantitatif atas kekuatan dan arah hubungan linear antara dua variabel numerik, dinyatakan dalam sebuah angka yang disebut **koefisien korelasi**, berkisar antara -1 hingga +1. Koefisien korelasi mendekati +1 menunjukkan hubungan linear positif yang kuat (ketika satu variabel naik, variabel lainnya cenderung ikut naik); koefisien mendekati -1 menunjukkan hubungan linear negatif yang kuat (ketika satu variabel naik, variabel lainnya cenderung turun); sedangkan koefisien mendekati 0 menunjukkan tidak adanya hubungan linear yang berarti antara kedua variabel.

```
# Cell: Menghitung matriks korelasi dan menampilkan heatmap
corr = df_clean.corr(numeric_only=True)
plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Heatmap Korelasi')
plt.show()
```

Metode `df_clean.corr(numeric_only=True)` menghitung koefisien korelasi Pearson antara setiap pasangan kolom numerik dalam dataset, menghasilkan sebuah matriks persegi di mana baris dan kolomnya sama-sama merepresentasikan nama-nama variabel numerik, dan setiap sel berisi koefisien korelasi antara pasangan variabel yang bersangkutan. Parameter `numeric_only=True` memastikan hanya kolom numerik yang disertakan dalam perhitungan, mengabaikan kolom kategorik yang tidak relevan secara matematis untuk perhitungan korelasi. Fungsi `sns.heatmap()` kemudian mengubah matriks angka tersebut menjadi visualisasi berwarna, di mana parameter `annot=True` menampilkan nilai koefisien korelasi secara numerik langsung di atas setiap sel warna, parameter `cmap='coolwarm'` menentukan skema warna (nilai negatif cenderung berwarna biru/dingin, nilai positif cenderung berwarna merah/hangat), dan parameter `vmin=-1, vmax=1` memastikan skala warna konsisten dengan rentang teoretis koefisien korelasi.

Heatmap korelasi menjadi alat yang sangat efisien untuk memindai (*scan*) puluhan bahkan ratusan pasangan variabel numerik sekaligus dalam satu visualisasi tunggal, jauh lebih cepat dibandingkan membuat *scatter plot* terpisah untuk setiap pasangan variabel. Dalam konteks industri manufaktur, teknik ini sangat berguna misalnya untuk memindai data sensor mesin yang memiliki puluhan parameter berbeda (suhu, tekanan, getaran, kecepatan putaran, dan sebagainya), dengan cepat mengidentifikasi pasangan parameter mana yang saling berkorelasi kuat — informasi yang penting baik untuk pemahaman proses fisik mesin maupun untuk menghindari redundansi fitur ketika membangun model prediktif pada pertemuan-pertemuan berikutnya.

Sebagai catatan penting yang harus selalu diingat mahasiswa, korelasi yang tinggi antara dua variabel **tidak serta-merta membuktikan** hubungan sebab-akibat (*causation*) di antara keduanya — prinsip yang dikenal luas dengan ungkapan “*correlation does not imply causation*”. Dua variabel dapat berkorelasi tinggi karena kebetulan, karena keduanya sama-sama dipengaruhi oleh variabel ketiga yang tidak teramati (*confounding variable*), atau karena hubungan yang sesungguhnya bersifat tidak langsung. Sebagai ilustrasi dalam konteks manufaktur, suhu mesin dan jumlah cacat produk mungkin menunjukkan korelasi tinggi, namun hal ini tidak secara otomatis berarti suhu tinggi *menyebabkan* cacat — bisa jadi keduanya sama-sama dipengaruhi oleh faktor lain seperti usia komponen mesin yang sudah aus. Kehati-hatian dalam menafsirkan korelasi ini merupakan salah satu kompetensi analitis kritis yang harus dimiliki oleh setiap praktisi data.

10. Sintesis: Merangkai Alur Kerja EDA yang Utuh

Setelah mempelajari setiap komponen EDA secara terpisah — mulai dari inspeksi struktur data, statistik deskriptif, penanganan *missing value*, hingga empat jenis visualisasi dasar — penting bagi mahasiswa untuk memahami bagaimana seluruh komponen tersebut dirangkai menjadi satu **alur kerja EDA yang koheren** dan bagaimana urutan pengerjaannya biasanya berlangsung dalam praktik nyata. Alur kerja standar yang telah didemonstrasikan sepanjang bab ini dapat diringkas menjadi enam tahap berurutan: (1) memuat data mentah ke dalam `DataFrame`; (2) melihat sekilas bentuk data melalui `head()`, `tail()`, dan `shape`; (3) memeriksa struktur dan tipe data melalui `info()` dan `dtypes`; (4) menghitung statistik deskriptif melalui `describe()` dan mengukur *missing value* melalui `isnull().sum()`; (5) menangani *missing value* yang teridentifikasi melalui `dropna()` atau `fillna()` sesuai konteks; dan (6) membangun visualisasi — histogram, *boxplot*, *scatter plot*, dan *heatmap* korelasi — untuk menggali wawasan dan pola tersembunyi dalam data.

Alur kerja ini bukanlah proses yang kaku dan satu arah; dalam praktiknya, seorang analis data seringkali harus kembali ke tahap sebelumnya setelah menemukan sesuatu pada tahap selanjutnya. Sebagai contoh, ketika histogram pada tahap keenam mengungkap distribusi yang sangat condong akibat *outlier* ekstrem, seorang analis mungkin perlu kembali ke tahap kelima untuk menerapkan strategi penanganan *outlier* tambahan sebelum melanjutkan visualisasi lain. Sifat iteratif dan eksploratif inilah yang menjadi ciri khas EDA, membedakannya dari analisis data yang bersifat konfirmatif dan linear.

Dalam konteks industri manufaktur, alur kerja EDA ini menjadi tahap yang tidak dapat dilewati sebelum membangun sistem analitik atau prediktif apa pun. Sebelum sebuah tim

rekayasa data membangun sistem *predictive maintenance* yang memprediksi kegagalan mesin, mereka terlebih dahulu harus memahami data sensor historis mesin tersebut secara mendalam melalui EDA — berapa banyak data yang hilang akibat sensor yang pernah rusak, bagaimana distribusi normal suhu dan getaran mesin dalam kondisi sehat, parameter mana saja yang saling berkorelasi, dan pola-pola tidak biasa apa yang pernah tercatat menjelang kejadian kegagalan di masa lalu. Tanpa pemahaman mendalam ini, model prediktif seanggih apa pun berisiko dibangun di atas asumsi yang keliru mengenai data, sehingga menghasilkan prediksi yang tidak dapat diandalkan di lapangan. Dengan demikian, penguasaan menyeluruh terhadap materi bab ini bukan sekadar prasyarat administratif untuk melanjutkan ke Pertemuan 2, melainkan fondasi kompetensi yang akan terus digunakan dan diasah sepanjang sisa mata kuliah ini, bahkan sepanjang karier mahasiswa sebagai praktisi data di industri manufaktur.

LATIHAN MANDIRI TERBIMBING

Bagian ini menyajikan latihan praktik yang harus dikerjakan mahasiswa secara mandiri, mengikuti langkah-langkah yang disusun secara sangat rinci sehingga dapat diselesaikan tanpa kehadiran dosen. Latihan ini mengubah rangkaian sel notebook yang biasanya didemonstrasikan langsung oleh dosen di kelas menjadi instruksi tertulis lengkap. Kerjakan setiap langkah secara berurutan pada sebuah *notebook* baru bernama `P1_Terbimbing>NamaAnda.ipynb`, tanpa melompati satu langkah pun, karena hasil dari satu langkah menjadi masukan bagi langkah berikutnya. Dataset yang digunakan adalah `retail_sales.csv`, sebagaimana telah diperkenalkan pada bagian Penyajian Materi.

Langkah 1 — Menyiapkan Lingkungan dan Mengimpor Pustaka

Tujuan: Memastikan seluruh pustaka yang dibutuhkan siap digunakan sebelum memulai analisis.

Instruksi: Buka Jupyter Notebook (lokal) atau Google Colab, buat *notebook* baru, lalu pada sel pertama tuliskan dan jalankan kode berikut.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline
sns.set_style('whitegrid')
print("Setup selesai.")
```

Penjelasan hasil yang diharapkan: Apabila seluruh pustaka telah terpasang dengan benar, sel ini akan mencetak teks `Setup selesai.` tanpa menampilkan pesan galat apa pun. Perintah `%matplotlib inline` adalah sebuah *magic command* khusus Jupyter/Colab (ditandai simbol persen di awal) yang memastikan setiap grafik yang dibuat pada sel-sel selanjutnya akan langsung ditampilkan tepat di bawah sel kode yang bersangkutan, bukan pada jendela terpisah. Perintah `sns.set_style('whitegrid')` mengatur gaya visual default seluruh grafik Seaborn pada *notebook* ini menjadi berlatar putih dengan garis kisi tipis, sebuah gaya yang umum digunakan karena memudahkan pembacaan nilai pada sumbu grafik. Jika muncul pesan `ModuleNotFoundError`, kembali ke Sub-bab 2.3 pada bagian Penyajian Materi untuk memasang pustaka yang belum tersedia.

Langkah 2 — Memuat Dataset Retail Sales

Tujuan: Memuat berkas `retail_sales.csv` ke dalam struktur `DataFrame` `pandas` dan memastikan proses pemuatan berhasil.

Instruksi: Pastikan berkas `retail_sales.csv` telah diunduh dari tautan yang dibagikan dosen dan diletakkan pada folder yang sama dengan *notebook* Anda (atau diunggah ke sesi Google Colab sebagaimana dijelaskan pada Sub-bab 6.2). Jalankan kode berikut pada sel baru.

```
# Ganti path sesuai lokasi file pada komputer/sesi Anda
df = pd.read_csv('retail_sales.csv')
df.head()
```

Penjelasan hasil yang diharapkan: Sel ini akan menampilkan sebuah tabel berisi lima baris pertama dataset, dengan delapan kolom: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, dan Country. Periksa apakah nilai-nilai pada setiap kolom tampak wajar — misalnya, Quantity dan UnitPrice berupa angka, sedangkan Country berupa nama negara. Apabila muncul pesan galat `FileNotFoundError`, periksa kembali apakah nama berkas dituliskan secara tepat (termasuk huruf besar/kecil) dan apakah berkas benar-benar berada pada lokasi yang diakses oleh perintah `pd.read_csv()`.

Langkah 3 — Inspeksi Awal Struktur dan Statistik Dataset

Tujuan: Memahami struktur, tipe data, dan sebaran statistik dataset sebelum melakukan pembersihan.

Instruksi: Jalankan kedua perintah berikut pada sel-sel terpisah agar keluaran masing-masing dapat diamati secara jelas.

```
df.info()
df.describe()
```

Penjelasan hasil yang diharapkan: Keluaran `df.info()` menunjukkan bahwa dataset memiliki 5.000 baris dan 8 kolom, dengan kolom CustomerID menunjukkan jumlah *non-null* yang lebih kecil dibandingkan kolom lain (sekitar 4.000), mengindikasikan keberadaan *missing value* yang perlu ditangani. Keluaran `df.describe()` menampilkan statistik deskriptif untuk kolom numerik Quantity, UnitPrice, dan CustomerID; amati secara khusus perbedaan antara nilai mean dan max pada kolom Quantity dan UnitPrice — perbedaan yang besar mengindikasikan keberadaan nilai ekstrem yang akan dieksplorasi lebih lanjut pada langkah visualisasi.

Langkah 4 — Mengukur Missing Value secara Presisi

Tujuan: Menghitung jumlah dan persentase *missing value* pada setiap kolom untuk menjadi dasar keputusan penanganan.

Instruksi: Jalankan kode berikut.

```
missing = df.isnull().sum()
missing_pct = (df.isnull().sum() / len(df)) * 100
missing_df = pd.DataFrame({'Missing': missing, 'Percent': missing_pct})
missing_df[missing_df['Missing'] > 0]
```

Penjelasan hasil yang diharapkan: Tabel yang dihasilkan hanya menampilkan kolom yang memiliki nilai kosong, yaitu CustomerID (sekitar 1.000 baris, 20%) dan Description (sekitar 10 baris, 0,2%). Catat kedua angka ini, karena akan menjadi dasar pertimbangan pada langkah berikutnya: kolom dengan persentase *missing* yang besar (CustomerID)

memerlukan pertimbangan berbeda dibandingkan kolom dengan persentase yang sangat kecil (Description).

Langkah 5 — Menangani Missing Value

Tujuan: Menerapkan strategi penanganan *missing value* yang berbeda untuk dua kolom dengan karakteristik missing yang berbeda pula.

Instruksi: Jalankan kode berikut, yang menggabungkan strategi penghapusan untuk CustomerID dan strategi pengisian untuk Description.

```
# Drop baris yang CustomerID-nya kosong
df_clean = df.dropna(subset=['CustomerID']).copy()

# Isi missing pada Description dengan 'Unknown'
df_clean['Description'] = df_clean['Description'].fillna('Unknown')

# Verifikasi ulang
df_clean.isnull().sum()
```

Penjelasan hasil yang diharapkan: Perintah `.copy()` di akhir baris pertama sengaja ditambahkan untuk memastikan `df_clean` menjadi salinan data yang independen dari `df`, sehingga modifikasi selanjutnya pada `df_clean` (seperti pengisian kolom Description) tidak memicu peringatan `SettingWithCopyWarning` yang umum muncul pada pandas apabila memodifikasi hasil filter tanpa penyalinan eksplisit. Verifikasi terakhir dengan `df_clean.isnull().sum()` seharusnya menunjukkan angka nol pada seluruh kolom, mengonfirmasi bahwa seluruh nilai kosong telah tertangani. Bandingkan pula `df_clean.shape` dengan `df.shape` — jumlah baris pada `df_clean` akan lebih sedikit (sekitar 4.000 dibandingkan 5.000), sebagai konsekuensi langsung dari penghapusan baris pada langkah `dropna()`.

Langkah 6 — Visualisasi Pertama: Histogram Distribusi Quantity

Tujuan: Memvisualisasikan sebaran nilai Quantity setelah data dibersihkan.

Instruksi: Jalankan kode berikut.

```
plt.figure(figsize=(10, 5))
df_clean['Quantity'].hist(bins=50, color='steelblue', edgecolor='black')
plt.title('Distribusi Quantity (Setelah Cleaning)')
plt.xlabel('Quantity')
plt.ylabel('Frekuensi')
plt.show()
```

Penjelasan hasil yang diharapkan: Grafik yang muncul menunjukkan mayoritas batang histogram terkonsentrasi pada rentang nilai Quantity yang kecil (mendekati nol hingga puluhan), dengan sejumlah kecil batang yang sangat rendah dan tersebar jauh di sisi kanan sumbu horizontal — pola distribusi condong ke kanan (*right-skewed*) sebagaimana telah dijelaskan pada Sub-bab 9.1. Tuliskan pada sel Markdown di bawah grafik ini sebuah

kalimat interpretasi, misalnya: “Sebagian besar transaksi memiliki jumlah pembelian antara 1 hingga 10 unit, dengan sejumlah kecil transaksi grosir yang jumlahnya jauh lebih besar.”

Langkah 7 — Visualisasi Kedua: Boxplot UnitPrice

Tujuan: Mengidentifikasi sebaran dan outlier pada kolom UnitPrice.

Instruksi: Jalankan kode berikut.

```
plt.figure(figsize=(8, 4))
sns.boxplot(x=df_clean['UnitPrice'], color='lightgreen')
plt.title('Boxplot UnitPrice')
plt.show()
```

Penjelasan hasil yang diharapkan: Kotak *boxplot* akan tampak relatif sempit dan terkonsentrasi di dekat nilai nol pada sumbu horizontal, sementara sejumlah besar titik individual muncul jauh di sisi kanan, menandakan banyak *outlier* bernilai tinggi. Fenomena ini merupakan hal yang wajar pada data harga ritel, di mana mayoritas produk bernilai murah namun terdapat sejumlah kecil produk bernilai jauh lebih tinggi.

Langkah 8 — Memfilter Outlier Berdasarkan IQR (Opsional namun Disarankan)

Tujuan: Mempersempit rentang data agar visualisasi selanjutnya lebih mudah dibaca, dengan menyaring nilai UnitPrice yang berada di luar batas wajar berdasarkan IQR.

Instruksi: Jalankan kode berikut.

```
Q1 = df_clean['UnitPrice'].quantile(0.25)
Q3 = df_clean['UnitPrice'].quantile(0.75)
IQR = Q3 - Q1
low = Q1 - 1.5 * IQR
high = Q3 + 1.5 * IQR

df_filtered = df_clean[(df_clean['UnitPrice'] >= low) &
(df_clean['UnitPrice'] <= high)]
print("Shape sebelum filtering:", df_clean.shape)
print("Shape sesudah filtering:", df_filtered.shape)

sns.boxplot(x=df_filtered['UnitPrice'])
plt.title('Boxplot UnitPrice Setelah Filtering')
plt.show()
```

Penjelasan hasil yang diharapkan: Bandingkan angka shape sebelum dan sesudah filtering yang tercetak di layar — jumlah baris pada `df_filtered` akan lebih sedikit dibandingkan `df_clean`, karena baris-baris dengan UnitPrice di luar batas IQR telah disaring keluar. *Boxplot* kedua yang dihasilkan akan tampak lebih “rapat” dan proporsional dibandingkan *boxplot* pertama pada Langkah 7, karena rentang nilai ekstrem telah dihilangkan, sehingga kotak dan *whisker* menjadi lebih mudah diinterpretasikan secara visual. Penting dicatat bahwa `df_filtered` dibuat sebagai dataset terpisah khusus untuk

keperluan visualisasi yang lebih jernih; keputusan apakah *outlier* tersebut sesungguhnya perlu dihapus secara permanen dari analisis bergantung pada tujuan analisis itu sendiri dan sebaiknya didiskusikan dengan pemilik proses bisnis atau produksi terkait.

Langkah 9 — Visualisasi Ketiga: Scatter Plot Quantity vs UnitPrice

Tujuan: Mengeksplorasi kemungkinan hubungan antara jumlah pembelian dan harga satuan menggunakan data yang telah difilter.

Instruksi: Jalankan kode berikut.

```
plt.figure(figsize=(8, 6))
plt.scatter(df_filtered['Quantity'], df_filtered['UnitPrice'], alpha=0.3)
plt.xlabel('Quantity')
plt.ylabel('Unit Price')
plt.title('Quantity vs UnitPrice (Filtered)')
plt.show()
```

Penjelasan hasil yang diharapkan: Titik-titik pada grafik akan tersebar tanpa membentuk pola linear yang tegas, meskipun sedikit kecenderungan dapat diamati bahwa titik dengan Quantity besar cenderung memiliki UnitPrice yang rendah. Karena data telah difilter dari outlier ekstrem pada langkah sebelumnya, sebaran titik akan tampak lebih terkonsentrasi dan mudah diamati dibandingkan jika menggunakan data mentah yang belum difilter.

Langkah 10 — Visualisasi Keempat: Heatmap Korelasi

Tujuan: Mengukur secara kuantitatif kekuatan hubungan linear antar-variabel numerik.

Instruksi: Jalankan kode berikut.

```
num_cols = ['Quantity', 'UnitPrice', 'CustomerID']
corr = df_filtered[num_cols].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title('Heatmap Korelasi')
plt.show()
```

Penjelasan hasil yang diharapkan: Matriks *heatmap* berukuran 3×3 akan menampilkan koefisien korelasi antara Quantity, UnitPrice, dan CustomerID. Diagonal utama matriks akan selalu bernilai 1,00, karena setiap variabel tentu berkorelasi sempurna dengan dirinya sendiri. Perhatikan nilai korelasi antara Quantity dan UnitPrice, yang umumnya akan bernilai negatif namun kecil (mendekati nol), mengonfirmasi kesimpulan dari *scatter plot* pada Langkah 9 bahwa hubungan linear antara kedua variabel tersebut lemah. Korelasi CustomerID dengan variabel lain umumnya juga mendekati nol dan secara substantif tidak bermakna, karena CustomerID adalah data pengenalan (*identifier*) yang nilainya bersifat arbitrer, bukan besaran kuantitatif yang memiliki makna matematis untuk dikorelasikan.

Langkah 11 — Analisis Tambahan: Sepuluh Negara dengan Transaksi Terbanyak

Tujuan: Melatih kemampuan analisis frekuensi pada data kategorik dan memvisualisasikannya menggunakan grafik batang.

Instruksi: Jalankan kode berikut.

```
top_countries = df_filtered['Country'].value_counts().head(10)
plt.figure(figsize=(10, 6))
top_countries.plot(kind='bar', color='salmon')
plt.title('Top 10 Negara dengan Transaksi Terbanyak')
plt.xlabel('Negara')
plt.ylabel('Jumlah Transaksi')
plt.xticks(rotation=45)
plt.show()
```

Penjelasan hasil yang diharapkan: Grafik batang yang dihasilkan akan menunjukkan sepuluh negara dengan jumlah transaksi terbanyak, diurutkan dari yang tertinggi ke terendah, dengan negara asal toko (umumnya United Kingdom pada dataset ini) mendominasi secara signifikan dibandingkan negara-negara lain. Parameter `plt.xticks(rotation=45)` memutar label nama negara pada sumbu horizontal sebesar 45 derajat agar tidak saling tumpang tindih ketika nama negara cukup panjang. Latihan ini menunjukkan bahwa metode `.plot()` dapat dipanggil langsung pada hasil `value_counts()`, sebuah kombinasi yang sangat sering digunakan untuk memvisualisasikan data kategorik secara cepat.

Langkah 12 — Menyimpulkan dan Mendokumentasikan Temuan

Tujuan: Melatih kemampuan menuliskan sintesis naratif dari seluruh proses eksplorasi yang telah dilakukan.

Instruksi: Tambahkan sebuah sel Markdown di bagian akhir *notebook*, kemudian tuliskan sekurang-kurangnya lima kalimat yang merangkum temuan utama dari latihan ini — misalnya mengenai distribusi Quantity, keberadaan outlier pada UnitPrice, kekuatan hubungan antar-variabel, dan negara dengan transaksi terbanyak. Simpan *notebook* dengan nama `P1_Terbimbing>NamaAnda.ipynb`.

Penjelasan mengapa langkah ini penting: Kemampuan menyintesis temuan teknis menjadi narasi yang dapat dipahami oleh pemangku kepentingan non-teknis (seperti manajer produksi atau kepala pabrik) adalah salah satu kompetensi yang membedakan seorang analis data yang baik dari sekadar operator perangkat lunak. Latihan mendokumentasikan kesimpulan pada setiap tahap analisis akan terus dilatih sepanjang mata kuliah ini dan menjadi bagian penting dari kriteria penilaian pada Studi Kasus Mandiri dan Studi Kasus Kelompok yang akan dibahas selanjutnya.

Selamat, Anda telah menyelesaikan Latihan Mandiri Terbimbing. Simpan *notebook* ini dengan baik, karena struktur dan pola kerja yang telah dilatih pada latihan ini akan menjadi rujukan utama ketika mengerjakan Studi Kasus Mandiri dan Studi Kasus Kelompok pada bagian selanjutnya.

STUDI KASUS MANDIRI

Bagian ini berisi tiga studi kasus yang harus dikerjakan mahasiswa secara **individu**, tanpa diberikan kode langkah demi langkah sebagaimana pada Latihan Mandiri Terbimbing. Tujuannya adalah menguji sejauh mana mahasiswa mampu menerapkan keterampilan EDA yang telah dipelajari pada dataset baru yang belum pernah dijumpai sebelumnya, sekaligus melatih kemandirian dalam mengambil keputusan analitis. Mahasiswa dapat memilih untuk mengerjakan ketiga studi kasus ini sebagai latihan tambahan, atau mengerjakan salah satu sesuai penugasan dosen. Setiap studi kasus dilengkapi deskripsi masalah, deskripsi dataset, tugas rinci, dan rubrik penilaian.

Studi Kasus Mandiri 1 — Eksplorasi Data Produksi Harian Pabrik

Deskripsi Masalah dan Konteks

Sebuah pabrik manufaktur komponen otomotif mengoperasikan empat lini mesin produksi (diberi label A, B, C, dan D) yang berjalan dalam tiga shift kerja setiap harinya, yaitu Pagi, Siang, dan Malam. Bagian pengendalian mutu dan perencanaan produksi pabrik tersebut mencatat data operasional harian dari setiap mesin, namun data tersebut selama ini hanya disimpan sebagai arsip tanpa pernah dianalisis secara sistematis. Manajemen pabrik ingin mengetahui pola-pola tersembunyi dalam data tersebut — misalnya, apakah ada shift kerja tertentu yang cenderung menghasilkan lebih banyak produk cacat, apakah suhu mesin berhubungan dengan tingkat kecacatan produk, dan mesin mana yang paling sering mengalami downtime (waktu henti operasi). Sebagai calon praktisi data di lingkungan industri, Anda ditugaskan melakukan eksplorasi data awal atas data ini untuk memberikan gambaran awal yang berguna bagi pengambilan keputusan manajemen.

Deskripsi Dataset

Dataset `factory_production.csv` berisi catatan produksi harian dengan delapan kolom sebagai berikut.

Nama Kolom	Keterangan
Tanggal	Tanggal pencatatan produksi
Shift	Shift kerja: Pagi, Siang, atau Malam
Mesin	Kode identitas mesin: A, B, C, atau D
Jumlah_Produksi	Jumlah unit produk yang dihasilkan pada shift tersebut
Jumlah_Cacat	Jumlah unit produk yang terdeteksi cacat pada shift tersebut
Downtime (menit)	Total waktu henti operasi mesin dalam satuan menit
Suhu_Mesin	Suhu rata-rata mesin selama shift berlangsung, dalam derajat Celsius
Kelembaban	Kelembaban relatif ruang produksi selama shift berlangsung, dalam persen

Tugas Rinci yang Harus Dikerjakan

1. Buat *notebook* baru bernama P1_Mandiri_NIM.ipynb dan muat dataset `factory_production.csv`, lalu tampilkan sepuluh baris pertama menggunakan `df.head(10)`.
2. Lakukan inspeksi menyeluruh menggunakan `df.info()`, `df.describe()`, dan `df.isnull().sum()` untuk memahami struktur, statistik, dan keberadaan *missing value* pada dataset.
3. Tangani *missing value* yang teridentifikasi menggunakan metode yang menurut Anda paling tepat (boleh berupa penghapusan atau imputasi), dan tuliskan pada sel Markdown alasan pemilihan metode tersebut berdasarkan konteks kolom yang bersangkutan.
4. Buat **sekurang-kurangnya tiga visualisasi berbeda**, dipilih dari opsi berikut: histogram `Jumlah_Produksi` atau `Downtime (menit)`; *boxplot* `Jumlah_Cacat` dikelompokkan berdasarkan `Shift` atau `Mesin` (gunakan parameter `x` dan `y` pada `sns.boxplot()` untuk mengelompokkan, misalnya `sns.boxplot(data=df, x='Shift', y='Jumlah_Cacat')`); *scatter plot* antara `Suhu_Mesin` dan `Jumlah_Cacat`; atau *heatmap* korelasi seluruh kolom numerik.
5. Di bawah setiap visualisasi, tuliskan dua hingga tiga kalimat insight yang menjelaskan apa yang dapat disimpulkan dari visualisasi tersebut.
6. Simpan dan kumpulkan *notebook* sesuai format pengumpulan yang ditentukan dosen.

Kriteria Penilaian (Rubrik Singkat)

Komponen Penilaian	Bobot
Ketepatan memuat data dan kelengkapan inspeksi awal	20%
Ketepatan penanganan <i>missing value</i> beserta justifikasi alasan	20%
Keragaman dan ketepatan teknis visualisasi (minimal 3 jenis)	30%
Kedalaman dan ketajaman insight yang dituliskan	30%

Studi Kasus Mandiri 2 — Eksplorasi Data Downtime Mesin Produksi

Deskripsi Masalah dan Konteks

Departemen perawatan (*maintenance*) sebuah pabrik pengolahan logam mencatat setiap kejadian *downtime* (waktu henti tak terjadwal) pada sepuluh unit mesin produksi selama periode enam bulan terakhir. Setiap kali sebuah mesin berhenti beroperasi di luar jadwal, operator wajib mencatat kejadian tersebut pada sistem pencatatan digital, termasuk penyebab utama yang diperkirakan menjadi sumber gangguan. Manajer perawatan pabrik curiga bahwa sebagian besar downtime disebabkan oleh beberapa jenis kegagalan yang sama secara berulang, namun belum ada analisis data yang membuktikan dugaan tersebut. Anda diminta melakukan eksplorasi data untuk mengidentifikasi pola downtime — mesin mana yang paling sering bermasalah, penyebab kegagalan apa yang paling dominan, dan apakah terdapat hubungan antara durasi downtime dan variabel operasional lain seperti usia mesin atau beban kerja harian.

Deskripsi Dataset

Dataset `machine_downtime.csv` berisi catatan setiap kejadian downtime dengan sembilan kolom sebagai berikut.

Nama Kolom	Keterangan
ID_Kejadian	Nomor identitas unik untuk setiap kejadian downtime
Tanggal_Kejadian	Tanggal terjadinya downtime
ID_Mesin	Kode identitas mesin (M01 hingga M10)
Usia_Mesin_Tahun	Usia mesin dalam satuan tahun sejak pertama kali dioperasikan
Penyebab_Downtime	Kategori penyebab: Kerusakan Mekanis, Kerusakan Elektrikal, Kekurangan Bahan Baku, Pergantian Alat (<i>Tooling Change</i>), atau Kesalahan Operator
Durasi_Downtime_Menit	Lama waktu henti mesin dalam satuan menit
Beban_Kerja_Harian_Persen	Persentase pemanfaatan kapasitas mesin pada hari kejadian, dalam persen
Shift_Kejadian	Shift saat kejadian terjadi: Pagi, Siang, atau Malam
Biaya_Perbaikan_Ribu_Rupiah	Estimasi biaya perbaikan akibat kejadian tersebut, dalam ribuan rupiah

Catatan penting: sebagian baris pada kolom `Biaya_Perbaikan_Ribu_Rupiah` bernilai kosong karena estimasi biaya belum sempat dicatat oleh teknisi untuk kejadian-kejadian tertentu, dan sebagian kecil baris pada kolom `Usia_Mesin_Tahun` tercatat sebagai nilai negatif akibat kesalahan entri data yang perlu diidentifikasi dan ditangani sebagai bagian dari proses pembersihan data.

Tugas Rinci yang Harus Dikerjakan

1. Buat *notebook* baru bernama P1_Mandiri2_NIM.ipynb, muat dataset `machine_downtime.csv`, dan tampilkan dimensi data (`shape`) beserta sepuluh baris pertama.
2. Lakukan inspeksi struktur data dan tipe data setiap kolom menggunakan `df.info()` dan `df.dtypes`, lalu identifikasi kolom mana yang seharusnya bertipe kategorik namun mungkin terbaca sebagai tipe lain.
3. Identifikasi dua jenis persoalan kualitas data pada dataset ini: *missing value* pada kolom `Biaya_Perbaikan_Ribu_Rupiah`, dan nilai tidak logis (usia mesin negatif) pada kolom `Usia_Mesin_Tahun`. Tangani kedua persoalan tersebut secara terpisah dengan metode yang sesuai, dan jelaskan alasan Anda secara tertulis. Sebagai petunjuk, nilai `Usia_Mesin_Tahun` yang negatif sebaiknya tidak diisi begitu saja dengan rata-rata, karena itu adalah indikasi kesalahan pencatatan, bukan nilai yang benar-benar hilang secara acak; pertimbangkan apakah baris tersebut sebaiknya dikoreksi (misalnya dengan mengambil nilai absolutnya jika masuk akal) atau dihapus.
4. Buat **sekurang-kurangnya tiga visualisasi berbeda** yang mengungkap pola downtime, misalnya: grafik batang frekuensi Penyebab_Downtime untuk mengetahui penyebab paling dominan; *boxplot* Durasi_Downtime_Menit dikelompokkan berdasarkan ID_Mesin untuk mengidentifikasi mesin paling bermasalah; atau *scatter plot* antara Usia_Mesin_Tahun dan Durasi_Downtime_Menit untuk menguji dugaan bahwa mesin yang lebih tua mengalami downtime lebih lama.
5. Tuliskan interpretasi naratif di bawah setiap visualisasi, termasuk rekomendasi awal yang dapat disampaikan kepada manajer perawatan berdasarkan temuan Anda.
6. Simpan dan kumpulkan *notebook* sesuai format yang ditentukan dosen.

Kriteria Penilaian (Rubrik Singkat)

Komponen Penilaian	Bobot
Ketepatan memuat data dan kelengkapan inspeksi struktur/tipe data	15%
Ketepatan mengidentifikasi dan menangani dua jenis persoalan kualitas data (<i>missing value</i> dan nilai tidak logis) beserta justifikasi	25%
Keragaman dan ketepatan teknis visualisasi (minimal 3 jenis)	30%
Kedalaman insight dan relevansi rekomendasi bagi konteks perawatan mesin	30%

Studi Kasus Mandiri 3 — Eksplorasi Data Sensor Kualitas Produk pada Lini Perakitan Elektronik

Deskripsi Masalah dan Konteks

Sebuah pabrik perakitan komponen elektronik memasang sensor otomatis pada stasiun uji akhir (*final test station*) untuk merekam beberapa parameter kelistrikan dari setiap unit produk yang telah selesai dirakit, sebelum unit tersebut dinyatakan lolos uji dan siap dikirim ke pelanggan. Setiap unit yang melewati stasiun uji ini diberi status akhir “Lolos” atau “Gagal” berdasarkan ambang batas yang ditetapkan oleh standar mutu perusahaan. Tim rekayasa kualitas ingin memahami karakteristik data hasil pengujian ini sebagai langkah awal sebelum membangun model klasifikasi otomatis pada pertemuan-pertemuan mendatang yang dapat memprediksi kegagalan produk lebih dini dalam proses produksi. Sebagai tahap pertama, Anda diminta melakukan eksplorasi data menyeluruh terhadap data hasil pengujian tersebut.

Deskripsi Dataset

Dataset `sensor_kualitas_elektronik.csv` berisi hasil pengujian akhir dengan tujuh kolom sebagai berikut.

Nama Kolom	Keterangan
ID_Unit	Nomor identitas unik untuk setiap unit produk yang diuji
Tegangan_Output_Volt	Tegangan keluaran yang terukur pada unit, dalam satuan volt
Arus_Output_Ampere	Arus keluaran yang terukur pada unit, dalam satuan ampere
Suhu_Komponen_Celcius	Suhu komponen utama saat pengujian berlangsung, dalam derajat Celsius
Waktu_Uji_Detik	Lama waktu yang dibutuhkan untuk menyelesaikan satu siklus pengujian, dalam detik
Line_Perakitan	Kode lini perakitan tempat unit tersebut dirakit: L1, L2, atau L3
Status_Akhir	Hasil akhir pengujian: “Lolos” atau “Gagal”

Catatan penting: kolom `Tegangan_Output_Volt` dan `Arus_Output_Ampere` memiliki sejumlah nilai kosong akibat kegagalan pembacaan sensor pada saat-saat tertentu (kegagalan sensor bersifat acak dan tidak berhubungan dengan kualitas unit yang diuji), sehingga strategi imputasi menggunakan ukuran pemusatan data dapat dipertimbangkan sebagai pilihan yang wajar untuk kasus ini.

Tugas Rinci yang Harus Dikerjakan

1. Buat *notebook* baru bernama `P1_Mandiri3_NIM.ipynb`, muat dataset `sensor_kualitas_elektronik.csv`, dan tampilkan sepuluh baris pertama beserta dimensi datanya.
2. Lakukan inspeksi menyeluruh menggunakan `df.info()`, `df.describe()`, `df.describe(include='object')`, dan `df.isnull().sum()`, lalu tuliskan ringkasan naratif mengenai apa yang Anda temukan dari hasil inspeksi tersebut, termasuk

proporsi unit dengan status “Lolos” dibandingkan “Gagal” menggunakan `df['Status_Akhir'].value_counts()`.

3. Tangani *missing value* pada kolom Tegangan_Output_Volt dan Arus_Output_Ampere menggunakan strategi imputasi yang menurut Anda paling sesuai (median atau rata-rata), dan jelaskan alasan pemilihan tersebut, termasuk pertimbangan mengapa imputasi lebih tepat dibandingkan penghapusan baris untuk kasus ini.
4. Buat **sekurang-kurangnya tiga visualisasi berbeda**, misalnya: histogram Suhu_Komponen_Celcius; *boxplot* Tegangan_Output_Volt dikelompokkan berdasarkan Status_Akhir untuk melihat apakah unit yang gagal memiliki karakteristik tegangan yang berbeda dari unit yang lolos; atau *scatter plot* antara Suhu_Komponen_Celcius dan Arus_Output_Ampere dengan pewarnaan titik berdasarkan Status_Akhir (dapat menggunakan parameter `hue='Status_Akhir'` pada `sns.scatterplot()`).
5. Tuliskan interpretasi di bawah setiap visualisasi, dengan penekanan khusus pada apakah ada indikasi awal bahwa parameter sensor tertentu berhubungan dengan status kelulusan unit — temuan ini akan menjadi dasar penting bagi pembangunan model klasifikasi pada pertemuan mendatang.
6. Simpan dan kumpulkan *notebook* sesuai format yang ditentukan dosen.

Kriteria Penilaian (Rubrik Singkat)

Komponen Penilaian	Bobot
Ketepatan memuat data dan kelengkapan inspeksi awal (termasuk analisis proporsi Lolos/Gagal)	20%
Ketepatan strategi imputasi <i>missing value</i> beserta justifikasi	15%
Keragaman dan ketepatan teknis visualisasi, termasuk penggunaan parameter <code>hue</code> untuk membandingkan kelompok (minimal 3 jenis)	30%
Kedalaman insight, khususnya terkait indikasi awal hubungan parameter sensor dengan status kelulusan	35%

STUDI KASUS KELOMPOK

Berbeda dari Studi Kasus Mandiri yang dikerjakan secara individu, dua studi kasus pada bagian ini dirancang untuk dikerjakan secara berkelompok, beranggotakan tiga hingga empat mahasiswa per kelompok. Studi kasus kelompok bertujuan melatih dua kompetensi sekaligus: kompetensi teknis EDA yang telah dipelajari, dan kompetensi kolaborasi tim dalam menganalisis dataset yang lebih kompleks serta menyusun laporan bersama — sebuah simulasi realistis dari cara kerja tim data di lingkungan industri, di mana analisis data jarang dikerjakan seorang diri.

Studi Kasus Kelompok 1 — Analisis Kualitas Udara di Lingkungan Kerja Industri

Deskripsi Masalah

Sebuah kawasan industri memasang stasiun pemantauan kualitas udara di sekitar area pabrik untuk mengawasi tingkat pencemaran gas hasil proses produksi, sebagai bagian dari kepatuhan terhadap regulasi lingkungan dan upaya menjaga kesehatan pekerja. Stasiun tersebut merekam konsentrasi berbagai gas polutan serta kondisi lingkungan (suhu dan kelembaban) setiap jam, selama beberapa bulan berturut-turut. Manajemen Kesehatan, Keselamatan, dan Lingkungan Kerja (K3L) kawasan industri tersebut ingin mengetahui pada jam-jam berapa kualitas udara berada pada kondisi terburuk, dan faktor lingkungan apa yang berkontribusi terhadap peningkatan konsentrasi polutan, agar dapat menyusun kebijakan mitigasi yang tepat sasaran, misalnya penjadwalan ventilasi tambahan atau pembatasan aktivitas luar ruangan pada jam-jam tertentu. Kelompok Anda ditugaskan sebagai tim analis data yang akan menjawab kebutuhan ini melalui eksplorasi data yang sistematis.

Deskripsi Dataset

Dataset `air_quality.csv` diadaptasi dari *UCI Air Quality Dataset*, memuat rekaman kualitas udara per jam dengan empat belas kolom sebagai berikut.

Nama Kolom	Keterangan
Date	Tanggal pencatatan
Time	Jam pencatatan
CO(GT)	Konsentrasi gas karbon monoksida hasil pengukuran referensi, dalam mg/m^3
PT08.S1(CO)	Respons sensor tin oksida terhadap CO
C6H6(GT)	Konsentrasi benzena hasil pengukuran referensi, dalam $\mu\text{g}/\text{m}^3$
PT08.S2(NMHC)	Respons sensor terhadap hidrokarbon non-metana
NOx(GT)	Konsentrasi oksida nitrogen hasil pengukuran referensi, dalam ppb
PT08.S3(NOx)	Respons sensor terhadap NOx
NO2(GT)	Konsentrasi nitrogen dioksida hasil pengukuran referensi, dalam $\mu\text{g}/\text{m}^3$
PT08.S4(NO2)	Respons sensor terhadap NO2

PT08.S5(03)	Respons sensor terhadap ozon
T	Suhu udara, dalam derajat Celsius
RH	Kelembaban relatif, dalam persen
AH	Kelembaban absolut, dalam g/m ³

Catatan penting: nilai -200 yang muncul pada berbagai kolom numerik dataset ini bukan merupakan nilai pengukuran yang valid, melainkan kode khusus yang digunakan oleh sistem pencatatan untuk menandai kegagalan sensor (nilai hilang yang “disamarkan” sebagai angka). Oleh karena itu, langkah pertama dan wajib sebelum melakukan analisis apa pun adalah mengganti seluruh nilai -200 tersebut menjadi representasi *missing value* standar pandas, yaitu `np.nan`, menggunakan perintah `df.replace(-200, np.nan, inplace=True)`. Kegagalan melakukan langkah ini akan menyebabkan seluruh statistik deskriptif dan visualisasi menjadi sangat menyesatkan, karena nilai -200 akan diperlakukan seolah-olah merupakan pengukuran yang sah.

Pembagian Peran dalam Tim

Agar pengerjaan berjalan efektif dan setiap anggota memiliki kontribusi yang jelas dan dapat dipertanggungjawabkan, disarankan pembagian peran sebagai berikut, yang dapat disesuaikan dengan jumlah anggota kelompok (3 hingga 4 orang):

- **Koordinator dan Penanggung Jawab Pra-pemrosesan Data:** bertanggung jawab memuat dataset, mengganti nilai -200 menjadi `np.nan`, melakukan inspeksi awal, dan menangani *missing value* yang tersisa, serta mengoordinasikan jadwal kerja kelompok.
- **Penanggung Jawab Rekayasa Fitur Waktu:** bertanggung jawab mengekstraksi kolom `Jam` dari kolom `Time` (misalnya menggunakan `df['Jam'] = pd.to_datetime(df['Time']).dt.hour` atau dengan memisahkan teks jam secara manual), dan membangun agregasi data berbasis waktu menggunakan `groupby('Jam')` untuk melihat pola rata-rata polutan per jam.
- **Penanggung Jawab Visualisasi:** bertanggung jawab membangun seluruh visualisasi yang disepakati tim (minimal lima visualisasi), memastikan setiap grafik memiliki judul dan label yang jelas.
- **Penanggung Jawab Penyusunan Laporan dan Interpretasi:** bertanggung jawab menyintesis seluruh temuan tim menjadi laporan akhir yang koheren, termasuk menuliskan rekomendasi kebijakan berdasarkan temuan data. Pada kelompok beranggotakan tiga orang, peran Rekayasa Fitur Waktu dapat digabungkan dengan peran Koordinator.

Pembagian peran ini bersifat panduan, bukan aturan kaku — yang terpenting adalah setiap anggota memiliki kontribusi nyata dan terukur yang dapat dicantumkan dalam laporan akhir.

Tugas Rinci yang Harus Dikerjakan

1. Sebagai tim, buka satu *notebook* bersama di Google Colab (dapat dibagikan melalui fitur *share* Google Drive agar seluruh anggota dapat mengedit secara kolaboratif).

2. **Pra-pemrosesan data:** muat dataset, ganti nilai -200 menjadi np.nan menggunakan `df.replace(-200, np.nan, inplace=True)`, kemudian identifikasi dan tangani *missing value* yang tersisa dengan metode yang disepakati tim (jelaskan alasannya secara tertulis).
3. **Rekayasa fitur waktu:** buat kolom baru Jam yang diekstraksi dari kolom Time, untuk memungkinkan analisis pola berdasarkan jam dalam sehari.
4. **Eksplorasi data,** mencakup sekurang-kurangnya lima visualisasi berbeda, meliputi minimal dua jenis visualisasi yang berbeda karakter, misalnya: grafik deret waktu (*time series*) konsentrasi CO terhadap tanggal/jam menggunakan `plt.plot()`; *scatter plot* hubungan antara suhu (T) dan konsentrasi benzena (C6H6(GT)); *boxplot* konsentrasi NO2(GT) dikelompokkan berdasarkan kolom Jam yang telah dibuat; dan *heatmap* korelasi antar-seluruh variabel numerik.
5. Gunakan `groupby('Jam')` untuk menghitung rata-rata konsentrasi CO per jam, lalu visualisasikan hasil agregasi tersebut untuk mengidentifikasi jam-jam dengan konsentrasi tertinggi.
6. **Menarik kesimpulan** sebagai tim: pada jam berapa konsentrasi CO cenderung tertinggi (pagi, siang, atau malam)? Apakah suhu atau kelembaban tampak berkorelasi dengan konsentrasi polutan? Rumuskan satu rekomendasi konkret bagi manajemen K3L, misalnya penjadwalan ventilasi tambahan pada rentang jam tertentu.
7. **Menyusun laporan** satu halaman berformat PDF yang memuat: ringkasan temuan utama (maksimal lima poin), satu visualisasi yang dianggap paling informatif beserta penjelasannya, rekomendasi kebijakan, dan daftar kontribusi setiap anggota tim (nama dan peran).
8. **Mempresentasikan** hasil analisis secara singkat (sekitar tiga menit per kelompok) di hadapan kelas, dengan satu juru bicara yang ditunjuk oleh tim.

Format Laporan yang Dikumpulkan

Laporan akhir dikumpulkan dalam dua berkas terpisah: (1) *notebook* kolaboratif (.ipynb) yang memuat seluruh kode, visualisasi, dan catatan analisis tim, diberi nama P1_Kelompok1>NamaKelompok.ipynb; dan (2) laporan ringkas satu halaman dalam format PDF bernama Laporan_Kelompok1>NamaKelompok.pdf, yang dapat disusun menggunakan Google Docs kemudian diekspor ke PDF, memuat struktur: judul dan anggota tim, ringkasan temuan, satu visualisasi utama, rekomendasi, dan pembagian kontribusi.

Rubrik Penilaian

Komponen Penilaian	Bobot
Ketepatan pra-pemrosesan (penanganan nilai -200 dan missing value)	20%
Ketepatan rekayasa fitur waktu dan penggunaan groupby untuk agregasi	15%
Keragaman dan kualitas teknis visualisasi (minimal 5, minimal 2 jenis berbeda)	25%
Kedalaman kesimpulan dan relevansi rekomendasi kebijakan	20%
Kualitas laporan tertulis (kejelasan, kelengkapan, kerapian) dan presentasi	20%

Studi Kasus Kelompok 2 — Analisis Data Logistik dan Ketepatan Waktu Pengiriman Komponen

Deskripsi Masalah

Sebuah perusahaan manufaktur peralatan berat bergantung pada pasokan komponen dari berbagai pemasok (*supplier*) yang tersebar di beberapa kota. Bagian logistik dan rantai pasok perusahaan mencatat setiap pengiriman komponen yang diterima dari pemasok selama satu tahun terakhir, termasuk apakah pengiriman tersebut tiba tepat waktu, terlambat, atau bahkan lebih awal dari jadwal. Keterlambatan pengiriman komponen dapat mengganggu jadwal produksi secara signifikan, sehingga manajemen rantai pasok ingin memahami pola keterlambatan tersebut — pemasok mana yang paling sering terlambat, moda transportasi apa yang paling andal, dan apakah jarak pengiriman berhubungan dengan risiko keterlambatan — sebagai dasar evaluasi dan negosiasi kontrak dengan pemasok pada periode berikutnya. Kelompok Anda ditugaskan melakukan eksplorasi data logistik ini untuk menghasilkan wawasan awal bagi manajemen.

Deskripsi Dataset

Dataset `logistik_pengiriman.csv` berisi catatan pengiriman komponen dengan sepuluh kolom sebagai berikut.

Nama Kolom	Keterangan
ID_Pengiriman	Nomor identitas unik setiap pengiriman
Nama_Pemasok	Nama perusahaan pemasok komponen
Kota_Asal	Kota tempat pemasok berada
Jarak_KM	Jarak tempuh pengiriman dari kota asal ke pabrik, dalam kilometer
Moda_Transportasi	Moda pengiriman: Truk, Kereta, atau Kapal Laut
Tanggal_Pesan	Tanggal pemesanan komponen dilakukan
Tanggal_Tiba_Rencana	Tanggal target kedatangan sesuai kontrak
Tanggal_Tiba_Aktual	Tanggal kedatangan aktual komponen di pabrik
Selisih_Hari	Selisih hari antara kedatangan aktual dan rencana (nilai positif berarti terlambat, negatif berarti lebih awal, nol berarti tepat waktu)
Biaya_Pengiriman_Ribu_Rupiah	Biaya pengiriman untuk pengiriman tersebut, dalam ribuan rupiah

Catatan penting: kolom `Selisih_Hari` pada sejumlah baris kosong karena `Tanggal_Tiba_Aktual` belum tercatat untuk pengiriman yang statusnya masih dalam perjalanan (*in-transit*) pada saat data diekspor. Baris dengan kondisi ini perlu diidentifikasi secara terpisah dan tidak dapat langsung dianggap sebagai bagian dari analisis ketepatan waktu, karena statusnya memang belum final, bukan karena datanya “hilang” dalam pengertian biasa.

Pembagian Peran dalam Tim

- **Koordinator dan Penanggung Jawab Pra-pemrosesan Data:** memuat data, mengidentifikasi baris dengan pengiriman yang masih *in-transit* (memisahkannya dari analisis ketepatan waktu), dan menangani *missing value* lain yang mungkin ditemukan.
- **Penanggung Jawab Analisis per Pemasok dan Moda Transportasi:** menyusun agregasi menggunakan `groupby('Nama_Pemasok')` dan `groupby('Moda_Transportasi')` untuk menghitung rata-rata `Selisih_Hari` dan tingkat keterlambatan pada setiap kelompok.
- **Penanggung Jawab Visualisasi:** membangun seluruh visualisasi yang disepakati tim, termasuk grafik perbandingan antar-pemasok dan antar-moda transportasi.
- **Penanggung Jawab Penyusunan Laporan dan Rekomendasi:** menyintesis temuan menjadi laporan akhir dan merumuskan rekomendasi bagi manajemen rantai pasok, termasuk usulan mengenai pemasok mana yang perlu dievaluasi ulang.

Tugas Rinci yang Harus Dikerjakan

1. Sebagai tim, buka satu *notebook* bersama di Google Colab dan muat dataset `logistik_pengiriman.csv`.
2. **Pra-pemrosesan data:** lakukan inspeksi struktur dan tipe data, ubah kolom `Tanggal_Pesan`, `Tanggal_Tiba_Rencana`, dan `Tanggal_Tiba_Aktual` menjadi tipe data tanggal (*datetime*) menggunakan `pd.to_datetime()`, kemudian identifikasi baris dengan `Selisih_Hari` kosong sebagai pengiriman yang masih dalam perjalanan, dan pisahkan baris tersebut ke dalam `DataFrame` terpisah agar tidak mengganggu analisis ketepatan waktu pada baris yang sudah selesai.
3. **Analisis deskriptif:** hitung statistik deskriptif `Selisih_Hari` dan `Biaya_Pengiriman_Ribu_Rupiah`, serta buat kategori baru bernama `Status_Ketepatan` yang mengelompokkan setiap pengiriman menjadi “Tepat Waktu” (`Selisih_Hari` sama dengan nol), “Terlambat” (`Selisih_Hari` lebih besar dari nol), atau “Lebih Awal” (`Selisih_Hari` kurang dari nol), misalnya menggunakan fungsi `apply()` dengan fungsi kustom atau `pd.cut()`.
4. **Eksplorasi data**, mencakup sekurang-kurangnya lima visualisasi, meliputi: grafik batang rata-rata `Selisih_Hari` per `Nama_Pemasok` untuk mengidentifikasi pemasok paling sering terlambat; *boxplot* `Selisih_Hari` dikelompokkan berdasarkan `Moda_Transportasi`; *scatter plot* antara `Jarak_KM` dan `Selisih_Hari` untuk menguji dugaan bahwa jarak yang lebih jauh berhubungan dengan risiko keterlambatan lebih tinggi; grafik lingkaran (*pie chart*) atau batang proporsi `Status_Ketepatan` secara keseluruhan; dan *heatmap* korelasi antar-variabel numerik.
5. **Menarik kesimpulan** sebagai tim: pemasok mana yang memiliki catatan ketepatan waktu terburuk? Moda transportasi mana yang paling andal? Apakah ada hubungan yang jelas antara jarak pengiriman dan keterlambatan? Rumuskan rekomendasi konkret bagi manajemen rantai pasok.
6. **Menyusun laporan** satu halaman PDF dengan struktur yang sama seperti pada Studi Kasus Kelompok 1: ringkasan temuan, satu visualisasi utama, rekomendasi, dan kontribusi anggota.
7. **Mempresentasikan** hasil analisis secara singkat di hadapan kelas.

Format Laporan yang Dikumpulkan

Sama seperti Studi Kasus Kelompok 1, laporan dikumpulkan dalam dua berkas: *notebook* kolaboratif bernama P1_Kelompok2_NamaKelompok.ipynb dan laporan PDF satu halaman bernama Laporan_Kelompok2_NamaKelompok.pdf dengan struktur serupa.

Rubrik Penilaian

Komponen Penilaian	Bobot
Ketepatan pra-pemrosesan (konversi tanggal, pemisahan data in-transit)	20%
Ketepatan analisis agregasi per pemasok dan moda transportasi	15%
Keragaman dan kualitas teknis visualisasi (minimal 5)	25%
Kedalaman kesimpulan dan relevansi rekomendasi bagi manajemen rantai pasok	20%
Kualitas laporan tertulis dan presentasi	20%

RANGKUMAN

Bab ini telah membawa mahasiswa menempuh perjalanan dari titik nol pemahaman mengenai ekosistem perangkat lunak data science hingga mampu melaksanakan eksplorasi data secara mandiri maupun berkelompok. Perjalanan tersebut dimulai dari pengenalan mengapa Python menjadi bahasa pilihan utama di bidang ilmu data, dilanjutkan dengan pengenalan empat pustaka inti yang akan terus digunakan sepanjang mata kuliah — NumPy sebagai fondasi komputasi numerik, pandas sebagai alat manipulasi data tabular, serta Matplotlib dan Seaborn sebagai alat visualisasi. Mahasiswa juga telah mempelajari dua opsi lingkungan kerja yang saling melengkapi, yaitu instalasi lokal berbasis Anaconda dan Jupyter Notebook, serta lingkungan awan Google Colaboratory, beserta cara memverifikasi kesiapan lingkungan tersebut sebelum memulai analisis.

Inti dari bab ini terletak pada penguasaan alur kerja Eksplorasi Data (EDA) yang sistematis: memuat data menggunakan `pd.read_csv()`; melihat sekilas bentuk data melalui `head()`, `tail()`, dan `shape`; memeriksa struktur dan tipe data melalui `info()` dan `dtypes`; menghitung statistik deskriptif melalui `describe()`; mengidentifikasi dan mengukur *missing value* melalui `isnull().sum()`; menangani *missing value* tersebut secara kontekstual melalui `dropna()` atau `fillna()`; dan akhirnya menggali wawasan melalui empat jenis visualisasi dasar — histogram untuk memahami distribusi, *boxplot* untuk mengidentifikasi outlier melalui prinsip IQR, *scatter plot* untuk mengeksplorasi hubungan dua variabel, dan *heatmap* korelasi untuk mengukur kekuatan hubungan linear antar-variabel secara kuantitatif. Sepanjang pembahasan, ditekankan pula prinsip kehati-hatian dalam menafsirkan hasil: keputusan penanganan *missing value* harus didasarkan pada pemahaman konteks mengapa data tersebut hilang, bukan sekadar kemudahan teknis; dan korelasi yang tinggi antara dua variabel tidak serta-merta membuktikan hubungan sebab-akibat di antara keduanya.

Keterampilan yang dibangun pada bab ini bukanlah sekadar rangkaian perintah teknis yang dihafalkan, melainkan cara berpikir sistematis dalam memahami data sebelum bertindak atasnya — sebuah kompetensi yang menjadi fondasi bagi seluruh materi pertemuan berikutnya, mulai dari pra-pemrosesan data lanjutan, pembangunan model regresi dan klasifikasi, hingga metode pembelajaran mesin yang lebih kompleks. Dalam konteks industri manufaktur, kemampuan EDA yang kokoh memungkinkan seorang praktisi data memahami data produksi, data sensor mesin, data kualitas produk, maupun data logistik secara mendalam sebelum data tersebut digunakan untuk membangun sistem prediktif seperti *predictive maintenance* atau *quality control* otomatis — memastikan bahwa keputusan berbasis data yang diambil oleh manajemen berpijak pada pemahaman data yang akurat, bukan asumsi yang keliru.

Melalui rangkaian Latihan Mandiri Terbimbing, tiga Studi Kasus Mandiri, dan dua Studi Kasus Kelompok yang mencakup beragam konteks industri — mulai dari data ritel, produksi pabrik, downtime mesin, sensor kualitas elektronik, kualitas udara lingkungan kerja, hingga logistik rantai pasok — mahasiswa telah dilatih untuk menerapkan keterampilan EDA pada berbagai jenis dataset dengan karakteristik dan tantangan kualitas data yang berbeda-beda, sekaligus melatih kemampuan menyintesis temuan teknis menjadi narasi dan rekomendasi yang bermakna bagi pengambilan keputusan bisnis.

maupun operasional. Penguasaan menyeluruh terhadap materi bab ini menjadi prasyarat mutlak untuk dapat mengikuti Pertemuan 2 mengenai Data Preprocessing dan EDA Lanjut dengan lancar, sebagaimana akan diuji melalui Tes Formatif pada bagian berikutnya.

TES FORMATIF

Bagian ini berisi soal-soal yang dirancang untuk mengukur ketercapaian Sub-CPMK 1.1 hingga 1.5 yang telah dirumuskan pada bagian Pendahuluan Bab. Kerjakan seluruh soal pilihan ganda dan esai berikut secara mandiri, tanpa melihat kunci jawaban dan pembahasan yang tersedia pada bagian setelahnya, agar hasil pengukuran mencerminkan pemahaman Anda yang sesungguhnya.

A. Soal Pilihan Ganda

Bagian ini terdiri atas 40 soal pilihan ganda dengan tingkat kesulitan bervariasi (mudah, sedang, sulit) dan jenis yang beragam (definisi, penerapan, analisis kode, dan interpretasi hasil). Pilihlah satu jawaban yang paling tepat untuk setiap soal berikut.

Soal 1. Manakah pengertian yang paling tepat mengenai *Exploratory Data Analysis* (EDA)?

A. Proses membangun model pembelajaran mesin dari data yang telah bersih B. Kegiatan meringkas karakteristik data menggunakan statistik dan visualisasi untuk menemukan pola, anomali, dan hubungan antar-variabel C. Proses menghapus seluruh data yang mengandung nilai kosong D. Teknik mengonversi data tabular menjadi basis data relasional E. Proses mengunggah data ke lingkungan Google Colab

Soal 2. Dalam konteks penerapan Machine Learning di industri manufaktur, istilah *predictive maintenance* merujuk pada penerapan untuk...

A. Mendesain tampilan antarmuka mesin produksi B. Memprediksi kapan sebuah mesin kemungkinan akan mengalami kegagalan berdasarkan data historis C. Menghitung gaji operator mesin secara otomatis D. Mengganti seluruh tenaga kerja manusia dengan robot E. Mencetak label kemasan produk secara otomatis

Soal 3. Pustaka Python manakah yang menyediakan struktur data array multidimensi bertipe homogen yang dioptimalkan untuk komputasi numerik dan menjadi fondasi bagi pustaka pandas?

A. Matplotlib B. Seaborn C. NumPy D. scikit-learn E. Jupyter

Soal 4. Alias konvensional yang digunakan komunitas data science ketika mengimpor pustaka pandas dan numpy secara berurutan adalah...

A. `import pandas as p` dan `import numpy as n` B. `import pandas as pd` dan `import numpy as np` C. `import pandas as panda` dan `import numpy as numeric` D. `import pandas as data` dan `import numpy as num` E. `import pandas as ds` dan `import numpy as npy`

Soal 5. Manakah kelemahan utama Google Colab dibandingkan instalasi Python lokal yang disebutkan dalam bab ini?

A. Tidak mendukung pustaka pandas sama sekali B. Berbayar sejak awal penggunaan C. Ketergantungan pada koneksi internet stabil dan sesi yang dapat terputus jika idle terlalu

lama D. Tidak dapat menyimpan hasil pekerjaan sama sekali E. Tidak dapat digunakan untuk memuat berkas CSV

Soal 6. Perintah `!pip install seaborn` yang dijalankan pada sebuah sel Jupyter/Colab berfungsi untuk...

A. Menghapus pustaka seaborn dari sistem B. Menjalankan perintah sebagai perintah sistem operasi untuk memasang pustaka seaborn dari PyPI C. Mengimpor pustaka seaborn ke dalam notebook D. Memeriksa versi seaborn yang terpasang E. Membuka dokumentasi resmi seaborn di peramban

Soal 7. Diberikan `shift = ['Pagi', 'Siang', 'Malam']`, ekspresi manakah yang akan menghasilkan nilai 'Pagi'?

A. `shift[1]` B. `shift[3]` C. `shift[0]` D. `shift['Pagi']` E. `shift.Pagi`

Soal 8. Diberikan `array` NumPy `arr = np.array([1, 2, 3, 4])`, hasil dari `arr * 2` adalah...

A. `[1, 2, 3, 4, 1, 2, 3, 4]` (list digandakan) B. `array([2, 4, 6, 8])` (setiap elemen dikalikan 2) C. Galat (*error*), karena array tidak bisa dikalikan D. `array([1, 2, 3, 4, 2])` E. 8 (jumlah seluruh elemen dikalikan 2)

Soal 9. Nilai `np.nan` dalam ekosistem NumPy dan pandas merepresentasikan...

A. Angka nol B. Nilai negatif tak terhingga C. Nilai yang hilang atau tidak diketahui (*Not a Number*) D. Nilai maksimum suatu kolom E. Nilai default untuk kolom bertipe teks

Soal 10. Struktur data pandas yang berbentuk tabel dua dimensi dengan baris dan kolom disebut...

A. Series B. Array C. DataFrame D. Dictionary E. Index

Soal 11. Jika `df` adalah sebuah DataFrame dengan kolom `Quantity` dan `UnitPrice`, manakah ekspresi berikut yang akan menghasilkan objek Series (bukan DataFrame)?

A. `df[['Quantity', 'UnitPrice']]` B. `df[['Quantity']]` C. `df['Quantity']` D. `df.loc[:, ['Quantity', 'UnitPrice']]` E. `df.iloc[:, [0, 1]]` (dengan asumsi kedua kolom berada di posisi 0 dan 1)

Soal 12. Fungsi pandas yang digunakan untuk memuat berkas berformat CSV ke dalam sebuah DataFrame adalah...

A. `pd.load_csv()` B. `pd.open_csv()` C. `pd.read_csv()` D. `pd.import_csv()` E. `pd.csv_to_df()`

Soal 13. Pada lingkungan Google Colab, cara paling tepat untuk mengakses berkas yang tersimpan secara permanen di Google Drive dan digunakan berulang kali pada beberapa sesi kerja berbeda adalah...

A. Menggunakan `files.upload()` setiap kali sesi dimulai B. Menghubungkan Google Drive menggunakan `drive.mount('/content/drive')` C. Mengirim berkas melalui email ke akun Colab D. Mencetak isi berkas ke layar terlebih dahulu E. Google Colab tidak dapat mengakses berkas eksternal sama sekali

Soal 14. Metode `df.head(10)` pada sebuah DataFrame akan menampilkan...

A. 10 baris terakhir dari DataFrame B. 10 kolom pertama dari DataFrame C. 10 baris pertama dari DataFrame D. Statistik deskriptif dari 10 kolom pertama E. Nama-nama 10 kolom pertama

Soal 15. Manakah informasi yang **tidak** ditampilkan oleh metode `df.info()`?

A. Jumlah total baris dan kolom B. Tipe data setiap kolom C. Jumlah nilai non-null pada setiap kolom D. Nilai rata-rata (mean) setiap kolom numerik E. Perkiraan penggunaan memori oleh dataset

Soal 16. Jika `df.info()` pada suatu dataset berisi 5000 baris menunjukkan kolom `CustomerID` memiliki 4000 non-null, maka jumlah nilai yang hilang pada kolom tersebut adalah...

A. 4000 B. 5000 C. 9000 D. 1000 E. 0

Soal 17. Mengapa kolom `CustomerID` yang secara konseptual berupa nomor identitas (bilangan bulat) dapat terbaca sebagai tipe data `float64` oleh pandas?

A. Karena pandas selalu membaca angka sebagai `float64` tanpa terkecuali B. Karena keberadaan nilai kosong (NaN) pada kolom tersebut memaksa kolom direpresentasikan sebagai bilangan pecahan C. Karena `CustomerID` mengandung huruf D. Karena kolom tersebut telah diurutkan (sorted) E. Karena jumlah barisnya genap

Soal 18. Metode `df.describe()` secara default menghitung statistik deskriptif untuk kolom bertipe...

A. Seluruh kolom tanpa terkecuali B. Hanya kolom bertipe object (teks) C. Hanya kolom bertipe numerik D. Hanya kolom yang tidak memiliki missing value E. Hanya kolom pertama pada DataFrame

Soal 19. Untuk menampilkan ringkasan statistik pada kolom kategorik (bertipe object), parameter tambahan yang perlu disertakan pada `df.describe()` adalah...

A. `df.describe(kind='categorical')` B. `df.describe(include='object')` C. `df.describe(type='text')` D. `df.describe(category=True)` E. `df.describe(numeric=False, text=True)`

Soal 20. Pada hasil `df.describe()` kolom `Quantity`, apabila nilai mean sekitar 9 namun nilai max mencapai 8099, hal ini paling mengindikasikan...

A. Kesalahan pada pustaka pandas B. Keberadaan nilai ekstrem (outlier) pada kolom tersebut C. Bahwa seluruh data pada kolom tersebut hilang D. Bahwa kolom tersebut bertipe data teks E. Bahwa dataset tidak memiliki kolom lain selain `Quantity`

Soal 21. Perintah `df.isnull().sum()` menghasilkan...

A. Total seluruh nilai dalam DataFrame dijumlahkan B. Jumlah nilai kosong (missing value) pada setiap kolom C. Jumlah baris yang tidak memiliki nilai kosong sama sekali D. Rata-rata nilai setiap kolom E. Jumlah kolom yang bertipe numerik

Soal 22. Untuk menghitung persentase missing value setiap kolom, formula yang tepat adalah...

A. `df.isnull().sum() * len(df)` B. `(df.isnull().sum() / len(df)) * 100` C. `df.isnull().mean()` D. `len(df) / df.isnull().sum()` E. `df.isnull().count() - 100`

Soal 23. Metode `df['Country'].value_counts()` digunakan untuk...

A. Menghitung jumlah nilai kosong pada kolom Country B. Menghitung frekuensi kemunculan setiap nilai unik pada kolom Country, diurutkan dari yang terbanyak C. Mengubah nilai kolom Country menjadi angka D. Menghapus duplikasi pada kolom Country E. Menghitung rata-rata panjang teks pada kolom Country

Soal 24. Perintah `df.dropna(subset=['CustomerID'])` akan...

A. Menghapus seluruh kolom pada DataFrame B. Menghapus baris yang memiliki nilai kosong pada kolom CustomerID saja C. Mengisi nilai kosong pada kolom CustomerID dengan nol D. Menghapus kolom CustomerID sepenuhnya dari DataFrame E. Mengurutkan data berdasarkan kolom CustomerID

Soal 25. Perintah `df['UnitPrice'] = df['UnitPrice'].fillna(df['UnitPrice'].median())` akan...

A. Menghapus seluruh baris yang UnitPrice-nya kosong B. Mengisi nilai kosong pada kolom UnitPrice dengan nilai median kolom tersebut C. Mengganti seluruh nilai UnitPrice menjadi median, termasuk yang sudah terisi D. Menghapus kolom UnitPrice E. Mengurutkan data berdasarkan UnitPrice

Soal 26. Median lebih disukai dibandingkan mean sebagai nilai pengisi (imputasi) pada kolom numerik yang memiliki outlier karena...

A. Median selalu bernilai lebih besar dari mean B. Median lebih mudah dihitung secara komputasi C. Median lebih tahan (robust) terhadap pengaruh nilai ekstrem dibandingkan mean D. Median hanya dapat dihitung pada data kategorik E. Median dan mean selalu menghasilkan nilai yang identik

Soal 27. Sebuah histogram digunakan terutama untuk memvisualisasikan...

A. Hubungan sebab akibat antar dua variabel B. Distribusi frekuensi suatu variabel numerik tunggal C. Lokasi geografis data D. Struktur basis data relasional E. Urutan kronologis kejadian

Soal 28. Pada histogram distribusi Quantity yang condong ke kanan (right-skewed), hal ini berarti...

A. Mayoritas nilai kecil terkonsentrasi di satu sisi, dengan sedikit nilai besar membentuk ekor panjang di sisi kanan B. Data terdistribusi merata sempurna C. Seluruh nilai bernilai negatif D. Tidak ada variasi nilai sama sekali dalam data E. Data tersebut tidak dapat divisualisasikan

Soal 29. Pada sebuah boxplot, kotak (box) merepresentasikan...

A. Nilai minimum dan maksimum data B. Rentang antar-kuartil (IQR), yaitu dari Q1 hingga Q3 C. Seluruh nilai outlier D. Rata-rata data saja E. Jumlah total data

Soal 30. Berdasarkan aturan IQR standar, sebuah nilai dianggap outlier apabila...

A. Berada tepat pada nilai median B. Berada di bawah $Q1 - 1,5 \times IQR$ atau di atas $Q3 + 1,5 \times IQR$ C. Berada di antara Q1 dan Q3 D. Sama dengan nilai rata-rata E. Merupakan nilai pertama dalam dataset

Soal 31. Diketahui $Q1 = 2$ dan $Q3 = 8$ pada suatu kolom data. Berapakah batas atas outlier menggunakan aturan $1,5 \times IQR$?

A. 8 B. 10 C. 17 D. 6 E. 14

Soal 32. Parameter $\alpha=0.3$ pada fungsi `plt.scatter()` berfungsi untuk...

A. Mengubah warna seluruh titik menjadi merah B. Mengatur tingkat transparansi titik agar overplotting pada data besar lebih mudah diamati C. Mengubah ukuran font judul grafik D. Menghapus sumbu x dan y E. Menyimpan grafik ke dalam berkas

Soal 33. Jika sebuah scatter plot antara dua variabel numerik tidak menunjukkan pola linear yang jelas, kesimpulan paling tepat adalah...

A. Kedua variabel pasti tidak memiliki hubungan apa pun dalam bentuk apa pun B. Hubungan linear yang kuat antar kedua variabel tersebut tidak terlihat, meskipun kemungkinan hubungan non-linear atau kecenderungan tertentu masih dapat diamati C. Data tersebut pasti mengandung kesalahan D. Scatter plot tidak dapat digunakan untuk data numerik E. Kedua variabel harus dihapus dari analisis

Soal 34. Fungsi `sns.heatmap(corr, annot=True, cmap='coolwarm')` digunakan untuk...

A. Menghitung nilai korelasi itu sendiri B. Menampilkan visualisasi matriks korelasi berwarna beserta nilai numeriknya pada setiap sel C. Menghapus nilai korelasi yang rendah D. Mengonversi data numerik menjadi kategorik E. Menyimpan dataset ke dalam format Excel

Soal 35. Pernyataan “korelasi tidak sama dengan kausalitas” (*correlation does not imply causation*) berarti...

A. Dua variabel yang berkorelasi tinggi pasti memiliki hubungan sebab-akibat B. Korelasi tinggi antara dua variabel tidak secara otomatis membuktikan salah satu variabel menyebabkan perubahan pada variabel lainnya C. Korelasi dan kausalitas adalah istilah yang sama persis D. Kausalitas hanya dapat diukur menggunakan heatmap E. Korelasi hanya berlaku untuk data kategorik

Soal 36. Manakah kombinasi kode berikut yang benar untuk memfilter baris DataFrame df di mana nilai UnitPrice berada di antara low dan high (inklusif)?

A. `df[df['UnitPrice'] >= low, df['UnitPrice'] <= high]` B. `df[(df['UnitPrice'] >= low) & (df['UnitPrice'] <= high)]` C. `df[df['UnitPrice'] >= low and df['UnitPrice'] <= high]` D. `df.filter(low, high)` E. `df['UnitPrice'].between()` (tanpa argumen)

Soal 37. Sebuah dataset sensor kualitas produk memiliki kolom Status_Akhir bernilai “Lolos” atau “Gagal”. Untuk membandingkan distribusi Tegangan_Output_Volt antara kedua kelompok status tersebut menggunakan boxplot, kode Seaborn yang paling tepat digunakan adalah...

A. `sns.boxplot(data=df, x='Status_Akhir', y='Tegangan_Output_Volt')` B. `sns.boxplot(data=df)` tanpa parameter tambahan C. `sns.histplot(data=df, x='Status_Akhir')` D. `plt.scatter(df['Status_Akhir'])` E. `df.dropna()`

Soal 38. Nilai -200 pada dataset kualitas udara (`air_quality.csv`) merepresentasikan...

A. Nilai pengukuran yang sah dan valid B. Kode khusus yang menandakan kegagalan sensor (missing value tersamar), yang harus diganti menjadi NaN sebelum dianalisis C. Suhu ruangan dalam derajat Fahrenheit D. Jumlah maksimum polutan yang diperbolehkan E. Kode error dari pandas

Soal 39. Perintah yang tepat untuk mengganti nilai -200 menjadi representasi missing value standar pandas pada seluruh DataFrame df adalah...

A. `df.fillna(-200)` B. `df.replace(-200, np.nan, inplace=True)` C. `df.dropna(-200)` D. `df['-200'] = np.nan` E. `df.describe(-200)`

Soal 40. Mengapa dalam Studi Kasus Kelompok mengenai data logistik pengiriman, baris dengan Selisih_Hari kosong (pengiriman masih *in-transit*) sebaiknya dipisahkan dan tidak langsung dianggap sebagai missing value biasa yang perlu diimputasi?

A. Karena nilai kosong tersebut merepresentasikan status yang memang belum final (pengiriman belum tiba), bukan kegagalan pencatatan data yang bersifat acak B. Karena kolom Selisih_Hari tidak penting untuk dianalisis C. Karena seluruh baris dengan nilai kosong harus selalu dihapus tanpa terkecuali dalam segala konteks D. Karena pandas tidak dapat memproses kolom dengan nilai kosong E. Karena Tanggal_Tiba_Aktual selalu terisi penuh tanpa nilai kosong

B. Soal Esai Analitis

Bagian ini terdiri atas sepuluh soal esai yang menuntut jawaban analitis dan naratif, bukan sekadar hafalan definisi. Jawablah setiap soal dengan kalimat lengkap disertai alasan (justifikasi) yang jelas, dan sertakan cuplikan kode bila diminta secara eksplisit. Estimasi waktu pengerjaan bagian ini adalah 60–90 menit.

Esai 1. Jelaskan mengapa mahasiswa Program Studi D4 Teknologi Rekayasa Informatika Industri perlu menguasai keterampilan Eksplorasi Data (EDA) sebelum mempelajari algoritma pembelajaran mesin yang lebih kompleks pada pertemuan-pertemuan berikutnya. Kaitkan jawaban Anda dengan prinsip *garbage in, garbage out* dan berikan satu contoh konkret dari konteks industri manufaktur.

Esai 2. Seorang mahasiswa bekerja pada dua lingkungan berbeda: instalasi Python lokal menggunakan Anaconda dan Jupyter Notebook, serta Google Colaboratory. Bandingkan kedua lingkungan tersebut dari segi kemudahan instalasi, ketergantungan pada koneksi internet, akses sumber daya komputasi, dan keberlangsungan sesi kerja. Menurut pendapat Anda, dalam situasi seperti apa sebaiknya seorang praktisi data memilih instalasi lokal, dan dalam situasi seperti apa Google Colab lebih tepat digunakan?

Esai 3. Perhatikan hasil `df.info()` berikut ini yang diperoleh dari sebuah dataset transaksi ritel berisi 5000 baris.

```
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 8 columns):
InvoiceNo      5000 non-null object
StockCode     5000 non-null object
Description    4990 non-null object
Quantity      5000 non-null int64
InvoiceDate    5000 non-null object
UnitPrice     5000 non-null float64
CustomerID     4000 non-null float64
Country       5000 non-null object
```

Berdasarkan keluaran tersebut, jelaskan: (a) berapa jumlah dan persentase nilai yang hilang pada masing-masing kolom `Description` dan `CustomerID`; (b) mengapa kolom `CustomerID`, yang secara konseptual merupakan bilangan bulat (identitas pelanggan), justru terbaca sebagai tipe `float64`; dan (c) strategi penanganan apa yang menurut Anda paling tepat untuk masing-masing kolom tersebut, disertai alasannya.

Esai 4. Jelaskan perbedaan mendasar antara metode `dropna()` dan `fillna()` dalam penanganan *missing value* pada pandas. Kemudian, rumuskan sebuah kaidah praktis (aturan umum) yang dapat digunakan seorang analis data pemula untuk memutuskan kapan sebaiknya menggunakan `dropna()` dan kapan sebaiknya menggunakan `fillna()`, dengan mempertimbangkan faktor persentase data yang hilang dan tingkat kepentingan kolom yang bersangkutan terhadap analisis.

Esai 5. Dataset `machine_downtime.csv` pada Studi Kasus Mandiri 2 memiliki dua persoalan kualitas data yang berbeda karakter: (1) nilai kosong pada kolom `Biaya_Perbaikan_Ribu_Rupiah`, dan (2) nilai negatif yang tidak logis pada kolom `Usia_Mesin_Tahun`. Jelaskan mengapa kedua persoalan ini **tidak dapat ditangani dengan pendekatan yang sama**, dan uraikan langkah penanganan yang tepat untuk masing-masing persoalan tersebut, termasuk kapan sebuah nilai negatif sebaiknya dikoreksi (misalnya diubah menjadi nilai absolutnya) dan kapan ia sebaiknya diperlakukan sebagai baris yang harus dihapus.

Esai 6. Empat jenis visualisasi dasar yang dipelajari pada bab ini adalah histogram, *boxplot*, *scatter plot*, dan *heatmap* korelasi. Untuk setiap jenis visualisasi tersebut, jelaskan: (a) jenis pertanyaan analitis apa yang paling tepat dijawab oleh visualisasi tersebut; dan (b) satu contoh skenario dari konteks industri manufaktur (produksi, kualitas, perawatan mesin, logistik, atau lingkungan kerja) di mana visualisasi tersebut akan sangat berguna.

Esai 7. Pada aturan IQR (*Interquartile Range*) untuk mendeteksi outlier, sebuah nilai dianggap outlier apabila berada di bawah $Q1 - 1,5 \times IQR$ atau di atas $Q3 + 1,5 \times IQR$. Diketahui suatu kolom *Durasi_Downtime_Menit* memiliki $Q1 = 15$ dan $Q3 = 45$. (a) Hitunglah IQR, batas bawah, dan batas atas outlier untuk kolom tersebut, tunjukkan seluruh langkah perhitungan. (b) Jika ditemukan sebuah nilai 200 menit pada kolom tersebut, apakah nilai itu tergolong outlier menurut aturan ini? Jelaskan implikasi praktis temuan tersebut bagi seorang manajer perawatan mesin.

Esai 8. Jelaskan mengapa pernyataan “korelasi tidak sama dengan kausalitas” (*correlation does not imply causation*) menjadi prinsip penting yang harus dipegang oleh seorang analis data. Berikan sebuah contoh hipotetis dari konteks manufaktur di mana dua variabel menunjukkan korelasi tinggi pada *heatmap*, namun kesimpulan bahwa satu variabel menyebabkan perubahan pada variabel lainnya adalah keliru — jelaskan kemungkinan adanya variabel ketiga (*confounding variable*) yang menjadi penyebab sesungguhnya.

Esai 9. Pada Studi Kasus Kelompok 1 (data kualitas udara), nilai -200 yang muncul pada berbagai kolom numerik bukan merupakan pengukuran yang valid, melainkan kode yang menandakan kegagalan sensor. Jelaskan mengapa nilai seperti ini disebut sebagai *missing value* yang “tersamar” (*disguised missing value*), mengapa nilai tersebut berbahaya apabila tidak ditangani sebelum analisis dimulai (jelaskan dampaknya terhadap *describe()* dan visualisasi), dan tuliskan satu baris kode pandas yang tepat untuk mengubah seluruh nilai -200 tersebut menjadi representasi *missing value* standar.

Esai 10. Bayangkan Anda adalah seorang praktisi data yang baru saja menyelesaikan proses EDA menyeluruh terhadap dataset produksi sebuah pabrik, dan menemukan bahwa: rata-rata jumlah cacat produk pada shift Malam jauh lebih tinggi dibandingkan shift Pagi dan Siang; terdapat korelasi positif sedang antara suhu mesin dan jumlah cacat; serta terdapat sejumlah *missing value* pada kolom kelembaban ruangan yang telah Anda tangani melalui imputasi median. Susunlah sebuah paragraf ringkasan naratif (minimal lima kalimat) yang dapat Anda sampaikan kepada manajer produksi yang tidak memiliki latar belakang teknis, yang merangkum temuan utama Anda beserta satu rekomendasi tindak lanjut yang konkret dan dapat ditindaklanjuti oleh manajemen.

KUNCI JAWABAN DAN PEMBAHASAN

C. Kunci Jawaban dan Pembahasan Soal Pilihan Ganda

Bagian ini memuat kunci jawaban beserta pembahasan lengkap untuk keempat puluh soal pilihan ganda pada bagian A. Setiap pembahasan tidak hanya menyebutkan mengapa pilihan yang ditetapkan sebagai kunci adalah benar, tetapi juga menjelaskan secara singkat mengapa pilihan-pilihan lain tidak tepat, sehingga mahasiswa yang menjawab keliru dapat memahami akar kesalahannya, bukan sekadar mengetahui jawaban yang benar.

Soal 1 — Jawaban: B. EDA didefinisikan secara eksplisit dalam bab ini sebagai kegiatan meringkas karakteristik data menggunakan statistik dan visualisasi untuk menemukan pola, anomali, dan hubungan antar-variabel. Pilihan A keliru karena membangun model adalah tahap yang berbeda dan datang setelah EDA. Pilihan C keliru karena EDA tidak identik dengan penghapusan seluruh data kosong — penghapusan hanyalah salah satu opsi penanganan yang dipertimbangkan secara kontekstual. Pilihan D dan E tidak berkaitan dengan definisi EDA sama sekali.

Soal 2 — Jawaban: B. *Predictive maintenance* secara definitif adalah penerapan ML untuk memprediksi kapan mesin akan mengalami kegagalan berdasarkan data historis, sebagaimana dijelaskan pada bagian Pendahuluan Bab. Pilihan A, C, D, dan E menggambarkan aktivitas yang tidak berkaitan dengan analitik prediktif berbasis data sensor mesin.

Soal 3 — Jawaban: C. NumPy adalah pustaka yang menyediakan struktur data array multidimensi bertipe homogen dan menjadi fondasi komputasi numerik bagi pandas. Matplotlib (A) dan Seaborn (B) adalah pustaka visualisasi, bukan struktur data numerik; scikit-learn (D) adalah pustaka pembelajaran mesin; Jupyter (E) adalah lingkungan kerja interaktif, bukan pustaka komputasi.

Soal 4 — Jawaban: B. Alias konvensional yang disepakati luas oleh komunitas data science adalah `import pandas as pd` dan `import numpy as np`. Alias-alias pada pilihan lain memang dapat berjalan secara teknis di Python, namun tidak digunakan secara konvensional sehingga akan membingungkan pembaca kode lain dan tidak konsisten dengan dokumentasi resmi maupun contoh kode di seluruh bab ini.

Soal 5 — Jawaban: C. Bab ini secara eksplisit menyebutkan kelemahan Google Colab adalah ketergantungan pada koneksi internet yang stabil serta risiko sesi terputus jika dibiarkan *idle* terlalu lama. Pilihan A dan E salah karena Colab justru mendukung pandas dan pemuatan berkas CSV. Pilihan B keliru karena Colab menyediakan tingkat penggunaan gratis. Pilihan D keliru karena hasil pekerjaan dapat disimpan ke Google Drive.

Soal 6 — Jawaban: B. Tanda seru (!) di awal sebuah sel Jupyter/Colab menandakan bahwa baris tersebut dijalankan sebagai perintah *shell* sistem operasi, bukan sebagai kode Python biasa; `!pip install seaborn` memasang pustaka seaborn dari PyPI ke lingkungan kerja aktif. Pilihan C keliru karena mengimpor pustaka menggunakan `import seaborn`, bukan `!pip install`. Pilihan A, D, dan E tidak sesuai dengan fungsi perintah `pip install`.

Soal 7 — Jawaban: C. Pengindeksan *list* pada Python dimulai dari 0, sehingga `shift[0]` merujuk elemen pertama, yaitu 'Pagi'. Pilihan A (`shift[1]`) akan menghasilkan 'Siang'. Pilihan B (`shift[3]`) akan menghasilkan galat `IndexError` karena indeks tersebut berada di luar jangkauan *list* yang hanya memiliki indeks 0 hingga 2. Pilihan D dan E keliru karena *list* Python tidak diakses menggunakan kunci teks seperti *dictionary*.

Soal 8 — Jawaban: B. Salah satu keunggulan *array* NumPy dibandingkan *list* Python biasa adalah operasi aritmetika seperti perkalian diterapkan secara *element-wise* (per elemen), sehingga `arr * 2` menghasilkan `array([2, 4, 6, 8])`. Pilihan A menggambarkan perilaku *list* Python biasa (penggandaan/duplikasi), bukan perilaku *array* NumPy. Pilihan C, D, dan E tidak mencerminkan mekanisme *broadcasting* NumPy yang sesungguhnya.

Soal 9 — Jawaban: C. `np.nan` (*Not a Number*) adalah representasi standar untuk nilai yang hilang atau tidak diketahui dalam ekosistem NumPy dan pandas. Pilihan A, B, D, dan E keliru karena `np.nan` bukan angka nol, bukan nilai tak terhingga, dan bukan nilai statistik apa pun — ia justru merepresentasikan ketiadaan nilai yang dapat diukur.

Soal 10 — Jawaban: C. `DataFrame` adalah struktur data pandas berbentuk tabel dua dimensi dengan baris dan kolom, mirip sebuah *spreadsheet*. `Series` (A) adalah struktur satu dimensi (setara satu kolom). `Array` (B) adalah struktur NumPy, bukan struktur tabel pandas. `Dictionary` (D) adalah struktur data Python bawaan berbasis pasangan kunci-nilai. `Index` (E) merujuk pada label baris/kolom, bukan struktur tabel itu sendiri.

Soal 11 — Jawaban: C. Mengakses satu kolom tunggal menggunakan tanda kurung siku tunggal, `df['Quantity']`, menghasilkan objek `Series`. Pilihan A, B, D, dan E semuanya menggunakan mekanisme yang menghasilkan `DataFrame` (dua dimensi), karena melibatkan daftar nama kolom (kurung siku ganda) atau irisan (*slice*) beberapa kolom sekaligus melalui `.loc` atau `.iloc`.

Soal 12 — Jawaban: C. Fungsi resmi pandas untuk memuat berkas CSV ke dalam `DataFrame` adalah `pd.read_csv()`. Pilihan A, B, D, dan E bukan merupakan nama fungsi yang benar-benar ada pada pustaka pandas.

Soal 13 — Jawaban: B. Untuk mengakses berkas yang tersimpan permanen di Google Drive dan digunakan berulang kali lintas sesi, cara paling tepat adalah menghubungkan (*mount*) Google Drive menggunakan `drive.mount('/content/drive')`. Pilihan A (`files.upload()`) hanya bersifat sementara pada sesi aktif dan harus diulang setiap sesi baru dimulai, sehingga kurang efisien untuk penggunaan berulang. Pilihan C, D, dan E bukan merupakan mekanisme akses berkas yang valid di Colab.

Soal 14 — Jawaban: C. Metode `df.head(n)` menampilkan `n` baris pertama dari `DataFrame`; dengan argumen 10, metode ini menampilkan sepuluh baris pertama. Pilihan A menggambarkan fungsi `df.tail(10)`, bukan `df.head(10)`. Pilihan B, D, dan E salah karena `head()` bekerja pada baris, bukan kolom, dan tidak menghasilkan statistik deskriptif.

Soal 15 — Jawaban: D. `df.info()` menampilkan jumlah baris dan kolom, tipe data setiap kolom, jumlah nilai non-null, serta estimasi penggunaan memori — namun **tidak** menampilkan nilai rata-rata (*mean*) atau statistik deskriptif lainnya; informasi tersebut

baru disajikan oleh `df.describe()`. Pilihan A, B, C, dan E justru merupakan informasi yang benar-benar ditampilkan oleh `df.info()`.

Soal 16 — Jawaban: D. Jika total baris adalah 5000 dan CustomerID memiliki 4000 nilai non-null, maka jumlah nilai yang hilang dihitung sebagai selisih keduanya: $5000 - 4000 = 1000$. Pilihan A (4000) adalah jumlah nilai yang *ada*, bukan yang hilang. Pilihan B, C, dan E adalah hasil perhitungan yang keliru secara aritmetika.

Soal 17 — Jawaban: B. Pandas merepresentasikan kolom numerik yang mengandung nilai NaN sebagai tipe `float64`, karena tipe `int64` (bilangan bulat murni) pada versi pandas standar tidak dapat menampung nilai NaN di dalamnya; keberadaan satu saja nilai kosong pada kolom numerik integer memaksa seluruh kolom “dipromosikan” menjadi tipe pecahan. Pilihan A keliru karena pandas dapat membaca kolom sebagai `int64` murni apabila tidak ada nilai kosong. Pilihan C, D, dan E tidak berkaitan dengan mekanisme representasi tipe data pandas yang sesungguhnya.

Soal 18 — Jawaban: C. Secara default, `df.describe()` hanya menghitung statistik deskriptif (count, mean, std, min, kuartil, max) untuk kolom-kolom bertipe numerik. Pilihan A keliru karena kolom teks tidak disertakan secara default. Pilihan B terbalik dari perilaku sesungguhnya. Pilihan D dan E tidak mencerminkan perilaku default `describe()`.

Soal 19 — Jawaban: B. Parameter `include='object'` pada `df.describe(include='object')` mengarahkan pandas untuk menghitung ringkasan statistik (count, unique, top, freq) khusus kolom bertipe object (teks/kategorik). Pilihan A, C, D, dan E menggunakan sintaksis parameter yang tidak dikenali oleh pandas.

Soal 20 — Jawaban: B. Selisih besar antara nilai mean (sekitar 9) dan max (8099) pada kolom yang sama merupakan indikasi kuat adanya nilai ekstrem (*outlier*) yang menarik rata-rata sekaligus menunjukkan sebaran data yang sangat tidak simetris. Pilihan A, C, D, dan E tidak menjelaskan fenomena statistik yang sesungguhnya terjadi pada kasus ini.

Soal 21 — Jawaban: B. `df.isnull()` menghasilkan tabel boolean (True/False) yang menandai sel kosong, dan `.sum()` yang dirantai setelahnya menjumlahkan nilai True (dihitung sebagai 1) per kolom, sehingga hasil akhirnya adalah jumlah nilai kosong pada setiap kolom. Pilihan A, C, D, dan E tidak mencerminkan mekanisme perhitungan yang sesungguhnya dari kombinasi kedua metode tersebut.

Soal 22 — Jawaban: B. Persentase *missing value* dihitung dengan membagi jumlah nilai kosong per kolom (`df.isnull().sum()`) dengan jumlah total baris (`len(df)`), lalu dikalikan 100, yaitu $(df.isnull().sum() / len(df)) * 100$. Pilihan A menggunakan operasi perkalian yang salah arah. Pilihan C hanya menghasilkan proporsi (skala 0–1), bukan persentase, sehingga kurang tepat tanpa dikalikan 100. Pilihan D dan E tidak menghasilkan makna persentase yang benar.

Soal 23 — Jawaban: B. `value_counts()` menghitung frekuensi kemunculan setiap nilai unik pada sebuah Series, dan secara default mengurutkan hasilnya dari yang paling sering muncul hingga paling jarang. Pilihan A, C, D, dan E menggambarkan fungsi lain yang berbeda dari `value_counts()`.

Soal 24 — Jawaban: B. `df.dropna(subset=[ 'CustomerID' ])` menghapus baris-baris yang memiliki nilai kosong **khusus** pada kolom yang disebutkan dalam parameter subset, yaitu CustomerID, tanpa memengaruhi baris dengan nilai kosong pada kolom lain. Pilihan A, C, D, dan E menggambarkan operasi yang berbeda dari `dropna(subset=...)`.

Soal 25 — Jawaban: B. Kode tersebut mengisi (imputasi) nilai kosong pada kolom UnitPrice dengan nilai median kolom tersebut, sementara nilai yang sudah terisi sebelumnya tidak diubah karena `fillna()` hanya memengaruhi sel yang bernilai NaN. Pilihan A menggambarkan perilaku `dropna()`, bukan `fillna()`. Pilihan C keliru karena `fillna()` tidak mengganti nilai yang sudah ada. Pilihan D dan E tidak sesuai dengan fungsi `fillna()`.

Soal 26 — Jawaban: C. Median lebih disukai sebagai nilai imputasi pada kolom dengan outlier karena median bersifat *robust* (tahan) terhadap pengaruh nilai ekstrem, sedangkan mean sangat mudah “ditarik” oleh nilai-nilai ekstrem tersebut sehingga menjadi kurang representatif terhadap kecenderungan data secara umum. Pilihan A, B, D, dan E tidak menjelaskan alasan statistik yang sesungguhnya di balik preferensi ini.

Soal 27 — Jawaban: B. Histogram dirancang khusus untuk memvisualisasikan distribusi frekuensi suatu variabel numerik tunggal, menunjukkan seberapa sering nilai-nilai dalam rentang tertentu muncul. Pilihan A lebih sesuai untuk *scatter plot*. Pilihan C, D, dan E tidak berkaitan dengan fungsi utama histogram.

Soal 28 — Jawaban: A. Distribusi *right-skewed* (condong ke kanan) berarti mayoritas nilai terkonsentrasi pada rentang kecil, dengan sedikit nilai besar yang membentuk ekor panjang memanjang ke arah kanan grafik histogram. Pilihan B, C, D, dan E tidak menggambarkan pengertian kemencengan (*skewness*) distribusi secara tepat.

Soal 29 — Jawaban: B. Kotak (*box*) pada boxplot merepresentasikan rentang antar-kuartil (*Interquartile Range/IQR*), yaitu rentang dari kuartil pertama (Q1) hingga kuartil ketiga (Q3), yang mencakup 50% data di bagian tengah. Pilihan A menggambarkan fungsi *whisker* (kumis), bukan kotak. Pilihan C, D, dan E tidak sesuai dengan makna kotak pada boxplot.

Soal 30 — Jawaban: B. Berdasarkan aturan IQR standar, sebuah nilai dianggap outlier apabila berada di bawah $Q1 - 1,5 \times IQR$ atau di atas $Q3 + 1,5 \times IQR$. Pilihan A, C, D, dan E tidak mencerminkan definisi outlier berdasarkan aturan IQR yang telah dibahas pada bab ini.

Soal 31 — Jawaban: C. Dengan $Q1 = 2$ dan $Q3 = 8$, maka $IQR = Q3 - Q1 = 8 - 2 = 6$. Batas atas outlier dihitung sebagai $Q3 + 1,5 \times IQR = 8 + (1,5 \times 6) = 8 + 9 = 17$. Pilihan A, B, D, dan E merupakan hasil perhitungan yang tidak sesuai dengan langkah aritmetika yang benar.

Soal 32 — Jawaban: B. Parameter alpha pada fungsi plotting Matplotlib/Seaborn mengatur tingkat transparansi elemen visual (dengan rentang nilai 0 hingga 1); nilai $\alpha=0.3$ membuat titik menjadi semi-transparan sehingga titik-titik yang saling bertumpuk (*overplotting*) pada dataset besar menjadi lebih mudah diamati kepadatannya. Pilihan A, C, D, dan E menggambarkan parameter lain yang tidak berkaitan dengan alpha.

Soal 33 — Jawaban: B. Ketidadaan pola linear yang jelas pada *scatter plot* hanya menunjukkan bahwa hubungan **linear** yang kuat tidak teramati; hal ini tidak serta-merta berarti kedua variabel sama sekali tidak berhubungan, karena mungkin saja terdapat hubungan non-linear atau kecenderungan tertentu yang tidak tertangkap oleh korelasi linear. Pilihan A, C, D, dan E menggambarkan kesimpulan yang terlalu ekstrem dan tidak didukung oleh penalaran statistik yang tepat.

Soal 34 — Jawaban: B. `sns.heatmap(corr, annot=True, cmap='coolwarm')` menampilkan visualisasi matriks korelasi berwarna, dengan `annot=True` menambahkan nilai numerik korelasi pada setiap sel dan `cmap='coolwarm'` menentukan skema pewarnaan. Pilihan A keliru karena `heatmap` tidak menghitung korelasi (perhitungan dilakukan oleh `.corr()`), melainkan hanya memvisualisasikan hasil yang sudah dihitung. Pilihan C, D, dan E tidak menggambarkan fungsi `sns.heatmap()`.

Soal 35 — Jawaban: B. Prinsip “korelasi tidak sama dengan kausalitas” berarti korelasi tinggi antara dua variabel tidak secara otomatis membuktikan bahwa salah satu variabel menyebabkan perubahan pada variabel lainnya — hubungan tersebut dapat disebabkan oleh kebetulan atau variabel ketiga (*confounding variable*) yang tidak teramati. Pilihan A bertentangan langsung dengan prinsip ini. Pilihan C, D, dan E tidak mencerminkan makna prinsip tersebut secara tepat.

Soal 36 — Jawaban: B. Untuk memfilter baris berdasarkan lebih dari satu kondisi pada `pandas`, setiap kondisi harus diapit tanda kurung dan digabungkan menggunakan operator bitwise `&` (bukan `and`), sehingga sintaksis yang benar adalah `df[(df['UnitPrice'] >= low) & (df['UnitPrice'] <= high)]`. Pilihan A menggunakan sintaksis koma yang tidak valid untuk pemfilteran kondisi ganda. Pilihan C keliru karena operator `and` Python biasa tidak bekerja dengan benar pada `Series` `pandas` yang berukuran banyak elemen (akan memunculkan galat *ambiguous truth value*). Pilihan D dan E tidak menggunakan sintaksis `pandas` yang valid.

Soal 37 — Jawaban: A. Untuk membandingkan distribusi suatu variabel numerik antar-kelompok kategorik menggunakan `boxplot` pada `Seaborn`, sintaksis yang tepat adalah menyertakan parameter `x` untuk kelompok kategorik dan `y` untuk variabel numerik: `sns.boxplot(data=df, x='Status_Akhir', y='Tegangan_Output_Volt')`. Pilihan B tidak menghasilkan pengelompokan yang diminta. Pilihan C menggunakan jenis grafik yang tidak sesuai (histogram untuk data kategorik tunggal). Pilihan D menggunakan fungsi yang salah (`scatter` untuk data kategorik tunggal, tanpa argumen `y`). Pilihan E tidak menghasilkan visualisasi sama sekali.

Soal 38 — Jawaban: B. Pada dataset `air_quality.csv`, nilai `-200` bukan merupakan pengukuran fisik yang valid (konsentrasi gas atau suhu tidak mungkin bernilai `-200` dalam konteks ini), melainkan kode khusus dari sistem pencatatan yang menandakan kegagalan pembacaan sensor — sebuah bentuk *missing value* yang “tersamar” sebagai angka, dan harus diubah menjadi `NaN` sebelum dianalisis. Pilihan A, C, D, dan E tidak mencerminkan makna nilai `-200` dalam konteks dataset ini.

Soal 39 — Jawaban: B. Perintah `df.replace(-200, np.nan, inplace=True)` mengganti seluruh kemunculan nilai `-200` pada `DataFrame` menjadi `np.nan`, dengan parameter

`inplace=True` memastikan perubahan diterapkan langsung pada `DataFrame` yang bersangkutan tanpa perlu menugaskan ulang (*reassign*) hasilnya. Pilihan A, C, D, dan E menggunakan sintaksis atau fungsi yang tidak sesuai untuk operasi penggantian nilai ini.

Soal 40 — Jawaban: A. Baris dengan `Selisih_Hari` kosong pada Studi Kasus Kelompok 2 merepresentasikan pengiriman yang statusnya memang belum final (masih *in-transit*), bukan kegagalan pencatatan data yang bersifat acak; oleh karena itu baris tersebut secara konseptual berbeda dari *missing value* biasa dan tidak dapat langsung diimputasi seolah-olah nilainya “hilang” secara acak, karena melakukan hal tersebut akan mendistorsi analisis ketepatan waktu pengiriman yang sudah selesai. Pilihan B, C, D, dan E tidak mencerminkan alasan konseptual yang sesungguhnya mengapa baris tersebut memerlukan penanganan khusus.

D. Jawaban Model dan Pembahasan Soal Esai

Bagian ini menyajikan jawaban model untuk kesepuluh soal esai pada bagian B. Perlu ditekankan bahwa jawaban esai bersifat terbuka dan naratif, sehingga jawaban mahasiswa tidak harus identik kata demi kata dengan jawaban model berikut; yang dinilai adalah ketepatan konsep, kedalaman penalaran, dan kejelasan penyampaian. Jawaban model ini berfungsi sebagai acuan standar minimal yang harus tercakup dalam jawaban mahasiswa agar dapat dinyatakan menguasai indikator yang bersangkutan.

Esai 1 — Jawaban Model. Mahasiswa D4 Teknologi Rekayasa Informatika Industri perlu menguasai EDA sebelum mempelajari algoritma ML yang lebih kompleks karena seluruh model prediktif, betapapun canggihnya, dibangun di atas data sebagai bahan bakunya. Prinsip *garbage in, garbage out* menegaskan bahwa apabila data masukan mengandung kesalahan yang tidak terdeteksi — seperti nilai hilang yang tidak ditangani, outlier ekstrem yang tidak diperiksa, atau tipe data yang keliru terbaca — maka model yang dibangun di atasnya, betapapun canggih algoritmanya, akan menghasilkan keluaran yang menyesatkan atau tidak dapat diandalkan. Sebagai contoh konkret dalam konteks manufaktur, apabila sebuah tim rekayasa membangun model *predictive maintenance* tanpa terlebih dahulu memeriksa bahwa sebagian data sensor suhu mesin memiliki nilai kosong akibat sensor yang sempat rusak, model tersebut mungkin belajar dari data yang tidak representatif dan gagal mendeteksi pola kegagalan yang sesungguhnya, sehingga menghasilkan prediksi keliru yang berpotensi menyebabkan kegagalan mesin tidak terdeteksi tepat waktu di lapangan. **Kriteria penilaian:** mencakup penjelasan prinsip *garbage in, garbage out* (40%), keterkaitan EDA sebagai prasyarat model (30%), dan contoh konkret manufaktur yang relevan (30%).

Esai 2 — Jawaban Model. Instalasi lokal berbasis Anaconda dan Jupyter Notebook unggul dalam hal tidak memerlukan koneksi internet setelah terpasang, memberikan kendali penuh atas versi pustaka yang digunakan, dan tidak memiliki batas waktu sesi kerja; namun proses instalasinya membutuhkan ruang penyimpanan dan spesifikasi komputer yang memadai, serta bergantung pada spesifikasi perangkat keras pengguna untuk kecepatan komputasi. Google Colab sebaliknya tidak memerlukan instalasi sama sekali dan menyediakan akses gratis ke sumber daya komputasi tambahan seperti GPU/TPU terbatas, namun sepenuhnya bergantung pada koneksi internet yang stabil dan sesi kerjanya dapat

terputus apabila dibiarkan *idle* terlalu lama. Instalasi lokal lebih tepat dipilih ketika praktisi bekerja pada data sensitif yang tidak boleh diunggah ke layanan awan, ketika koneksi internet tidak dapat diandalkan, atau ketika proyek memerlukan konfigurasi pustaka khusus dalam jangka panjang. Google Colab lebih tepat digunakan untuk pembelajaran, kolaborasi tim secara *real-time*, atau ketika perangkat pengguna tidak memiliki spesifikasi memadai untuk komputasi lokal. **Kriteria penilaian:** perbandingan mencakup minimal tiga aspek berbeda (40%), rekomendasi kontekstual yang logis untuk instalasi lokal (30%) dan untuk Colab (30%).

Esai 3 — Jawaban Model. (a) Kolom Description memiliki $5000 - 4990 = 10$ nilai hilang, setara $10/5000 \times 100\% = 0,2\%$; kolom CustomerID memiliki $5000 - 4000 = 1000$ nilai hilang, setara $1000/5000 \times 100\% = 20\%$. (b) CustomerID terbaca sebagai float64 karena keberadaan nilai NaN pada kolom tersebut memaksa pandas merepresentasikan seluruh kolom sebagai bilangan pecahan, sebab tipe integer standar (int64) tidak mampu menampung nilai NaN di dalamnya. (c) Untuk Description, karena persentase missing sangat kecil (0,2%) dan kolom ini bersifat deskriptif (bukan kunci analisis kritis), strategi yang tepat adalah mengisi nilai kosong dengan string konstan seperti 'Unknown' menggunakan `fillna('Unknown')`, agar tidak kehilangan baris data secara sia-sia. Untuk CustomerID, karena persentase missing cukup besar (20%) namun kolom ini merupakan identitas kunci yang tidak dapat diimputasi secara bermakna (tidak ada cara logis menebak ID pelanggan yang hilang), strategi yang tepat adalah menghapus baris yang bersangkutan menggunakan `dropna(subset=['CustomerID'])`. **Kriteria penilaian:** perhitungan jumlah dan persentase missing yang benar (25%), penjelasan mekanisme tipe float64 (25%), strategi penanganan yang tepat dan berbeda untuk masing-masing kolom beserta alasannya (50%).

Esai 4 — Jawaban Model. `dropna()` menghapus baris (atau kolom) yang mengandung nilai kosong, sehingga mengurangi jumlah data yang tersedia untuk dianalisis; `fillna()` sebaliknya mempertahankan seluruh baris dengan mengisi nilai kosong tersebut menggunakan nilai pengganti tertentu (konstanta, statistik seperti mean/median, atau metode *forward/backward fill*). Kaidah praktis yang dapat dirumuskan adalah: gunakan `dropna()` apabila persentase nilai yang hilang pada kolom tersebut relatif kecil (sehingga penghapusan tidak banyak mengurangi ukuran data) **dan** kolom tersebut merupakan kolom kritis yang sulit atau tidak bermakna untuk diimputasi (misalnya identitas unik); sebaliknya, gunakan `fillna()` apabila persentase nilai hilang cukup besar sehingga penghapusan akan menghilangkan terlalu banyak data berharga, **atau** ketika kolom tersebut bersifat deskriptif/numerik yang nilainya secara wajar dapat diperkirakan dari nilai lain dalam kolom yang sama. Keputusan akhir tetap harus mempertimbangkan konteks domain data, bukan sekadar aturan mekanis. **Kriteria penilaian:** perbedaan konseptual `dropna` vs `fillna` dijelaskan dengan tepat (40%), kaidah praktis yang logis dan mempertimbangkan persentase serta kepentingan kolom (60%).

Esai 5 — Jawaban Model. Kedua persoalan tidak dapat ditangani dengan pendekatan yang sama karena keduanya memiliki akar penyebab yang berbeda secara fundamental: nilai kosong pada `Biaya_Perbaikan_Ribu_Rupiah` merupakan *missing value* murni (data yang memang belum tercatat), sedangkan nilai negatif pada `Usia_Mesin_Tahun` bukanlah nilai yang hilang, melainkan nilai yang **ada namun tidak logis** (kesalahan entri data), karena

usia mesin secara fisik tidak mungkin bernilai negatif. Untuk Biaya_Perbaikan_Ribu_Rupiah, penanganan yang tepat adalah imputasi menggunakan median (karena kemungkinan terdapat outlier biaya perbaikan yang besar) atau, jika persentasenya kecil, penghapusan baris yang bersangkutan. Untuk Usia_Mesin_Tahun, penanganan yang tepat adalah terlebih dahulu memeriksa apakah nilai negatif tersebut kemungkinan besar merupakan kesalahan tanda (misalnya usia 5 tahun tercatat sebagai -5) yang dapat dikoreksi secara masuk akal dengan mengambil nilai absolutnya (`abs()`); nilai negatif sebaiknya dikoreksi dengan cara ini apabila polanya konsisten dengan kesalahan pencatatan sederhana, namun apabila magnitude nilai tersebut tidak masuk akal secara fisik meskipun sudah diabsolutkan (misalnya -500), maka baris tersebut sebaiknya dihapus karena kemungkinan besar mengandung kesalahan pencatatan yang lebih kompleks yang tidak dapat dipercaya begitu saja. **Kriteria penilaian:** penjelasan perbedaan akar masalah missing value vs nilai tidak logis (40%), strategi penanganan yang berbeda dan tepat untuk masing-masing kolom (40%), kriteria kapan mengoreksi vs menghapus dijelaskan dengan masuk akal (20%).

Esai 6 — Jawaban Model. - *Histogram*: menjawab pertanyaan mengenai distribusi frekuensi suatu variabel numerik tunggal — apakah data terkonsentrasi pada rentang tertentu, apakah distribusinya simetris atau condong (skewed). Contoh manufaktur: melihat distribusi Downtime (menit) seluruh mesin untuk mengetahui apakah mayoritas downtime berdurasi singkat dengan sedikit kejadian downtime sangat panjang. - *Boxplot*: menjawab pertanyaan mengenai perbandingan sebaran dan identifikasi outlier suatu variabel numerik, baik secara tunggal maupun dikelompokkan berdasarkan kategori. Contoh manufaktur: membandingkan sebaran Jumlah_Cacat antar-shift kerja (Pagi, Siang, Malam) untuk mengetahui shift mana yang memiliki variabilitas cacat produk tertinggi. - *Scatter plot*: menjawab pertanyaan mengenai hubungan atau pola antara dua variabel numerik. Contoh manufaktur: mengeksplorasi apakah ada hubungan antara Suhu_Mesin dan Jumlah_Cacat untuk menduga apakah suhu berlebih berkontribusi pada cacat produksi. - *Heatmap korelasi*: menjawab pertanyaan mengenai kekuatan hubungan linear antar-banyak pasangan variabel numerik sekaligus dalam satu visualisasi. Contoh manufaktur: memindai puluhan parameter sensor mesin (suhu, tekanan, getaran, kecepatan) untuk dengan cepat mengidentifikasi pasangan parameter yang saling berkorelasi kuat sebelum membangun model prediktif.

Kriteria penilaian: keempat jenis visualisasi dijelaskan dengan pertanyaan analitis yang tepat (50%) dan contoh manufaktur yang relevan dan spesifik (50%).

Esai 7 — Jawaban Model. (a) $IQR = Q3 - Q1 = 45 - 15 = 30$. Batas bawah = $Q1 - 1,5 \times IQR = 15 - (1,5 \times 30) = 15 - 45 = -30$. Batas atas = $Q3 + 1,5 \times IQR = 45 + (1,5 \times 30) = 45 + 45 = 90$. (b) Nilai 200 menit berada jauh di atas batas atas (90), sehingga nilai tersebut **tergolong outlier** menurut aturan IQR. Implikasi praktisnya bagi manajer perawatan mesin adalah kejadian downtime sepanjang 200 menit tersebut merupakan kejadian yang secara statistik tidak biasa dan patut diselidiki lebih lanjut secara khusus — kemungkinan mengindikasikan kerusakan yang jauh lebih serius dibandingkan downtime rutin, kesalahan prosedural yang tidak biasa, atau bahkan kesalahan pencatatan waktu; manajer perlu menelusuri catatan detail kejadian tersebut untuk menentukan apakah diperlukan tindakan perbaikan preventif khusus agar kejadian serupa tidak terulang. **Kriteria**

penilaian: perhitungan IQR, batas bawah, dan batas atas yang benar dengan langkah ditunjukkan (50%), identifikasi status outlier yang tepat (20%), interpretasi implikasi praktis yang relevan bagi konteks perawatan mesin (30%).

Esai 8 — Jawaban Model. Prinsip “korelasi tidak sama dengan kausalitas” penting dipegang karena mengambil kesimpulan kausal (sebab-akibat) dari sekadar korelasi statistik dapat menyesatkan pengambilan keputusan — tindakan perbaikan yang diambil berdasarkan asumsi kausalitas yang keliru dapat gagal menyelesaikan masalah yang sesungguhnya, bahkan membuang sumber daya secara sia-sia. Contoh hipotetis dalam konteks manufaktur: misalkan *heatmap* korelasi menunjukkan korelasi positif tinggi antara Jumlah_Produksi (jumlah unit yang diproduksi per shift) dan Jumlah_Cacat (jumlah unit cacat per shift). Seorang analis yang tergesa-gesa mungkin menyimpulkan bahwa memproduksi lebih banyak unit *menyebabkan* lebih banyak cacat, lalu merekomendasikan pengurangan target produksi. Namun kemungkinan besar terdapat variabel ketiga (*confounding variable*) yang menjadi penyebab sesungguhnya dari kedua variabel tersebut, misalnya shift Malam yang secara kebetulan memiliki baik Jumlah_Produksi maupun Jumlah_Cacat yang lebih tinggi karena faktor pengawasan yang lebih longgar atau kelelahan operator pada shift tersebut — dalam hal ini, Shift adalah *confounding variable* yang memengaruhi kedua variabel secara independen, dan solusi yang tepat bukanlah mengurangi produksi, melainkan memperbaiki kondisi kerja pada shift yang bersangkutan. **Kriteria penilaian:** penjelasan pentingnya prinsip dan risiko kesimpulan kausal yang keliru (40%), contoh hipotetis manufaktur yang koheren (30%), identifikasi *confounding variable* yang masuk akal (30%).

Esai 9 — Jawaban Model. Nilai -200 disebut *missing value* yang “tersamar” (*disguised missing value*) karena secara sintaksis ia terlihat seperti angka yang sah dan dapat diproses oleh pandas tanpa galat, namun secara semantik ia sesungguhnya bukan pengukuran yang valid, melainkan kode khusus yang digunakan sistem pencatatan untuk menandai kegagalan sensor — berbeda dari NaN yang secara eksplisit dikenali pandas sebagai nilai kosong. Bahaya utamanya adalah apabila nilai -200 tidak diubah menjadi NaN terlebih dahulu, maka `df.describe()` akan menghitung statistik (mean, std, min, dan sebagainya) dengan menyertakan nilai -200 sebagai pengukuran yang sah, sehingga nilai rata-rata akan tertarik jauh ke bawah secara tidak realistis dan statistik sebaran menjadi sangat menyesatkan; demikian pula visualisasi seperti histogram atau *scatter plot* akan menampilkan titik-titik pada nilai -200 yang mendominasi tampilan grafik dan mengaburkan pola sesungguhnya dari data yang valid. Baris kode yang tepat untuk mengatasi hal ini adalah: `df.replace(-200, np.nan, inplace=True)`. **Kriteria penilaian:** penjelasan konsep *disguised missing value* (30%), penjelasan dampak terhadap `describe()` dan visualisasi (40%), kode yang tepat (30%).

Esai 10 — Jawaban Model. Contoh paragraf ringkasan naratif yang sesuai: “Berdasarkan hasil eksplorasi data produksi pabrik selama periode yang dianalisis, ditemukan bahwa shift Malam secara konsisten menghasilkan rata-rata jumlah produk cacat yang jauh lebih tinggi dibandingkan shift Pagi dan Siang, sebuah temuan yang perlu mendapat perhatian khusus dari manajemen. Selain itu, ditemukan pula kecenderungan bahwa suhu mesin yang lebih tinggi berasosiasi dengan jumlah cacat yang lebih banyak, meskipun hubungan ini bersifat sedang dan belum tentu menunjukkan hubungan sebab-akibat langsung

sehingga memerlukan investigasi lebih lanjut. Sebagian kecil data kelembaban ruangan yang sebelumnya kosong telah dilengkapi menggunakan nilai tengah (median) dari data yang tersedia, sehingga analisis ini tetap dapat dilakukan secara menyeluruh tanpa kehilangan banyak data berharga. Sebagai tindak lanjut, kami merekomendasikan agar manajemen melakukan investigasi lebih lanjut terhadap kondisi kerja dan pengawasan pada shift Malam, termasuk memeriksa apakah terdapat masalah pada pengaturan suhu mesin selama shift tersebut, karena perbaikan pada titik ini berpotensi menurunkan tingkat kecacatan produk secara signifikan. Kami juga menyarankan agar pemantauan suhu mesin dilakukan lebih ketat sebagai langkah pencegahan dini terhadap risiko peningkatan cacat produksi.” **Kriteria penilaian:** paragraf memuat minimal lima kalimat naratif yang mengalir (20%), merangkum ketiga temuan yang diberikan pada soal secara akurat (40%), bahasa dapat dipahami oleh pembaca non-teknis tanpa istilah statistik yang tidak dijelaskan (20%), rekomendasi tindak lanjut yang konkret dan dapat ditindaklanjuti (20%).

UMPAN BALIK DAN TINDAK LANJUT

Setelah mengerjakan seluruh soal pilihan ganda dan esai pada Tes Formatif serta mencocokkannya dengan kunci jawaban dan pembahasan pada bagian sebelumnya, mahasiswa perlu mengukur secara objektif sejauh mana penguasaannya terhadap materi bab ini sebelum memutuskan untuk melanjutkan ke Pertemuan 2 atau perlu mengulang bagian-bagian tertentu terlebih dahulu. Bagian ini menyediakan rumus penghitungan skor beserta kriteria dan rekomendasi tindak lanjut yang terstruktur.

Rumus Penghitungan Skor

Skor akhir Tes Formatif dihitung dengan menggabungkan capaian pada bagian pilihan ganda (bobot 70%) dan bagian esai (bobot 30%), sesuai proporsi yang mencerminkan bahwa soal pilihan ganda mengukur keluasan pemahaman konsep sementara soal esai mengukur kedalaman penalaran analitis. Rumus yang digunakan adalah sebagai berikut.

$$\text{Skor Akhir} = (\text{Jumlah Jawaban Benar PG} / 40 \times 70) + (\text{Skor Total Esai} / 10 \times 30)$$

Keterangan:

- **Jumlah Jawaban Benar PG** adalah banyaknya soal pilihan ganda (dari total 40 soal) yang dijawab tepat sesuai kunci jawaban pada Bagian C.
- **Skor Total Esai** adalah jumlah skor mentah dari kesepuluh soal esai, dengan setiap soal esai diberi skor pada rentang 0 hingga 1 berdasarkan tingkat kesesuaian jawaban mahasiswa terhadap kriteria penilaian yang tercantum pada setiap jawaban model (Bagian D), sehingga skor total esai maksimum adalah 10.

Contoh perhitungan. Seorang mahasiswa menjawab benar 34 dari 40 soal pilihan ganda, dan memperoleh skor total esai 7,5 dari maksimum 10 setelah dinilai menggunakan kriteria pada jawaban model. Maka skor akhirnya dihitung sebagai berikut:

$$\begin{aligned}\text{Skor Akhir} &= (34 / 40 \times 70) + (7,5 / 10 \times 30) \\ &= (0,85 \times 70) + (0,75 \times 30) \\ &= 59,5 + 22,5 \\ &= 82,0\end{aligned}$$

Mahasiswa pada contoh ini memperoleh skor akhir 82,0, yang berdasarkan kriteria pada bagian berikutnya dinyatakan telah mencapai ambang kelulusan dan dapat melanjutkan ke Pertemuan 2.

Kriteria Ketuntasan dan Rekomendasi Tindak Lanjut

Ambang batas ketuntasan minimal (*passing grade*) untuk Tes Formatif bab ini ditetapkan sebesar 80%, sejalan dengan standar penguasaan kompetensi praktik yang dituntut pada mata kuliah ini. Berdasarkan skor akhir yang diperoleh, mahasiswa dapat menentukan langkah selanjutnya sesuai tabel berikut.

Rentang Skor Akhir	Kategori Penguasaan	Rekomendasi Tindak Lanjut
80 – 100	Tuntas (Kompeten)	Mahasiswa dinyatakan telah menguasai seluruh Sub-CPMK 1.1 hingga 1.5 dan dapat melanjutkan ke materi Pertemuan 2 (Data Preprocessing dan EDA Lanjut) tanpa perlu mengulang bagian mana pun dari bab ini.
60 – 79	Belum Tuntas (Perlu Perbaikan Sebagian)	Mahasiswa disarankan meninjau ulang secara khusus sub-bab yang berkaitan dengan soal-soal yang dijawab keliru, mengerjakan ulang Latihan Mandiri Terbimbing pada bagian yang relevan, kemudian mengerjakan ulang soal-soal yang sebelumnya salah sebelum melanjutkan ke Pertemuan 2.
Di bawah 60	Belum Tuntas (Perlu Pengulangan Menyeluruh)	Mahasiswa disarankan mempelajari ulang keseluruhan bagian Penyajian Materi bab ini dari awal, mengerjakan kembali seluruh Latihan Mandiri Terbimbing langkah demi langkah, mengerjakan salah satu Studi Kasus Mandiri sebagai latihan tambahan, kemudian mengulang Tes Formatif secara utuh sebelum diizinkan melanjutkan ke Pertemuan 2.

Panduan Remediasi Berdasarkan Pola Kesalahan

Selain skor akhir secara keseluruhan, mahasiswa juga disarankan memperhatikan pola soal mana yang paling sering dijawab keliru, karena pola tersebut dapat mengindikasikan sub-bab tertentu yang belum dikuasai dengan baik. Panduan berikut dapat digunakan untuk menargetkan bagian mana yang perlu dipelajari ulang secara spesifik.

- Apabila kesalahan banyak terjadi pada soal nomor 1–13 (ekosistem perangkat lunak, lingkungan kerja, struktur data dasar), mahasiswa disarankan mempelajari ulang sub-bab Materi A mengenai pengenalan ekosistem Python dan lingkungan kerja.
- Apabila kesalahan banyak terjadi pada soal nomor 14–20 (inspeksi data, statistik deskriptif), mahasiswa disarankan mempelajari ulang sub-bab Materi C bagian awal mengenai `head()`, `info()`, `dtypes`, dan `describe()`.
- Apabila kesalahan banyak terjadi pada soal nomor 21–26 (missing value), mahasiswa disarankan mempelajari ulang sub-bab Materi C bagian akhir mengenai `isnull()`, `dropna()`, dan `fillna()`, serta mengulang kembali Langkah 5–7 pada Latihan Mandiri Terbimbing.
- Apabila kesalahan banyak terjadi pada soal nomor 27–35 (visualisasi), mahasiswa disarankan mempelajari ulang sub-bab Materi D mengenai histogram, boxplot, scatter plot, dan heatmap korelasi, serta mengulang Langkah 8–11 pada Latihan Mandiri Terbimbing.
- Apabila kesalahan banyak terjadi pada soal nomor 36–40 (sintaksis pemfilteran dan kasus khusus missing value tersamar), mahasiswa disarankan mempelajari ulang keseluruhan sub-bab Materi C dan D secara terintegrasi, serta membaca kembali

catatan penting pada Studi Kasus Kelompok 1 dan 2 mengenai penanganan data yang tidak biasa.

Skor pada Tes Formatif ini bersifat formatif, bukan sumatif — artinya tujuan utamanya adalah membantu mahasiswa dan dosen mengidentifikasi kesenjangan pemahaman sedini mungkin, bukan sekadar menentukan nilai akhir. Dosen pengampu dapat menggunakan hasil rekapitulasi skor seluruh kelas untuk mengidentifikasi topik mana yang perlu diulas kembali secara klasikal sebelum memulai Pertemuan 2, sementara mahasiswa secara individu dapat menggunakan panduan remediasi di atas untuk mengarahkan waktu belajar mandiri secara lebih efisien dan tepat sasaran.

DAFTAR PUSTAKA

Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems* (3rd ed.). O'Reilly Media.

McKinney, W. (2022). *Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.

VanderPlas, J. (2016). *Python data science handbook: Essential tools for working with data*. O'Reilly Media.

NumPy Developers. (2024). *NumPy documentation*. NumPy. <https://numpy.org/doc/stable/>

pandas Development Team. (2024). *pandas documentation*. pandas. <https://pandas.pydata.org/docs/>

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>

Matplotlib Development Team. (2024). *Matplotlib documentation*. Matplotlib. <https://matplotlib.org/stable/>

Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. <https://doi.org/10.21105/joss.03021>

Seaborn Development Team. (2024). *Seaborn: Statistical data visualization documentation*. Seaborn. <https://seaborn.pydata.org/>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

scikit-learn Developers. (2024). *scikit-learn: Machine learning in Python — Documentation*. scikit-learn. <https://scikit-learn.org/stable/documentation.html>

Project Jupyter. (2024). *Jupyter Notebook documentation*. Project Jupyter. <https://jupyter-notebook.readthedocs.io/>

Google Research. (2024). *Google Colaboratory: Frequently asked questions*. Google Colab. <https://research.google.com/colaboratory/faq.html>

Chen, S., & De Vito, S. (2016). *Air Quality [Dataset]*. UCI Machine Learning Repository. <https://doi.org/10.24432/C59K5F>

Chen, D. (2015). *Online Retail [Dataset]*. UCI Machine Learning Repository. <https://doi.org/10.24432/C5BW33>

Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.

Wickham, H., & Grolemund, G. (2017). *R for data science: Import, tidy, transform, visualize, and model data*. O'Reilly Media.

Kelleher, J. D., & Tierney, B. (2018). *Data science*. MIT Press.